

Техническое Задание

ЗАКАЗЧИК:

- ФИО: Yessen Amanay Bekezhanuly
- Страна: Kazakhstan
- Адрес: Astana City, 15, kv.97, ulitsa Aiteke Bi
- Email: support@flyprox.com

ИСПОЛНИТЕЛЬ:

- ФИО: Kudr8vceva Antonina Olegovna
- Страна: Russian Federation
- Адрес: ul. Dzerzhinskogo 53, Stavropol, Stavropol Krai, 355000, Russian Federation
- Email: atrkk2024@gmail.com

Техническое задание

Мобильное приложение для школьников, позволяющее анонимно участвовать в ежедневных опросах о своих одноклассниках. Анонимность голосования: Тяжело отследить кто за кого голосовал. Нельзя голосовать за себя.

Формат опросов:

- Ежедневно доступно 50 опросов
- В каждом — 12 вопросов (всего 600 вопросов в день при условии он не будет ждать таймер и активно звать друзей за день)
- Перерыв между опросами — 40 минут
- Вопросы разделены на три категории:
 - Юмор («Кто похож на Илона Маска?») 4 вопроса
 - Обычные («Кто самый умный в классе?») 4 вопроса
 - Симпатия («Кто тебе нравится?») 4 вопроса

Анонимность голосования:

- Нельзя отследить, кто за кого голосовал
- Один пользователь — один голос на вопрос
- Запрещено голосовать за себя (сам никогда у себя не появляешься)
- Голосование идет только в своей школе.

Вопросы:

- Вопросы создаёт администратор через панель
- Школьники могут только предлагать вопросы
- Окончательный выбор делает администратор

Предоставляемая инфраструктура

Репозиторий и серверы

- GitHub Organization с настроенным Docker Compose
- Express.js + Redis
- Управляемый PostgreSQL сервер
- **Mobile:** Expo (React Native)

Аккаунты для публикации

- App Store Connect (iOS)
- Google Play Console (Android)

Сервисы и аналитика

- Firebase Analytics — аналитика, краши, performance monitoring, push-уведомления (FCM)
- Amplitude — product analytics, Session Replay, воронки и поведение пользователей
- Google Analytics — веб-аналитика
- Cloudflare CDN для картинок
- Sentry — мониторинг ошибок

Интеграция аналитики

Все сервисы аналитики уже настроены. Вам нужно будет добавить SDK в мобильное приложение (несколько строк кода):

Docker compose: API, Worker, PostgreSQL, Redis, Meilisearch и тд

Монетизация (IAP)

- Подписки через Google Play и App Store
- Тестирование test flight
-

План поэтапного релиза

Этап 1: Базовая версия (v1.0)

Публикация: App Store и Google Play (бесплатно)

Разрешения: НЕТ

Цель: Пройти первичную модерацию, получить базу пользователей можно даже простое подать mvp

Этап 2: Контакты и уведомления (v1.1)

Публикация: Через 3 дня после Этапа 1

Добавляются разрешения:

- **iOS:** NSContactsUsageDescription, NSUserNotificationsUsageDescription
- **Android:** READ_CONTACTS, POST_NOTIFICATIONS

Обязательно:

- Privacy Policy с разделом про контакты
 - App Privacy заполнена в App Store Connect
 - Notes for Reviewer: "Контакты обрабатываются локально, не загружаются на сервер"
-

Этап 3: Фото (v1.2) и геолокация (возможно фото вообще не будет)

Публикация: Через 3 дня после Этапа 2

Добавляются разрешения: на геолокацию, но нужно грамотно объяснить в приложении почему мы берем геолокацию но при этом не палится что это школа.

- **iOS:** NSPhotoLibraryUsageDescription, NSCameraUsageDescription
 - **Android:** READ_MEDIA_IMAGES, CAMERA
-

Этап 4: Монетизация (v1.3)

Публикация: Через 3 дня после Этапа 3

Добавляется: IAP (подписки)

Разрешения: Без изменений

Важно: Каждый этап = отдельный релиз с полноценным review в сторх.

Тестирование кода (обязательно):

- Unit тесты (критичная бизнес-логика)
- E2E тесты (регистрация, опросы, результаты)
- K6 нагрузочное тестирование (1000+ пользователей)
- Все тесты проходят в CI/CD

TestFlight бета-тестирование:

****Этап 0. 5: TestFlight Internal Testing****

- Исполнитель загружает сборку в TestFlight
- Internal Testing с 5-10 тестерами
- Исправление критичных багов (если найдены)

- Финальная сборка для production

Критерии приемки:

Работа считается выполненной после:

1. Все тесты (Unit, E2E, K6) проходят
2. TestFlight Internal Testing пройден (нет критичных багов)
3. Приложение опубликовано в App Store и Google Play

****Используется:**** Firebase Test Lab или аналог (BrowserStack, AWS Device Farm)

****Цель:****

Автоматический прогон E2E тестов на реальных iOS/Android устройствах перед публикацией в TestFlight/сторы.

Безопасность

- Шифрование данных — при передаче (HTTPS/TLS) и хранении
- Защита API — rate limiting, аутентификация
- Соответствие требованиям — GDPR, защита данных несовершеннолетних (COPPA)
- Валидация данных — защита от некорректного ввода

Поддерживаемые устройства

iOS

- Версии: от iOS 14.0 – до iOS 19.0+

Android

- Версии: от Android 9. 0 (API 28) – до Android 15. 0+ (API 35+)

Авторизация пользователей:

Passport.js (Express.js) или аналог — бесплатный open source и без лимитов на пользователей.

Тестирование

— Unit-тесты для всей критичной бизнес-логики

— E2E-тесты ключевых сценариев: регистрация, вход, прохождение опроса

— Автоматический запуск тестов в CI/CD

— Нагрузочное тестирование (k6): стабильная работа при 1000+ одновременных пользователей.

Обязательно покрывает без падений, ошибок 5xx и задержек свыше 2 секунд:

Оплата

Для удобства и прозрачности ориентируемся на следующие материалы:

- technical_specification.pdf
- design.pdf
- api.dfp

Критерии по завершению работы:

- Приложение опубликовано в App Store и Google Play
- Все основные тесты (E2E, Unit, K6) проходят корректно
- Функционал и дизайн соответствуют ТЗ
- Если какая-то часть не соответствует ТЗ или тесты не проходят, просто дорабатываем до нужного состояния — и после этого завершаем проект.

Оплата — через безопасную сделку на площадке фриланса, чтобы было комфортно и вам, и мне

Локализация

- Все тексты приложения выносятся в JSON файлы локализации
- ****Формат:**** один файл на язык (`ru.json`, `en. json`)
- ****Структура:**** вложенные объекты, логически сгруппированные по экранам/модулям
- Без `hardcode` текстов в коде (все тексты только из JSON)
- Все видимые тексты должны быть вынесены в JSON
- `ru.json` и `en. json` должны содержать идентичные ключи
- Ключи должны иметь понятные названия (не `text1`, `label2`)
- Файлы должны быть валидны (проходить JSON validator)

Исполнитель предоставляет готовый файл:

-  `ru.json` - все тексты на русском языке

Дальше наши переводчики уже переводят на другие языки.

Интернационализация (i18n):

Tier 1 (полная поддержка): - Английский - Русский

Требования: UI корректно адаптируется под текст, без обрезки и поломок layout.

Tier 2 (базовая поддержка, mvp): - Японский, Корейский, Испанский, Португальский, Тайский, Хинди
Требования: Техническое подключение через JSON, корректное отображение символов, UI функционален.
Детальная полировка не требуется. Общие требования: ✓ JSON-локализация для всех языков ✓
Переключение языка без перезагрузки ✓ Текст отображается без квадратиков □ ✓ Приложение не
крашится при смене языка.

Админ панель базовые вещи для добавления опросов и тд распишем позже

ТИПОГРАФИКА

ШРИФТЫ

iOS: SF Pro (system default)

Android: Roboto (system default)

РАЗМЕРЫ

H1: 28-32pt/sp Bold

H2: 20-24pt/sp Semibold

H3: 18-20pt/sp Semibold

Body: 16pt/sp Regular

Caption: 14pt/sp Regular

Small: 12pt/sp Regular

Line-height: 1.4-1.5

ТРЕБОВАНИЯ

- iOS: использовать pt (points) для поддержки Dynamic Type
- Android: использовать sp (scale-independent pixels) для Font Scaling
- React Native, Flutter автоматически делают и для Андроид и для IOS

ВАЖНО: писать комиты в нашем гит хаб организейшн дадим доступ

SMS-авторизация для iOS/Android приложений

SuperTokens (self-hosted, Apache 2.0), authentic (желательно super tokens))или любой open source аналог без скрытых лицензий с коммерческой лицензией.

Почему не Firebase/Supabase (их нельзя)

- Firebase при 1M юзеров = \$25,000/мес
- SuperTokens при 1M юзеров = \$200/мес (только сервер)

Стек

- Backend: Node.js + Express + SuperTokens
- БД: PostgreSQL
- SMS: Twilio API
- Деплой: Docker на VPS

Результат

-  Unlimited пользователей
-  Платим только за сервер + SMS
-  Никаких скрытых лицензий

Документацию сделать:

- **README** — описание проекта, назначение, базовый запуск
- **API-документация** — список эндпоинтов, методы, параметры, ответы
- **Деплой и серверы** — требования, настройка окружения, запуск, обновление

Медиа-файлы (требования):

- Иконки — размер файла **до 50 КБ**
- Изображения — размер файла **до 250 КБ**
- Все файлы оптимизированы без заметной потери качества

Звуки и вибрация:

- Использовать по усмотрению разработчика, инициатива приветствуется

- Предоставляется 3 трека заказчиком
 - Треки проигрываются по очереди (по кругу)
- Приветственное видео/картинка заставка при открытии моб приложения предоставляется заказчиком

Дизайн:

- Если видите, что можно улучшить дизайн (кнопки, отступы, цвета и т.д.) — смело улучшайте
- Допускаются изменения на ваше усмотрение, если это улучшает общий вид и UX
- Например: если фон синий, но лучше смотрится белый — можно заменить
- Инициатива приветствуется

Реферальная система промокодов: **nanoid** (лицензия MIT) или **Gravity Referral (Node.js)**. Это обеспечит полную автономность проекта, отсутствие платных подписок и безопасность данных без использования сторонних SDK. Так как open source

REDIS ОПТИМИЗАЦИЯ

1. ГОЛОСОВАНИЯ (4 ученика) - 2 часа

Задача: Кэшировать список учеников школы/класса

Redis ключ: `students:school_{id}:grade_{n}:gender_{filter}`

TTL: 5 минут

Логика: Проверил кэш → нет? → запрос БД → сохранил → выбрал случайные 4

2. ГОЛОСОВАНИЯ (ТАЙМЕР) - 4 часа

Задача: Хранить прогресс пользователя в Redis, таймер считать на клиенте

Redis ключ: `progress:user_{id}`

Что хранить: `{last_round_time, rounds_today, current_question, date}`

TTL: До конца дня

Логика: Клиент запрашивает статус 1 раз → сам считает 40 минут локально → сервер проверяет только при старте раунда

Результат: 50М проверок → 2-3М запросов

3. ACTIVITY FEED - 1 день

Задача: 3 уровня кэша (телефон 30 мин → Redis 30 мин → БД)

Redis ключ: `activity:school_{id}:grade_{n}`

Что хранить: JSON массив 1000 последних голосов

TTL: 30 минут

Логика: Телефон проверил локальный кэш → нет? → сервер проверил Redis → нет? → тяжёлый SQL с JOIN → сохранил в Redis и отдал клиенту

Важно: Показываем голоса 3А учеников моего класса (не от моего класса)

Результат: Запрос 5-10 сек → 50мс

4. ДРУЗЬЯ ДРУЗЕЙ - 2-3 дня

Задача: Кэшировать список "друзья моих друзей"

Redis ключ: `recommendations:fof:user_{id}`

Что хранить: JSON 200 пользователей с рейтингом общих друзей

TTL: 1 час

Логика простая (6/10): Первый раз тяжёлый SQL 5 сек → сохранил в кэш → дальше быстро

Логика продвинутая (7/10): При добавлении друга → задача в Bull MQ → worker пересчитал фоном → пользователь видит сразу готовые данные

Результат: Запрос 5 сек → 10мс

5. ОДНА ШКОЛА - 4 часа

Задача: Кэшировать одноклассников с рейтингом общих друзей

Redis ключ: `recommendations:school:user_{id}`

Что хранить: JSON 200 учеников, сортировка по общим друзьям DESC

TTL: 30 минут

Логика: SQL с подзапросом COUNT(общих друзей) → сортировка → LIMIT 200 → кэш

Результат: Запрос 2-3 сек → 10мс

6. PUSH-УВЕДОМЛЕНИЯ - 1-2 дня

Задача: Отправлять через очередь Bull MQ (не синхронно!)

Очередь: `push-notifications`

Логика: API получил голос → добавил задачу в очередь → сразу ответил "OK" → worker отправил push в фоне

Настройки: Retry 3 раза, приоритет HIGH, 5 workers, rate limit 100/сек

Результат: API 300мс → 50мс, push приходит через 1-2 сек (норм)

7. КОНТАКТЫ SYNC - 1 день

Задача: Хешировать номера телефонов на клиенте (SHA-256)

Логика: Клиент читает контакты → нормализует номера → хеширует → отправляет массив хешей → сервер ищет совпадения

Важно: Сервер НЕ видит реальные номера (GDPR)

Ограничение: Не чаще 1 раза в день

Результат: Безопасно + быстро

1. Unit-тесты (модульное тестирование)

Минимум 70% покрытия критичной бизнес-логики через Jest/pytest. Обязательно покрыть: валидацию данных, подсчет голосов, начисление звездочек, алгоритм "Лепесток", систему друзей, лимиты и таймеры, работу с Premium подписками. Все тесты должны проходить без ошибок в CI/CD.

2. E2E тесты (end-to-end)

Покрытие критических пользовательских сценариев через Detox/Appium на iOS и Android эмуляторах. Обязательные сценарии: регистрация и онбординг, прохождение голосования с таймером 40 минут, социальные функции (поиск, друзья), просмотр результатов и раскрытие имен, тестовая покупка Premium подписки. Все сценарии должны выполняться без сбоев.

3. Нагрузочное тестирование (K6)

Проверка стабильности при 1000+ одновременных пользователей через K6. Критичные эндпоинты: отправка голоса, поиск пользователей, загрузка активности, авторизация. Требования: время ответа < 2 сек (95 перцентиль), 0% ошибок 5xx, > 99% успешных запросов, среднее время ответа < 1 секунды.

ИНДЕКСЫ PostgreSQL

-- 1. Голосования - выборка учеников

```
CREATE INDEX idx_users_voting
```

```
ON users(school_id, grade, gender)
```

```
INCLUDE (id, name, avatar, class)
```

```
WHERE deleted_at IS NULL;
```

-- 2. Activity feed - голоса за класс

```
CREATE INDEX idx_votes_activity
```

```
ON votes(winner_school_id, winner_grade, created_at DESC)
```

```
INCLUDE (winner_id, voter_grade, voter_gender, poll_id);
```

-- 3. Лайки пользователя

```
CREATE INDEX idx_votes_winner
```



```
ON votes(winner_id, created_at DESC)
```

```
INCLUDE (poll_id, voter_id);
```

```
-- 4. Друзья - основной индекс
```

```
CREATE INDEX idx_friends_user
```

```
ON friends(user_id, status)
```

```
INCLUDE (friend_id, created_at)
```

```
WHERE status = 'accepted';
```

```
-- 5. Друзья - обратный индекс (для рекомендаций)
```

```
CREATE INDEX idx_friends_friend
```

```
ON friends(friend_id, status)
```

```
INCLUDE (user_id)
```

```
WHERE status = 'accepted';
```

```
-- 6. Запросы в друзья - входящие
```

```
CREATE INDEX idx_friends_pending
```

```
ON friends(receiver_id, status, created_at DESC)
```

```
INCLUDE (sender_id)
```

```
WHERE status = 'pending';
```

```
-- 7. Поиск по школе
```

```
CREATE INDEX idx_users_school
```

```
ON users(school_id, grade)
```

```
INCLUDE (id, name, avatar, class)
```

```
WHERE deleted_at IS NULL;
```

```
-- 8. Поиск по username (для search)
```

```
CREATE INDEX idx_users_username
```

```
ON users(username)
```

```
INCLUDE (id, name, avatar, school_id, grade)
```

```
WHERE deleted_at IS NULL;
```

```
-- 9. Контакты - хеши телефонов
```

```
CREATE INDEX idx_contact_hash
```

```
ON contact_hashes(phone_hash)
```

```
INCLUDE (user_id);
```

-- 10. Premium подписки

```
CREATE INDEX idx_subscriptions_user  
  
ON subscriptions(user_id, status, expires_at)  
  
WHERE status = 'active';
```

-- 11. Опросы - активные по школе

```
CREATE INDEX idx_polls_active  
  
ON polls(school_id, status, created_at DESC)  
  
WHERE status = 'active';
```

Админ панель:

1. Вопросы

- Добавить вопрос и возможность добавлять xlx файл в вопросы или как то сразу много вопросов через вставить текст.
- Удалить вопрос
- Выбрать категорию
- Список вопросов

2. Города

- Добавить город
- Удалить город
- Список городов
- Редактировать название города
- Назначить регион/область

3. Школы

- Добавить школу и привязать к городу
- Удалить школу
- Список школ (с фильтром по городу)
- Количество участников в школе

4. Пользователи

- Поиск (по имени / email / username)
- Посмотреть профиль
- Редактировать профиль (для изменения имени, школы)
- Заблокировать
- Удалить

5. Дашборд

- Топ-10 школ
- Топ-10 городов
- Общее количество

6 Промокоды

- Всего рефералов: количество приглашенных по промокодам
- Конверсия рефералов: сколько процентов пришло имеено с рефералок по промокодам
- Кто пригласил (номер) | Кого пригласил (номер) | Дата

Push-уведомления для приложения

1. Голосование и активность

1.1 Новый голос за меня

Когда: Кто-то выбрал меня в опросе

Условие: по умолчанию все уведомления включены

Примеры:

- "За тебя проголосовали! 🗳️ Открой приложение и узнай подробности"
 - "Кто-то выбрал тебя в опросе! 💙 Смотри в разделе Лайки"
 - "Ты получил новый голос! ⭐ Интересно, кто это был?"
-

1.2 Таймер истёк (новые опросы готовы)

Когда: Прошло 40 минут после завершения раунда

Условие: Всегда (критичное уведомление)

Примеры:

- "Новые 12 вопросов готовы! 🎯 Начинай голосовать"
 - "Твои опросы обновились! ⌚ 12 новых вопросов ждут тебя"
 - "Время истекло! 🔥 Проходи новый раунд голосований"
-

1.3 Друг установил приложение (обход таймера)

Когда: Приглашённый друг зарегистрировался по моему промокоду

Примеры:

- "Твой друг установил приложение! 🎉 Можешь голосовать прямо сейчас"
- "Ура! Твой друг присоединился 🧑‍🤝‍🧑 Таймер сброшен - голосуй!"
- "Друг зарегистрировался! 🎁 Бонус: пропусти ожидание"

2.1 Новый запрос в друзья

Когда: Кто-то отправил запрос в друзья

Примеры:

- "Иван Петров хочет добавить тебя в друзья 🧑🧑"
- "У тебя новый запрос в друзья от Маши! 😊"
- "Александр отправил запрос в друзья 🤝 Примешь?"

2.2 Запрос принят

Когда: Мой запрос в друзья приняли

Примеры:

- "Мария приняла твой запрос в друзья! 🎉"
- "Теперь вы друзья с Петей! 🧑🧑 Поздравляем"
- "Катя добавила тебя в друзья! ✅ Открой профиль"

3. Подписка Premium

3.1 Подписка скоро истекает

Когда: До окончания подписки осталось 3 дня

Условие: Настройка "Подписка Premium" включена

Примеры:

- "Твоя подписка Premium истекает через 3 дня 🕒"
 - "Не забудь продлить Premium! ⌚ Осталось 3 дня"
 - "Твой Premium заканчивается ⚡ Продли сейчас"
-

3.2 Подписка продлена

Когда: Автоматическое продление подписки успешно

Условие: Настройка "Подписка Premium" включена

Примеры:

- "Premium успешно продлён! 🎉 Спасибо за поддержку"
 - "Твоя подписка обновлена! 🙌 Продолжай пользоваться"
 - "Подписка продлена автоматически ✅ Всё работает"
-

5.2.1 День 1 (24 часа) - Мягкое напоминание

Когда: Пользователь не открывал приложение 24 часа
Тон: Дружелюбный, без давления

Примеры:

- "Эй! 🙌 Твои друзья голосуют, а ты пропускаешь веселье"
- "За тебя проголосовали вчера! 💙 Узнай кто это был"
- "12 новых вопросов ждут тебя! 🎯 Быстрый раунд?"
- "Ты пропустил 15 голосов! 😞 Кто-то активно выбирал тебя"
-

Логика: Мягко напомнить, может просто забыл

5.2.2 День 3 - Социальное давление

Когда: Пользователь не открывал приложение 3 дня
Примеры:

- "Ты пропустил 15 голосов! 😞 Кто-то активно выбирал тебя"
- "Твои друзья спрашивают где ты 😊 Вернись в игру!"
- "За 3 дня ты мог получить 3000 звездочек ⭐ Упс..."

Логика: Показать что он теряет (не давая бонус!)

5.2.3 День 7 - Критическое напоминание

Когда: Пользователь не открывал приложение 7 дней
Примеры:

- "Неделя без тебя 😞 Твой класс скучает! Возвращайся"
- "Ты всё ещё в топе школы! 🏆 Но позиция падает..."
- "Последний шанс: твои лимиты сгорят через 2 часа ⏰"

Логика: Последняя попытка вернуть через эмоции

6.1 Новый вход в аккаунт

Когда: Вход с нового устройства
Условие: Всегда (безопасность)

Примеры:

- "Вход в аккаунт с нового устройства 📱 Это ты?"
 - "Замечена активность на новом телефоне 📱 Проверь"
 - "Новый вход в систему 🔒 Если не ты - смени пароль"
-

6.2 Подозрительная активность

Когда: Необычная активность (например, 50 запросов в минуту)
Условие: Всегда (безопасность)

Примеры:

- "Подозрительная активность в аккаунте 🚨 Проверь настройки"
 - "Обнаружено необычное поведение ⚠️ Смени пароль"
 - "Защити аккаунт! 🛡️ Была попытка взлома"
-

7. Административные уведомления

7.1 Модерация контента

Когда: Администратор заблокировал/удалил контент
Условие: Всегда (важное)

Примеры:

- "Твой профиль нарушает правила ⚠️ Проверь почту"
- "Контент удалён модератором 🚫 Читай правила"
- "Предупреждение: изменения в аккаунте 📧 Открой письмо"

7.2 Обновление правил

Когда: Администратор обновил Terms или Privacy Policy

Условие: Всегда (юридическое требование)

Примеры:

- "Обновлены правила приложения 📖 Ознакомься"
- "Новая политика конфиденциальности 📄 Прочитай"
- "Важные изменения в условиях 📋 Требуется согласие"

Пуш оставьте отзыв после того как его первый раз выбрали в голосовании. Больше не надо

Панель управления (для администратора)

Ручная отправка уведомлений:

1. **Целевая аудитория:**
 - Все пользователи
 - Только Premium
 - Только без Premium
 - Конкретная школа
 - Конкретный класс
 - Неактивные (7+ дней)
 2. **Шаблоны:**
 - Скидка на подписку
 - Кастомное сообщение
 3. **Планирование:**
 - Отправить сейчас
 - Запланировать на дату/время
 - Повторять (ежедневно/еженедельно)
-

Итого: 15 типов автоматических + ручная отправка

Критичные (нельзя отключить):

- Таймер истёк
- Новый вход в аккаунт
- Подозрительная активность

Остальные: можно отключить в настройках у пользователя

НАСТРОЙКА MEILISEARCH (для всех экранов)

ОБЩИЕ ПРАВИЛА: В любом поиске использовать Meilisearch

1. Debounce везде: 300мс

Это золотая середина (не слишком быстро, не слишком медленно)

2. Минимальная длина:

- Пользователи: 2 символа
- Города: 2 символа (или фильтр локально)
- Школы: 3 символа

3. Typo tolerance (опечатки):

- 1 опечатка на 5 букв
- "Иван" найдёт "Ифан"
- "Москва" найдёт "Маскв"

Экраны Регистрации

number_registration_screen

Валидация:

- Код: 6 цифр, обязательное поле
- Автоотправка при вводе 6-й цифры

Ошибки:

- Неверный код → "Неверный код. Попробуйте ещё раз" (красный текст под полем)
- Код истёк → "Код истёк. Запросите новый"
- Слишком много попыток → "Слишком много попыток. Попробуйте через 1 минуту"

Состояния:

 Telegram

Код отправлен в Telegram

Введите 6-значный код
из сообщения от @HidavoBot

[123456]

[Открыть Telegram] ← Deep Link: tg://resolve?domain=OisterBot

Не приходит код?

WhatsApp доступен через 0:28 

[Получить код в WhatsApp] (disabled)

Таймер:

- Обратный отсчёт: 30 → 0 секунд
- Формат: "0:28", "0:27", .., "0:01", "0:00"

Через 30 секунд:

- Кнопка "Получить код в WhatsApp" становится активной (зелёная)

Кнопка "Открыть Telegram":

- Открывает Telegram через deep link
- URL: tg://resolve?domain=OisterBot&start=auth_{sessionId}
- Прокручивает к последнему сообщению (с кодом)

Сообщение в Telegram от @OisterBot:

Code

 Код для входа в Hidavo

123456

Введите этот код в приложении.
Никому не сообщайте код!

Код действителен 5 минут.

Не запрашивали код? Пройгнорируйте это сообщение.

WhatsApp Channel:

Default (если Telegram нет ИЛИ нажал "Получить код в WhatsApp"):

 WhatsApp

Код отправлен в WhatsApp

Введите 6-значный код
из сообщения в WhatsApp

[123456] (рандомный 6 значный код)

Не приходит код?
SMS доступно через 0:28 🕒

[Получить SMS код] (disabled)

Через 30 секунд:

- Кнопка "Получить SMS код" становится активной (синяя)
-

SMS Channel:

Default (если WhatsApp нет ИЛИ нажал "Получить SMS код"):

Code

📱 SMS

Код отправлен по SMS
на +7 701 234 56 78

Введите 6-значный код

[123456] (рандомный 6 значный код)

Не приходит код?
[Отправить повторно] (доступна через 0:60)

Через 60 секунд:

- Кнопка "Отправить повторно" становится активной
-

Code Input:

Typing:

- Автофокус на первое поле
- После ввода цифры → фокус на следующее поле
- Backspace → фокус на предыдущее поле
- После ввода 6-й цифры → автоматическая отправка на проверку

Verifying:

- Loading spinner
- Все поля disabled
- Запрос: POST /api/auth/verify-code

Success:

- ✅ Код верный
- Переход на следующий экран (Home / Profile_Completion)

Error:

- Красный текст под полями: "Неверный код"
 - Поля очищаются
 - Можно ввести снова
-

ЛОГИКА КАСКАДНОЙ ОТПРАВКИ:

Приоритет 1: Telegram (БЕСПЛАТНО) 📠

Code

1. Пользователь ввёл номер → "Получить код"
2. Сервер проверяет: есть ли Telegram аккаунт?
 - Проверка по базе (telegramUserId для этого номера)
3. Если Telegram ✅:
 - Генерирует код: 123456
 - Отправляет через Telegram Bot API (БЕСПЛАТНО)

- Response: { channel: "telegram", waitTime: 30, nextChannel: "whatsapp" }
4. На экране: Verification_Code_Screen (Telegram)
 5. Пользователь:
 - Нажимает "Открыть Telegram" (опционально)
 - Копирует код из Telegram
 - Вводит в поле
 6. ☒ Вход (стоимость: \$0.00)

Если НЕ ввёл код за 30 секунд:

7. Кнопка "Получить код в WhatsApp" → активна
8. Пользователь нажимает → переход на WhatsApp

Приоритет 2: WhatsApp (ДЕШЕВЛЕ SMS)

Code

1. Если Telegram ☒ ИЛИ пользователь нажал "Получить код в WhatsApp"
2. Сервер проверяет: есть ли WhatsApp?
 - Проверка через WhatsApp Business API
3. Если WhatsApp ☒
 - Генерирует код: 654321
 - Отправляет через WhatsApp API (~\$0.005-0.01)
 - Response: { channel: "whatsapp", waitTime: 30, nextChannel: "sms" }
4. На экране: Verification_Code_Screen (WhatsApp)
5. Пользователь вводит код из WhatsApp
6. ☒ Вход (стоимость: \$0.01)

Если НЕ ввёл код за 30 секунд:

Code

7. Кнопка "Получить SMS код" → активна
8. Пользователь нажимает → отправка SMS

Приоритет 3: SMS (ДОРОЖЕ)

Code

1. Если WhatsApp ☒ ИЛИ пользователь нажал "Получить SMS код"
2. Сервер отправляет SMS через Twilio (~\$0.08)
3. Response: { channel: "sms", waitTime: 0, nextChannel: null }
4. На экране: Verification_Code_Screen (SMS)
5. Пользователь вводит код
6. ☒ Вход (стоимость: \$0.08)

Важно:

- ⚠ Cost optimization: каскадная система Telegram → WhatsApp → SMS
- ⚠ Telegram Deep Link: быстрый доступ к коду через tg://resolve?domain=OisterBot
- ⚠ GDPR compliant: 6-значный код = двухфакторная аутентификация
- ⚠ 30 second wait: таймер между каналами для мотивации использовать текущий
- ⚠ Auto-send on complete: после ввода 6-й цифры автоматическая проверка
- ⚠ Code expires: код действителен 5 минут
- ⚠ Rate limiting: максимум 3 попытки ввода кода, потом блокировка на 1 минуту
- ⚠ Telegram pre-link: пользователь должен связать Telegram при первой регистрации
- ⚠ Session tracking: каждый запрос кода создаёт уникальную сессию

registration_screen

UI элементы:

- Input: Имя, Фамилия (required, auto-fill из Google/Apple если доступно)

ЫВ

- Radio: Пол (Мужской/Женский/Небинарный, required, default: Мужской)
- Custom Slider: Возраст (1-100, required, default: 16, строго 16-20 лет (включительно))
- Warning: "Приложение доступно только для детей в возрасте от 16 до 20 лет"
- Button: "Сохранить"

Валидация:

Имя/Фамилия: string, 2-50 символов, буквы/пробел/дефис, required

Пол: "male"|"female"|"non-binary", required

Возраст: integer, 16-20 лет строго (COPPA, GDPR, App Store), default 16, required

Ошибки:

- Пустое → "Введите [поле]"
- < 2 символов → "[Поле] слишком короткое"
- Возраст вне 16-20 лет → warning красный + кнопка disabled

Состояния:

Default: поля пустые/автозаполнены, кнопка активна, slider = 16

Validation Error: красные borders + текст ошибки под полем

Loading: кнопка disabled + spinner "Сохранение..."

Success: toast "✅ Профиль создан" → navigate дальше

Error: toast "❌ Ошибка сохранения" → кнопка активна

Важно:

⚠️ Возраст < 16 или больше 20 лет : slider красный, warning красный, кнопка disabled. Он обязан выбрать возраст от 16 до 20 лет(включительно)

⚠️ Button "Назад": confirm "Выйти? Данные не сохраняются"

⚠️ Валидация: client + server

ВАЖНО ОН НЕ МОЖЕТ ПРОДОЛЖИТЬ ПОКА НЕ ПОСТАВИТ ГАЛОЧКУ НА ПОТВЕРЖДАЮ ЧТО МНЕ 16 ЛЕТ

API:

POST /api/users/profile

Body: { firstName, lastName, gender, age }

location_screen

UI элементы:

- Back Button: Стрелка назад (← черная, top-left)
- Header Image: Иллюстрация городского пейзажа (синие и розовые здания, rounded corners)
- Title: "Выберите ваш город" (черный текст, крупный шрифт, bold)
- Search Input: Поле поиска с иконкой лупы, placeholder: "Астана" (rounded corners, white background, grey border)
- Radio List: Список городов с radio buttons
 - Radio Item 1: "Астана" + "Акмолинская область (регион)" (selected, blue radio button)
 - Radio Item 2: "Алматы" + "Акмолинская область (регион)" (unselected, grey radio button)
 - Radio Item 3: "Москва" + "Московский регион" (unselected, grey radio button)
- Link: "Нет вашей локации?" + "Написать в поддержку" (blue text, clickable, с иконкой вопроса)
- Button: "Сохранить" (primary, rounded, white text, blue background, centered)

Валидация:

- Город: required (обязательное поле, должен быть выбран один город из списка)
- Выбор города: radio button, только один вариант может быть выбран
- Американские города пишутся на латинице

Ошибки:

- Город не выбран → "Выберите город" (required validation)
- Поиск не дал результатов → "Город не найден. Попробуйте другое название"
- Ошибка загрузки списка → "Не удалось загрузить список городов. Попробуйте позже"

Состояния:

- Default: список городов загружен, поиск пустой (placeholder "Астана"), один город выбран (Астана), кнопка активна
- Empty: список городов загружен, но ничего не выбрано, кнопка disabled (required field empty)
- Search Active: пользователь вводит текст в поиск → список фильтруется в реальном времени
- Search No Results: поиск не дал результатов → показать текст "Город не найден" + ссылка "Нет вашей локации?"
- City Selected: город выбран (blue radio button), кнопка активна
- Loading: skeleton loader для списка городов при первой загрузке
- Success: toast "✅ Город сохранен" → navigate на следующий экран (например, school_selection_screen)
- Error: toast "❌ Не удалось сохранить город. Попробуйте снова" → кнопка активна

Важно:

- ⚠️ Required field: обязательно нужно выбрать город, кнопка "Сохранить" disabled пока ничего не выбрано
- ⚠️ Preloaded cities: список городов загружается заранее с сервера (GET /api/cities)
- ⚠️ Search: поиск работает локально (frontend filtering) по названию города
- ⚠️ "Нет вашей локации?": клик открывает email клиент с предзаполненным письмом на корпоративную почту (mailto:support@company.com)
- ⚠️ Email template:
 - Тема: "Добавьте мой город"
 - Тело: "Здравствуйте! Прошу добавить город: [город, который ввел пользователь]" ⚠️ Button "Назад": возврат на предыдущий экран (number_registration_screen)
 - ⚠️ Single selection: можно выбрать только один город (radio buttons, не checkboxes)
 - ⚠️ Region display: под каждым городом показывается регион/область
 - ⚠️ Auto-scroll: если список длинный, автоскролл к выбранному городу

GET /api/cities

POST /api/users/profile/location

school_screen

UI элементы:

- Back Button: Стрелка назад (← черная, top-left)
- Header Image: Иллюстрация абстрактных фигур (розовые, синие, коричневые элементы на темно-синем фоне, rounded corners)
- Title: "Выберите школу" (черный текст, крупный шрифт, bold, centered)
- Search Input: Поле поиска с иконкой лупы, placeholder: "Назарбаевская школа Астаны" (rounded corners, light grey background)
- Radio List: Список школ с radio buttons
 - Radio Item 1: "1 школа имени Сайфуллина" + "Москва, 78 участников" (selected, blue radio button)
 - Radio Item 2: "2 школа имени Грозного" + "Москва, 8 участников" (unselected, grey radio button)

- Radio Item 3: "3 школа имени Васильева" + "Москва, 25 участников" (unselected, grey radio button)
 - Radio Item 4: "4 школа имени Григоря" + "Москва, 13 участников" (unselected, grey radio button)
- Link: "Нет вашей школы?" + "Написать в поддержку" (grey text + clickable, с иконкой чата) (пишется письмо на нашу корпоративную почту)
- Button: "Сохранить" (primary, rounded, white text, blue background, centered)

Валидация:

- Школа: required (обязательное поле, должна быть выбрана одна школа из списка)
- Выбор школы: radio button, только один вариант может быть выбран

Ошибки:

- Школа не выбрана → "Выберите школу" (required validation)
- Поиск не дал результатов → "Школа не найдена. Попробуйте другое название или напишите в поддержку"
- Ошибка загрузки списка → "Не удалось загрузить список школ. Попробуйте позже"

Состояния:

- Default: список школ загружен для выбранного города, поиск пустой, одна школа выбрана (1 школа имени Сайфуллина), кнопка активна
- Empty: список школ загружен, но ничего не выбрано, кнопка disabled (required field empty)
- Search Active: пользователь вводит текст в поиск → список фильтруется в реальном времени по названию школы
- Search No Results: поиск не дал результатов → показать текст "Школа не найдена" + ссылка "Нет вашей школы?"
- School Selected: школа выбрана (blue radio button), кнопка активна
- Loading: skeleton loader для списка школ при первой загрузке или при смене города
- Success: toast "✅ Школа сохранена" → navigate на следующий экран (например, class_selection_screen)
- Error: toast "❌ Не удалось сохранить школу. Попробуйте снова" → кнопка активна

Важно:

В этом экране должна быть кнопка дать доступ к геолокации потому что он здесь только может искать школу, списка нет как такого он смотрит по геолокации и находит близлежащие школы в городе, то есть человек может выбрать то учебное заведение по поиску списка как такого нет. ЭТО приввела GDPR поэтому здес нужно тонко подойти к моменту

ВАЖНО Без геолокации если пользователь не выбрал то нужно сообщение что мы не можем продолжить без геолокации.

- ⚠️ Required field: обязательно нужно выбрать школу, кнопка "Сохранить" disabled пока ничего не выбрано
- ⚠️ City-based schools: список школ загружается на основе выбранного города на предыдущем экране (location_screen)
- ⚠️ Search: поиск работает через API (server-side filtering) с debounce 300ms
- ⚠️ Participants count: под каждой школой показывается количество участников (учеников в приложении)
- ⚠️ "Нет вашей школы?": клик открывает mailto с предзаполненным письмом
- ⚠️ Email/Support template:
 - Тема: "Добавьте мою школу"
 - Тело: "Здравствуйте! Прошу добавить школ"
 - ⚠️ Single selection: можно выбрать только одну школу (radio buttons, не checkboxes)
 - ⚠️ City display: город показывается под названием школы
 - ⚠️ Auto-scroll: если список длинный, автоскролл к выбранной школе
 - ⚠️ Empty state: если в городе нет школ → показать "В вашем городе пока нет школ" + кнопка "Написать в поддержку"

login_input_screen

UI элементы:

- Back Button: Стрелка назад (← белая, top-left, на синем фоне)
- Background: Сплошной синий фон (solid blue, #4A90E2 или похожий)
- Title: "Придумайте логин вашего аккаунта" (белый текст, крупный шрифт, bold, centered, вертикально по центру экрана)
- Input: Логин (required, placeholder: "Введите логин", white background, rounded corners, grey border)
- Button: "Сохранить" (secondary style, rounded, blue text, white background, centered)

Валидация:

- Логин: string, 3-20 символов, буквы латиницы, цифры, подчеркивание, точка, required
- Формат: lowercase, начинается с буквы, уникальный (проверка на сервере)
- Запрещенные символы: пробелы, спецсимволы (кроме . и _)

Ошибки:

- Пустое поле → "Введите логин" (required validation)
- Логин < 3 символов → "Логин должен содержать минимум 3 символа"
- Логин > 20 символов → "Логин должен содержать максимум 20 символов"
- Некорректный формат → "Логин может содержать только латинские буквы, цифры, . и _"
- Логин начинается с цифры → "Логин должен начинаться с буквы"
- Логин уже занят → "Этот логин уже занят. Попробуйте другой"
- Запрещенный логин → "Этот логин недоступен"

Состояния:

- Default: поле пустое (placeholder "Введите логин"), кнопка disabled (required field empty), синий фон
- Focus: белый border на поле ввода
- Typing: пользователь вводит текст, валидация в реальном времени (debounce 500ms для проверки уникальности)
- Valid: логин валиден и доступен, показать зеленую галочку справа от поля, кнопка активна
- Invalid: логин невалиден, красный border + текст ошибки под полем (белый или красный текст)
- Checking: проверка уникальности на сервере, показать spinner справа от поля
- Validation Error: красный border на поле + текст ошибки под полем
- Loading: кнопка disabled + spinner "Сохранение..." + поле disabled
- Success: toast "✅ Логин сохранен" → navigate на следующий экран (profile_completion_screen или dashboard)
- Error: toast "❌ Не удалось сохранить логин. Попробуйте снова" → кнопка активна

Важно:

- ⚠️ Required field: обязательно нужно ввести логин, кнопка "Сохранить" disabled пока поле пустое или невалидное
- ⚠️ Username uniqueness: проверка уникальности логина на сервере (GET /api/users/check-username)
- ⚠️ Real-time validation: валидация формата на client-side, проверка уникальности с debounce 500ms
- ⚠️ Case insensitive: логин сохраняется в lowercase, но проверка учитывает регистр
- ⚠️ Reserved usernames: admin, support, root и т.д. - запрещены
- ⚠️ Back Button: кнопка "Назад" → возврат на предыдущий экран (school_screen)
- ⚠️ Auto-lowercase: автоматически переводить ввод в lowercase
- ⚠️ Suggestions: показывать варианты доступных логинов, если введенный занят (опционально)
- ⚠️ Centered layout: элементы размещены вертикально по центру экрана

contacts_screen

UI элементы:

- Back Button: Стрелка назад (← белая, top-left, на синем фоне)
- Background: Сплошной синий фон (solid blue gradient)
- Title: "Найди друзей за 10 секунд и получай комплименты!" (белый текст, крупный шрифт, bold, centered)
- Subtitle: "Добавь контакты — мы автоматически найдем одноклассников из твоей школы" (белый текст, меньший размер, centered)
- Button: "Включить контакты" (primary, rounded, blue text, white background, centered, required action)
- Privacy Text Block:
 - Header: "Никаких звонков или спама" (белый текст, bold)
 - Description: "Мы уважаем приватность и никогда не свяжемся с вашими контактами." (белый текст, меньший размер)

Валидация:

- Разрешение на контакты: required (обязательное действие для продолжения регистрации)
- Кнопка "Включить контакты": единственный способ перейти дальше

Ошибки:

- Разрешение отклонено → показать alert "Для продолжения необходимо предоставить доступ к контактам" + кнопка "Попробовать снова"
- Ошибка доступа к контактам → "Не удалось получить доступ к контактам. Попробуйте позже"
- Ошибка отправки контактов → "Не удалось отправить контакты. Попробуйте снова"

Состояния:

- Default: экран с описанием, кнопка "Включить контакты" активна
- Permission Requested: системный диалог запроса разрешения на контакты (iOS/Android native)
- Permission Granted:
 - i. Считывает контакты с устройства
 - ii. Хеширует номера телефонов на фронтенде
 - iii. Отправляет хешированные контакты на сервер (POST /api/users/contacts/sync)
 - iv. Показывает spinner "Отправка..."
 - v. После успешной отправки → автоматический переход на следующий экран (без toast, без уведомлений)
- Permission Denied: alert "Для продолжения необходимо предоставить доступ к контактам" + кнопка "Попробовать снова" (нельзя пропустить)
- Loading: после получения разрешения показать spinner "Готово..." + кнопка disabled
- Success: автоматический переход на следующий экран
- Error: toast "❌ Не удалось отправить контакты. Попробуйте снова" → кнопка активна для повтора

Важно:

- ⚠️ Required permission: доступ к контактам обязателен, нельзя пропустить этот шаг
- ⚠️ No skip option: нет кнопки "Пропустить", пользователь должен дать разрешение
- ⚠️ Privacy first: явное указание на приватность и отсутствие спама
- ⚠️ System permission dialog: при нажатии "Включить контакты" → нативный диалог iOS/Android
- ⚠️ Back Button: возврат на предыдущий экран (login_input_screen)
- ⚠️ Backend hashing: хеширование номеров телефонов происходит на фронтенде (SHA-256 или другой алгоритм)
- ⚠️ Silent sync: после отправки контактов автоматический переход дальше (хешируем их на сервере и добавляем в базу на сервере), без показа результатов
- ⚠️ No matching feedback: на этом экране НЕ показывается количество найденных друзей
- ⚠️ Server-side processing: сервер сохраняет хеши в таблицу для последующего матчинга
- ⚠️ Auto-navigation: после успешной отправки автоматически переходит на следующий экран

photo_upload_screen

UI элементы:

- Skip Button: "Пропустить" (белый текст, top-right, на синем фоне)
- Background: Сплошной синий фон (solid blue gradient)
- Avatar Placeholder: Большая круглая иконка с силуэтом человека (белый круг с иконкой профиля, centered)
- Title: "Добавьте фото чтобы получать больше голосов" (белый текст, крупный шрифт, centered)
- Button 1: "Выбрать фото" (primary, rounded, blue text, white background, centered)
- Button 2: "Снять на камеру" (secondary, rounded, blue text, white background, centered)

Валидация:

- Фото: optional (можно пропустить этот шаг)
- Формат: JPEG, PNG, WebP, HEIC
- Размер исходного файла: максимум 10 МБ (до обработки)
- Итоговый размер: автоматическое сжатие до ≤250 КБ

Ошибки:

- Файл слишком большой (>10 МБ) → "Файл слишком большой. Выберите фото меньше 10 МБ"
- Неверный формат → "Неподдерживаемый формат. Выберите JPEG, PNG или WebP"
- Ошибка обработки → "Не удалось обработать фото. Попробуйте другое"
- Ошибка загрузки → "Не удалось загрузить фото. Попробуйте снова"
- ⚠️ Релиз 1 (без разрешений): Разрешение камеры отклонено → "Для использования камеры необходимо предоставить разрешение в настройках". В первом релизе можно даже скрыть экран. Во втором релизе уже будем этот экран видимым делать

Состояния:

- Default: экран с placeholder аватара, две кнопки активны, кнопка "Пропустить" видна
- Choose Photo Clicked: открывается галерея устройства (системный picker)
- Camera Clicked: открывается камера устройства (системный camera view)
- Photo Selected:
 - Показывает скорпер для обрезки фото до квадрата (пользователь может выбрать область)
 - После подтверждения обрезки → показывает spinner "Обработка..."
 - обрезаем до квадрата и масштабируем до 640×640 px
 - сжимаем фото у клиента на телефоне, не на сервере есть специальные сервисы до ≤250 КБ (JPEG/WebP, качество ~75%)
 - Отправляет обработанное фото на сервер (POST /api/users/profile/avatar)
 - Показывает preview загруженного фото вместо placeholder
- Processing: spinner "Обработка..." + кнопки disabled
- Uploading: spinner "Загрузка..." + progress bar (опционально) + кнопки disabled
- Upload Success: toast "✅ Фото загружено" + кнопки "Выбрать фото" и "Снять на камеру" меняются на "Изменить фото" и "Удалить фото"
- Upload Error: toast "❌ Не удалось загрузить фото. Попробуйте снова" + кнопки активны
- Photo Uploaded: preview загруженного фото, кнопки "Изменить фото" и "Удалить фото" активны, кнопка "Пропустить" меняется на "Далее"
- Skip: автоматический переход на следующий экран без загрузки фото

Важно:

⚠️ Optional step: загрузка фото опциональна, пользователь может пропустить
ВАЖНО ОБЪЯСНИТЬ почему мы берем разрешение. Мы используем ваши фото, чтобы вы могли выбрать аватар из галереи или сделать фото на камеру для аватара (2 релиз в сторы, 1 релиз без этого экрана можно)

- ⚠️ Skip button: кнопка "Пропустить" всегда видна в правом верхнем углу
- ⚠️ Auto-crop: фото автоматически обрезается до квадрата (1:1)
- ⚠️ Auto-resize: масштабируется до 640×640 px
- ⚠️ Auto-compress: сжимается у клиента на телефоне до ≤250 КБ (JPEG/WebP, качество ~75%)
- ⚠️ Original not saved: оригинал не сохраняется, только обработанная версия
- ⚠️ Quality: достаточно чёткости для просмотра в полноэкранном режиме с возможностью увеличения
- ⚠️ Image cropper: пользователь может выбрать область обрезки (drag & zoom)
- ⚠️ Two options: выбор из галереи или съемка с камеры
- ⚠️ 📱 РЕЛИЗ 1 (первая версия в App Store): функционал работает БЕЗ запроса разрешений на камеру и галерею (используются системные API без явного запроса)
- ⚠️ 📱 РЕЛИЗ 2 (вторая версия): будет добавлен явный запрос разрешений с пояснениями для пользователя что мы берем доступ к фото и камере для того чтобы они могли создать аватар. Фото и камера это очень чувствительная проверка для app store поэтому их оставим на потом.
- ⚠️ Replace photo: после загрузки можно изменить или удалить фото
- ⚠️ Motivation: явное указание на преимущество ("получать больше голосов")

⚠️ Каждый пользователь получает свой уникальный промокод при регистрации (например, JOHN2024, ALICE123)

⚠️ Новый пользователь вводит промокод друга → система записывает связь "кто кого пригласил"

⚠️ У каждого пользователя свой постоянный промокод

before_vote_screen

Валидация:

- Нельзя начать голосование без доступа к контактам (показывается alert)

Ошибки:

- Нет доступа к контактам + нажал "Начать голосование" → Alert: "Нельзя голосовать так как нет доступа к контактам"
- Ошибка запроса контактов → Toast: "Не удалось получить доступ к контактам. Попробуйте позже"

Состояния:

Default (по умолчанию):

Загрузка экрана:

- Показывается сообщение: "В вашей школе пока меньше 5 учеников"
- Объяснение: "Чтобы начать голосования, нужно дать доступ контактам"

- Две кнопки:
 - "Включить контакты" (primary - основная)
 - "Начать голосование" (secondary - второстепенная)
- Блок с гарантиями приватности (3 пункта)

Контекст:

- Экран показывается когда в школе пользователя меньше 5 зарегистрированных учеников, если в школе есть больше 5 учеников то сразу показывается экран голосования
- Пользователь ещё не дал доступ к контактам
- Это обязательный шаг для начала использования приложения

Back Button Clicked (стрелка назад):

- Пользователь нажал стрелку ←
- Возврат на предыдущий экран (откуда пришли)

Skip Button "Пропустить" Clicked:

Процесс:

1. Пользователь нажал "Пропустить" (вверху справа)
2. Экран закрывается
3. Переход на Before_Vote_2_Screen
4. НЕ запрашивает контакты (пропускает этот шаг)

Что такое Before_Vote_2_Screen:

- Следующий экран в onboarding
- "Пропустить" НЕ даёт начать голосовать
- Просто переходит дальше по onboarding
- Пользователь сможет дать контакты позже

Button "Включить контакты" Clicked:

Процесс:

1. Пользователь нажал "Включить контакты"
2. Запрашивается системное разрешение на контакты:
 - iOS: System Alert "Приложение хочет получить доступ к вашим контактам"
 - Кнопки: "Не разрешать" / "Разрешить"
 - Android: System Dialog "Разрешить приложению доступ к контактам?"
 - Кнопки: "Запретить" / "Разрешить"
3. Если пользователь РАЗРЕШИЛ:
 - Показывается Loading: "Загружаем контакты..."
 - Контакты читаются локально (на телефоне)
 - Хэши номеров телефонов отправляются на сервер
 - Сервер сопоставляет с зарегистрированными пользователями
 - Показывается список найденных друзей (если есть)
 - Или предложение пригласить (если никого не нашли)
 - Пользователь может начать голосовать по контактам
 - Переход на главный экран (с опросами из контактов)
4. Если пользователь ОТКАЗАЛ (нажал "Не разрешать"):
 - Возврат на Before_Registration_Screen (этот же экран)
 - Кнопка "Включить контакты" остаётся активной
 - Можно попробовать снова
 - Или показывается подсказка: "Без контактов голосования недоступны"

Button "Начать голосование" Clicked:

Процесс:

1. Пользователь нажал "Начать голосование"
2. Проверка: дал ли доступ к контактам?
3. Если ДА (доступ к контактам есть):
 - Начинается голосование по контактам пользователя
 - Переход на главный экран с опросами
 - Опросы показываются с людьми из контактов
 - Важно: Это работает даже если в школе меньше 5 человек
 - Пользователь голосует среди своих контактов (не среди школы)
4. Если НЕТ (доступа к контактам нет):

- Показывается Alert (всплывающее окно):
- Code

Заголовок: "Нужен доступ к контактам"

Сообщение:

"Нельзя голосовать так как нет доступа к контактам.

Разрешите доступ, чтобы начать голосования."

Кнопки:

[Дать доступ] [Отмена]

-
- Если нажал "Дать доступ" → запрос системного разрешения (как кнопка "Включить контакты")
- Если нажал "Отмена" → alert закрывается, остаётся на Before_Registration_Screen

Логика работы (почему так устроено):

Два режима голосования:

Режим 1: Голосование по школе (идеальный)

Code

Условие: В школе 5+ зарегистрированных учеников

Как работает:

- Пользователь видит опросы с одноклассниками
- Голосует за людей из своей школы
- Это основной режим (для чего создано приложение)

Этот экран НЕ показывается (сразу работает)

Режим 2: Голосование по контактам (временный)

Code

Условие: В школе меньше 5 учеников

Как работает:

- Пользователь даёт доступ к контактам
- Голосует за людей из своих контактов (которые в OISTER)
- Это временный режим (пока школа не наберёт 5 человек)

Этот экран показывается (нужны контакты)

Зачем нужны контакты:

- Чтобы приложение работало сразу (не ждать пока 5 человек в школе)
- Пользователь может голосовать за друзей из контактов
- Мотивирует пригласить одноклассников (чтобы перейти в режим школы)

Переход в режим "Голосование по школе":

Когда происходит:

- Когда в школе появилось 5+ зарегистрированных учеников
- Автоматически (пользователь не делает ничего)

Что меняется:

- Before_Registration_Screen больше не показывается
- Опросы показываются с одноклассниками (не с контактами)
- Голосования становятся релевантными для школы

Уведомление:

- Пользователь получает push: "👋 В вашей школе 5+ учеников! Теперь доступны голосования"

Важно:

- ⚠️ Two voting modes: 2 режима голосования (по школе / по контактам)
- ⚠️ School mode: если в школе 5+ человек → голосование по школе (идеальный режим)
- ⚠️ Contacts mode: если в школе меньше 5 → голосование по контактам (временный режим)
- ⚠️ Contacts required: без контактов нельзя начать голосование (alert)
- ⚠️ Skip available: кнопка "Пропустить" → переход на Before_Vote_2_Screen (не даёт голосовать)
- ⚠️ Two buttons: "Включить контакты" (запрос разрешения) и "Начать голосование" (проверка + старт)
- ⚠️ System permission: используется нативное системное окно iOS/Android для контактов
- ⚠️ Can retry: если отказал в контактах → можно попробовать снова
- ⚠️ Privacy guaranteed: контакты не сохраняются на сервере (только хэши для сопоставления)
- ⚠️ Automatic transition: когда в школе 5+ человек → автоматический переход в режим школы
- ⚠️ Not blocking: экран не блокирует приложение (можно пропустить через "Пропустить")
- ⚠️ Legal for Apple: соответствует требованиям Apple (есть объяснение + можно пропустить)

before_vote_2_screen

Валидация:

Нет валидации - это информационный экран с кнопками действий

Ошибки:

- Ошибка копирования ссылки → Toast: "Не удалось скопировать ссылку. Попробуйте позже"
- Ошибка генерации ссылки → Toast: "Не удалось создать ссылку приглашения"

Состояния:

Default (по умолчанию):

Загрузка экрана:

- Показывается эмодзи 🙌 (пожимание плечами)
- Сообщение: "Здесь пока ничего нет"
- Объяснение: "Чтобы начать голосования, найдите друзей из вашей школы"
- Две кнопки:
 - "📱 Поделиться ссылкой" (primary - выделена розовой обводкой)
 - "📱 Найти друзей" (secondary)
- Внизу предупреждение: "⚠️ Для голосования нужно минимум 5 человек из вашей школы"

Контекст:

- Экран показывается после Before_Registration_Screen
- Пользователь нажал "Пропустить" (не дал контакты на предыдущем экране)
- В школе меньше 5 человек
- Голосование пока недоступно (нужно пригласить друзей или дать контакты)

Back Button Clicked (стрелка назад):

- Пользователь нажал стрелку ←
- Возврат на Before_Registration_Screen (предыдущий экран)
- Там можно дать доступ к контактам

Button "Поделиться ссылкой" Clicked:

Процесс:

1. Пользователь нажал "📱 Поделиться ссылкой"
2. Генерация персональной ссылки приглашения:
 - Отправляется запрос на сервер: POST /api/invite/generate
 - Сервер создаёт уникальную ссылку:

- Code

<https://oister.app/invite/abc123def>

-

- Параметр abc123def = ID пригласившего пользователя

3. Копирование ссылки в буфер обмена:

- Ссылка автоматически копируется
- Показывается Toast: "✔ Ссылка скопирована"

4. Пользователь сам делится ссылкой:

- Вставляет в WhatsApp, Telegram, Instagram, SMS, etc.
- Отправляет друзьям из школы

5. Что происходит когда друг переходит по ссылке:

- Открывается веб-страница или App Store/Google Play
- Друг скачивает приложение
- При регистрации автоматически привязывается к пригласившему
- Засчитывается в "5 человек" для школы

Формат ссылки:

Code

<https://oister.app/invite/abc123def>

Параметры:

- abc123def = inviteCode (уникальный код пользователя)

Что показывается по ссылке (веб):

HTML

Button "Найти друзей" Clicked:

Процесс:

1. Пользователь нажал "📱 Найти друзей"
2. Возврат на Before_Registration_Screen:
 - Экран закрывается
 - Открывается Before_Registration_Screen (предыдущий экран)
 - Там пользователь может дать доступ к контактам
3. Если даст доступ к контактам:
 - Запрашивается системное разрешение
 - Контакты синхронизируются
 - Начинается голосование по контактам (временный режим)
 - Переход на главный экран с опросами
4. Если не даст доступ:
 - Остаётся на Before_Registration_Screen
 - Может снова нажать "Пропустить" → вернуться на Before_Vote_2_Screen

Логика голосования (два режима):

Режим 1: Голосование по контактам (временный) 📱

Когда активируется:

- Пользователь дал доступ к контактам
- В школе меньше 5 человек

Как работает:

- Опросы показываются с людьми из контактов пользователя
- Голосует за тех, кто в его телефонной книге (и в OISTER)
- Не важно из какой они школы
- Это временный режим (пока школа не наберёт 5 человек)

Пример:

Code

У Ивана в контактах:

- Маша (его одноклассница, школа №1)
- Петя (друг из другой школы №5)
- Катя (сестра, школа №10)

Опрос: "Кто самый крутой?"

Варианты: Маша, Петя, Катя

Иван голосует за людей из контактов (даже из разных школ)

Режим 2: Голосование по школе (настоящий) 🏫

Когда активируется:

- В школе 5+ зарегистрированных учеников
- Автоматически (не нужно ничего делать)

Как работает:

- Опросы показываются с одноклассниками
- Голосует только за учеников своей школы
- Это основной режим (для чего создано приложение)
- Голосования становятся релевантными и честными

Пример:

Code

В школе №1 зарегистрировались:

- Иван (9 класс)
- Маша (9 класс)
- Петя (10 класс)
- Катя (10 класс)
- Олег (11 класс)

Опрос: "Кто самый крутой в 9 классе?"

Варианты: Иван, Маша (только из 9 класса школы №1)

Иван голосует за любого ученика в школе

Переход между режимами:

Когда происходит:

- Когда в школе появился 5-й ученик
- Система автоматически проверяет каждый час

Что меняется:

1. Уведомление пользователю:
 - Push: 🎉 В вашей школе 5+ учеников! Теперь доступны настоящие голосования"
2. Опросы обновляются:
 - Старые опросы (по контактам) скрываются
 - Новые опросы (по школе) показываются
 - Варианты ответов = только одноклассники
3. Before_Vote_2_Screen больше не показывается:
 - При следующем запуске приложения
 - Сразу показываются опросы по школе

Если пользователь остаётся на экране (ничего не нажал):

Что происходит:

- Пользователь остаётся на Before_Vote_2_Screen
- Экран не закрывается автоматически
- Нельзя перейти дальше (нет кнопки "Продолжить")
- Нужно либо:
 - Нажать "Поделиться ссылкой" (пригласить друзей)
 - Нажать "Найти друзей" (дать контакты)
 - Нажать стрелку назад (вернуться)

Зачем:

- Мотивировать пользователя действовать
- Без контактов приложение не работает
- Нужно либо пригласить друзей, либо дать контакты

Важно:

- ⚠ Two voting modes: 2 режима голосования (по контактам → по школе)
- ⚠ Contacts mode first: сначала голосование по контактам (если дал доступ)
- ⚠ School mode at 5+: когда 5+ в школе → автоматический переход на голосование по школе
- ⚠ Share link: кнопка "Поделиться ссылкой" копирует персональную ссылку в буфер
- ⚠ Find friends: кнопка "Найти друзей" возвращает на Before_Registration (дать контакты)
- ⚠ Cannot skip: нельзя пропустить этот экран (нет кнопки "Продолжить")
- ⚠ Must act: пользователь должен либо пригласить друзей, либо дать контакты
- ⚠ Back available: можно вернуться назад (стрелка ←)
- ⚠ Toast feedback: при копировании ссылки показывается toast "Ссылка скопирована"
- ⚠ Invite tracking: приглашения отслеживаются (кто по чьей ссылке зарегистрировался)
- ⚠ Auto transition: переход в режим школы автоматический (когда 5+ человек)
- ⚠ Persistent screen: экран не исчезает пока не выполнишь действие

Экраны Голосования

ВАЖНО: ЛОГИКА ГОЛОСОВАНИЯ ВАЖНО ПРОЧИТАТЬ ЕСЛИ НЕПОНЯТНО СПРАШИВАЙТЕ БУДУ РАД ПОМОЧЬ

У нас есть два типа голосования, которые зависят от количества учеников в школе:

Тип 1: Школа с менее чем 4 учениками

Процесс:

1. Сценарий А: Доступ к контактам НЕ предоставлен
 - Показываем информационное окно с объяснением необходимости доступа
 - Пользователь не может продолжить голосование без доступа
2. Сценарий Б: Доступ к контактам предоставлен
 - Загружаем контакты из телефона пользователя
 - Фильтруем контакты по ключевым словам:
 - одноклассник/одноклассница
 - друг/подруга
 - школа/класс
 - сосед по парте
 - нравится
 - краш/crush
 - симпатия
 - ❤️ / 💕 / 😍
 - Начинаем голосование с отфильтрованными контактами. Если эти контакты закончились то нужно показывать случайно из телефонной книги. ВАЖНО ЧТОБЫ ВСЕГДА ПОКАЗЫВАЛИСЬ РАЗНЫЕ ЛЮДИ ИЗ ЕГО ТЕЛЕФОННОЙ КНИГИ. Простыми словами если в школе меньше 5 человек то тогда мы его контакты случайно показываем начиная с ключевых слов а потом остальных. ВАЖНО нужно чтобы контакты не повторялись всегда разные были даже если он будет голосовать 300 раз по контактам. И в конце также экран "Монеты получает потом пригласить друга."
 - Имена из контактов отображаются в интерфейсе голосования

Важно: Поиск должен работать с учетом регистра и возможных вариаций написания слов.

Тип 2: Школа с 5 или более учениками

Процесс:

- Голосование начинается автоматически
- Используется стандартный алгоритм подбора
- Пользователи из базы школы попадают в голосование согласно алгоритму матчинга

Алгоритм "Лепесток" для голосования

 Баланс 25/25/25/25

12 вопросов = 4 категории по 3 вопроса:

- Вопросы 1,5,9: Общие друзья (25%)
- Вопросы 2,6,10: Мало голосов (25%)
- Вопросы 3,7,11: Общие контакты (25%)
- Вопросы 4,8,12: Популярные (25%)

Итог: 50% знакомые + 50% новые

 Механика "Колода карт"

Для каждой категории создаём **отсортированный список** всех учеников школы. Идём по списку **по очереди**, показывая по 4 человека. Когда список кончается → начинаем сначала. **Сортировка**: категория "Мало голосов" = по возрастанию голосов (меньше голосов → в начале списка → показывается чаще). Остальные категории = по убыванию (друзья/контакты/популярность) + меньше голосов как второй критерий.

 Обновление каждые 24 часа

В 00:00 каждую ночь: пересчитываем голоса за 7 дней для всех учеников → заново сортируем все 4 списка → сбрасываем позиции на 0. **Результат**: кто получил много голосов вчера → опускается вниз списка → показывается реже. Новички/непопулярные → поднимаются вверх → показываются чаще. Автоматическая балансировка!

 Redis кэширование

Что храним в Redis (TTL = 24 часа):

- `school:{schoolId}:category:{categoryName}:list` = отсортированный массив ID учеников [12, 45, 78, ...]
- `school:{schoolId}:category:{categoryName}:position` = текущая позиция в списке (число 0-100)
- `user:{userId}:votes_7days` = количество голосов за неделю

Логика: при каждом запросе берём из Redis текущую позицию → показываем 4 ученика → сдвигаем позицию +4 → сохраняем в Redis. Обновление списков раз в сутки (00:00). **Производительность**: 1 запрос к Redis вместо 10 запросов к PostgreSQL = в 50 раз быстрее.

Если в категории нету то берем случайно

 Гарантии

Все ученики школы появятся в голосованиях. За 50 раундов × 150 показов категории "Мало голосов" = каждый новичок появится минимум 2-3 раза в день. Популярные появятся реже. Баланс 25/25/25/25 строго соблюдается. Справедливо + разнообразно + автоматическая балансировка!

vote1_screen, vote2_screen, vote3_screen, vote4_screen

UI элементы:

- Background: Сплошной синий фон (solid blue gradient)
- Progress Indicator: "1 из 12" (белый текст, top-center, показывает прогресс в текущем раунде)
- Emoji: 😊 (большой эмодзи, centered, фиксированный для каждого вопроса, есть 12 эмодзи они всегда стоят в одинаковой последовательности)
- Question Text: "Кто самый красивый?" (белый текст, крупный шрифт, centered, под эмодзи)
- Student Cards: 4 карточки учеников (white background, rounded corners, с аватаром, именем и классом)
 - Card 1: Арман Никитенко, 9 класс
 - Card 2: Александр Иванов, 10 класс
 - Card 3: Вигаман Магомедов, 11 класс
 - Card 4: Анна Исхакова, 10 класс
- Shuffle Button: "Поменять имена" (белый текст, bottom-left, исчезает после 2 использований за раунд (1 раунд 12 вопросов))
- Skip Button: "Следующее голосование" (белый текст, bottom-right, исчезает после 2 использований за раунд (1 раунд 12 вопросов))
- Continue Button: "Нажмите чтобы продолжить" (white background, rounded, blue text, centered, появляется только после выбора ученика)

Валидация:

- Выбор ученика: required (обязательно выбрать одного ученика)
- Только один выбор: нельзя выбрать двух и более учеников одновременно

- Школа: пользователь видит только учеников своей школы (фильтрация по school_id)
- Лимиты:
 - Поменять имена: максимум 2 раза за раунд (1 раунд=12 вопросов)
 - Следующее голосование: максимум 2 раза за раунд (1 раунд=12 вопросов)
 - Голосований: 50 раундов по 12 вопросов = 600 вопросов в день
 - Между раундами: таймер 40 минут (обход через реферальную систему)

Ошибки:

- Ошибка загрузки учеников → "Не удалось загрузить учеников. Попробуйте снова"
- Ошибка отправки голоса → "Не удалось отправить голос. Попробуйте снова"
- Ошибка сети → "Проверьте подключение к интернету"
- Превышен лимит голосований → "Вы достигли лимита голосований на сегодня"
- Таймер не истёк → "Следующее голосование доступно через [время]". когда он в будущем будет нажимать на Главная и если время не прошло то там будет появляться экран с таймером

Состояния:

- Default:
 - Показаны 4 ученика с белым фоном карточек
 - Кнопки "Поменять имена" и "Следующее голосование" видны (если не исчерпан лимит)
 - Кнопка "Продолжить" невидима (не появилась)
 - Прогресс "1 из 12" (или текущий номер вопроса)
- Student Selected:
 - Выбранная карточка меняет фон на серый
 - Остальные карточки остаются белыми
 - Кнопка "Продолжить" появляется (становится видимой)
- Switching Selection:
 - При клике на другого ученика:
 - Предыдущая карточка возвращается к белому фону
 - Новая карточка становится серой
 - Кнопка "Продолжить" остаётся видимой
- Submitting Vote:
 - После нажатия "Продолжить":
 - Показывается loader/spinner на кнопке или на экране
 - Кнопки "Поменять имена" и "Следующее голосование" становятся disabled
 - Карточки учеников становятся disabled
- Vote Submitted:
 - Без toast - сразу автоматический переход к следующему вопросу
 - Прогресс меняется: 1/12 → 2/12
 - Показываются новые 4 ученика для нового вопроса
 - Кнопка "Продолжить" снова исчезает
- Shuffle Clicked:
 - Показывается loader на карточках учеников
 - Загружаются новые 4 ученика согласно Алгоритму "Лепесток"
 - Счётчик "Поменять имена" уменьшается (2 → 1 → 0)
 - После 2 использований кнопка "Смешать" исчезает
- Skip Clicked:
 - Вопрос пропускается навсегда для этого пользователя
 - Прогресс меняется: 1/12 → 2/12
 - Показываются новые 4 ученика для нового вопроса
 - Счётчик "Следующее голосование" уменьшается (2 → 1 → 0)
 - После 2 использований кнопка "Пропустить" исчезает
- Round Completed (12/12):
 - После 12-го вопроса раунд завершается
 - Переход к следующему экрану получения монет автоматически
- Loading:
 - Skeleton loader для карточек учеников при первой загрузке
 - Spinner на кнопке "Продолжить" при отправке голоса
- Error:
 - Toast **✗** Не удалось отправить голос. Попробуйте снова"
 - Кнопки и карточки снова становятся активными
 - Пользователь может попробовать ещё раз

Важно:

- ⚠ School filtering: пользователь видит только учеников своей школы (school_id), фильтрация на бэкенде
- ⚠ Single selection: можно выбрать только одного ученика за раз
- ⚠ Button appears: кнопка "Продолжить" появляется только после выбора ученика
- ⚠ Can switch: до нажатия "Продолжить" можно переключаться между учениками

- ⚠ No undo: после нажатия "Продолжить" отмена невозможна
- ⚠ No toast: без уведомлений, сразу переход к следующему вопросу
- ⚠ Daily limits:
 - Смешать имена: 2 раза в день → потом исчезает
 - Следующее голосование: 2 раза в день → потом исчезает
 - Голосования: 50 раундов × 12 вопросов = 600 вопросов в день ⚠ 40-minute timer: между раундами (12 вопросов) - таймер 40 минут
 - ⚠ Referral bypass: можно обойти таймер, пригласив друга (push-уведомление при установке)
 - ⚠ Fixed emojis: каждый из 12 вопросов имеет фиксированный эмодзи
 - ⚠ Questions don't repeat: вопросы не повторяются в течение дня для одного пользователя
 - ⚠ Strict sequence: пользователь не может перескакивать вопросы (строгая последовательность 1/12 → 2/12 → ... → 12/12)
 - ⚠ Progress saved: позиция сохраняется между сессиями
 - ⚠ Backend validation: все лимиты, таймеры и ограничения проверяются на бэкенде
 - ⚠ No navigation: нет кнопки "Назад", нет нижней панели - пользователь должен пройти раунд до конца
 - ⚠ 4 students always: всегда показываются 4 ученика (алгоритм "Лепесток" на бэкенде)
- ⚠ Важно просто есть 4 цвета фона экрана поэтому там 4 экрана и они постоянно меняются чтобы было красочнее но по сути они одинаковые.
- ⚠ Если к примеру пользователь вышел и прошел 5 опросов, то когда он вернется и нажмет на главную то пусть сразу будет на 6 вопросе.
- ⚠ При нажатии на аватарку лицо человека должно открываться на весь экран с крестиком маленьким в углу для возможности закрытия. Также можно делать увеличение фотки. фотки будут в формате квадрат.

Алгоритм "Друзья Друзей"

🌀 Алгоритм расчёта

Шаги:

1. Взять всех моих друзей (accepted)
2. Для каждого друга взять его друзей
3. **Исключить:**
 - Меня
 - Моих текущих друзей (accepted)
 - Заблокированных (обе стороны)
4. Посчитать баллы для каждого кандидата
5. Отсортировать: больше баллов → выше
6. Взять топ 100

НЕ исключать:

- Pending запросы (показываем с кнопкой "Запрос отправлен")
- Входящие запросы (показываем с кнопками "Принять"/"Удалить")

📊 Формула расчёта баллов (как Instagram/Facebook)

Базовая формула:

Итоговый балл =

Количество общих друзей × 1

+ (та же школа × 2)

+ (тот же класс × 3)

+ (тот же город × 1)

+ (одинаковый возраст ±1 год × 0.5)

Примеры расчёта:

Кандидат 1: Маша

- 5 общих друзей = 5 баллов
- Та же школа = +2 балла
- Тот же класс (9 класс) = +3 балла
- **Итого: 10 баллов → 1 место**

Кандидат 2: Петя

- 7 общих друзей = 7 баллов
- Другая школа = 0 баллов
- **Итого: 7 баллов → 2 место**

Кандидат 3: Катя

- 3 общих друга = 3 баллов
- Та же школа = +2 балла
- Другой класс = 0 баллов
- **Итого: 5 баллов → 3 место**

Сортировка:

1. Маша (10 баллов)
2. Петя (7 баллов)
3. Катя (5 баллов)

 Формат данных в Redis

Сохраняем:

- **Ключ:** `suggestions:friends_of_friends:{user_id}`
- **TTL:** 24 часа
- **Значение:** JSON массив топ 100 кандидатов

Структура каждого кандидата:

- ID пользователя
 - Имя, фамилия
 - Username
 - Аватар
 - Класс
 - **Количество общих друзей**
 - **Итоговый балл** (для сортировки)
 - Статус отношений (none/pending_sent/pending_received)
-

Расписание

Background Job (Cron):

- **Когда:** Каждую ночь в 3:00 AM один раз в день или если ток зарегистрировался то сразу один раз потом ток раз в день ночью
- **Что:**
 1. Для каждого пользователя считает баллы по формуле
 2. Сортирует по убыванию баллов
 3. Берёт топ 100
 4. Сохраняет в Redis
- **Время:** 30-60 минут для 50,000 пользователей

API днём:

- **Endpoint:** GET `/api/search/suggestions/friends-of-friends`
- **Логика:** Достает готовый список из Redis (5 мс)
- **Нагрузка на PostgreSQL:** 0 запросов

Кэширование

Уровень 1: Redis (сервер)

- Данные на 24 часа
- Обновление ночью

Уровень 2: AsyncStorage (телефон)

- Скачивает при первом открытии
- Хранит 24 часа
- Следующие открытия → с телефона

Уровень 3: RAM (память)

- При прокрутке → из памяти
- 0 запросов

Алгоритм "Пригласить друзей"

Цель

Показать контакты из телефона, которых НЕТ в приложении, отсортированные по количеству общих друзей.

Шаги алгоритма

1. Получение данных

- Читаем все контакты из телефона
- Хешируем номера (SHA-256)
- Отправляем хеши на сервер

2. Фильтрация

- Исключаем хеши, которые УЖЕ есть в приложении
- Остаются только те, кого НЕТ в приложении

3. Расчёт баллов

Для каждого контакта считаем: **Сколько моих друзей (из приложения) тоже имеют этот контакт в своей телефонной книге**

4. Сортировка

По убыванию количества общих друзей

5. Результат

Топ 100 контактов для приглашения



Формула баллов

Балл = Количество моих друзей, у которых есть этот контакт

Пример:

- Петя: 8 моих друзей имеют его в контактах → 8 баллов → 1 место
- Маша: 3 друга → 3 балла → 2 место
- Саша: 0 друзей → 0 баллов → 3 место



Данные в БД

Таблица user_contacts:

- user_id
- contact_hash
- created_at

Назначение: Хранить хеши контактов каждого пользователя для подсчёта общих



Обновление

Вариант А: В реальном времени (при открытии экрана) **Вариант В:** Предрасчёт ночью в 3 AM → Redis на 24 часа



Итог

Люди с максимальным количеством общих друзей показываются первыми → выше вероятность, что я их знаю и приглашу.

star_received_screen

UI элементы:

- Background: Сплошной синий фон (solid blue gradient)
- Gift Image: 3D изображение подарка с золотым бантом (статичная картинка, centered, сверху)
- Congratulations Text: "Босс, поздравляю" (белый текст, крупный шрифт, bold, centered)
- Reward Text: "Вам начислено 1000 звездочек за прохождение 1 уровня" (белый текст, средний размер, centered, под поздравлением)
- Stars Animation: 5 звёздочек (★★★★★) с анимацией падения сверху (золотые звёздочки, centered)
- Button: "★ Пополнить свою коллекцию" (white background, rounded, blue text с иконкой звезды, centered внизу)

Валидация:

- Нет валидации - это экран награды, не требует ввода данных

Ошибки:

- Ошибка начисления звёздочек → "Не удалось начислить награду. Попробуйте позже"
- Ошибка загрузки экрана → "Произошла ошибка. Перезагрузите приложение"

ВАЖНО: Уровень меняется просто инкрементом, 1,2,3 уровень. В день 50 раундов по 12 вопросов максимально можно значить в день максимум 50 уровней и каждый раз поздравляем. Можно по дизайну фанфары анимацию добавить на ваше усмотрение. На след день если он прошел 50 то 51 уровень, если прошел 15 значит на след день 16 уровень.

Состояния:

- Loading:
 - Skeleton loader или spinner при загрузке экрана
- Appearing:
 - Экран появляется с fade-in анимацией
 - Изображение подарка появляется (scale-in или bounce)
 - Текст "Босс, поздравляю" появляется
 - Текст "Вам начислено 1000 звездочек..." появляется
 - 5 звёздочек падают сверху (stagger animation, одна за другой)
 - Звук успеха проигрывается (например, "ta-da!" или звон монет)
 - Кнопка "Пополнить свою коллекцию" появляется снизу
- Default:
 - Все элементы видны
 - Анимация звёздочек завершена
 - Кнопка активна и ожидает нажатия
- Button Clicked:
 - Показывается loader/spinner на кнопке
 - Кнопка становится disabled
 - Переход на следующий экран (описание в следующем экране)
- Success:
 - Звёздочки успешно начислены на баланс пользователя
 - Автоматический переход на следующий экран после нажатия кнопки
- Error:
 - Toast  Не удалось начислить награду. Попробуйте позже"
 - Кнопка остаётся активной для повтора

Важно:

- ⚠ Always shows: экран всегда показывается после завершения раунда из 12 вопросов
- ⚠ Fixed reward: всегда фиксированно 1000 звёздочек, не меняется
- ⚠ Stars animation: 5 звёздочек падают сверху (stagger animation, одна за другой с задержкой ~0.2s) плюс вибрация
- ⚠ Success sound: при появлении экрана проигрывается звук успеха (например, "ta-da!", звон монет, фанфары)
- ⚠ No skip: нельзя пропустить этот экран - обязательно нужно нажать кнопку
- ⚠ No balance display: на этом экране не показывается текущий баланс звёздочек
- ⚠ Static gift: изображение подарка - статичная картинка (не анимированная)
- ⚠ Stars accumulate: звёздочки накапливаются на балансе пользователя
- ⚠ In-game currency: звёздочки можно потратить на внутриигровые функции

- ⚠ Mandatory click: пользователь должен нажать кнопку "Пополнить свою коллекцию" для продолжения
- ⚠ Next screen: после нажатия кнопки → переход на новый экран с другой анимацией(будет описан отдельно)
- ⚠ Backend credit: начисление звёздочек происходит на бэкенде при завершении раунда
- ⚠ Round completion: экран показывается только после полного прохождения 12 вопросов

star_taking_screen

UI элементы:

- Background: Сплошной синий фон (solid blue gradient)
- Gift Image: 3D изображение подарка с золотым бантом (статичная картинка, centered, сверху)
- Confirmation Text: "Ваш коллекция пополнена" (белый текст, крупный шрифт, bold, centered)
- Stars Animation Source: 5 звёздочек (★★★★★) под текстом (золотые звёздочки, centered) - начальная позиция перед полётом
- Button: "👉 Хочу еще вопросы" (white background, rounded, blue text с иконкой, centered, изначально disabled/неактивна)
- Balance Display: "★ 1000" (большая звёздочка + число, белый текст, centered внизу) - целевая позиция для анимации

Валидация:

- Нет валидации - это экран анимации и подтверждения начисления

Ошибки:

- Ошибка загрузки баланса → "Не удалось загрузить баланс. Попробуйте позже"
- Ошибка обновления данных → "Произошла ошибка. Перезагрузите приложение"

Состояния:

- Initial Load:
 - Экран появляется после нажатия "Пополнить свою коллекцию" на предыдущем экране
 - Показывается подарок, текст "Ваш коллекция пополнена"
 - 5 звёздочек видны под текстом (начальная позиция)
 - Блок баланса внизу показывает старый баланс (например, 3000)
 - Кнопка "Хочу еще вопросы" disabled (серая, неактивна)
- Animation Sequence:
 - 5 звёздочек летят вниз к блоку баланса (по очереди, одна за другой)
 - Звёздочка 1 летит → задержка ~0.2s → Звёздочка 2 летит → задержка ~0.2s → ...
 - Длительность полёта каждой звёздочки: ~600-800ms
 - Траектория: от позиции под текстом → к блоку "★ 1000" внизу
 - Звук полёта для каждой звёздочки (опционально, лёгкий "whoosh")
 - Когда звёздочки долетают к блоку баланса:
 - Звук фанфар и звон монет проигрывается
 - Вибрация (опционально, haptic feedback)
 - Звёздочки исчезают (или объединяются с блоком баланса)
 - Counter animation: число в блоке баланса анимированно увеличивается
 - Было: 3000 звездочек → Стало: 4000 звездочек (за ~1-1.5 секунды)
 - Анимация счётчика с эффектом "rolling numbers"
 - После завершения анимации:
 - Кнопка "Хочу еще вопросы" становится активной (синий цвет, можно нажать)
- Animation Complete:
 - Все 5 звёздочек долетели и исчезли
 - Баланс обновлён (старый баланс + 1000)
 - Кнопка активна и ожидает нажатия
 - Звук завершён
- Button Clicked:
 - Показывается loader/spinner на кнопке
 - Переход на следующий экран при нажатии "Хочу еще вопросы"
- Loading:
 - Skeleton loader при загрузке данных баланса
- Error:
 - Toast "❌ Не удалось загрузить баланс. Попробуйте позже"

Важно:

- ⚠ Always shows: этот экран всегда показывается после каждого завершённого раунда (до 50 раз в день)
- ⚠ Stars fly down: 5 звёздочек летят вниз по очереди (stagger animation, одна за другой с задержкой ~0.2s)
- ⚠ Balance updates: блок "★ 1000" показывает общий баланс пользователя (старый баланс + 1000)
- ⚠ Counter animation: число увеличивается анимированно (например, 3000 → 4000)
- ⚠ Fanfare sound: при достижении звёздочками блока баланса - звук фанфар + звон монет+ вибрация
- ⚠ Vibration: опционально haptic feedback при завершении анимации
- ⚠ Button disabled initially: кнопка "Хочу еще вопросы" изначально неактивна (disabled)
- ⚠ Button activates: кнопка становится активной только после завершения анимации
- ⚠ Cannot skip: нельзя пропустить анимацию (обязательно дождаться завершения)
- ⚠ Balance grows: баланс растёт с каждым раундом (3000 → 4000 → 5000 → ... до 53000 если пройти все 50 раундов)
- ⚠ Backend sync: баланс синхронизируется с бэкендом
- ⚠ Static gift: изображение подарка остаётся статичным (не открывается)

room_screen (этот экран формальность только для того чтобы пройти ревью, потом он не нужен, после 1 релиза его можно удалить и его не обязательно делать круто просто чтобы был для ревьюера)

UI элементы:

- Background: Белый фон
- Main Content:
 - Icon: 🏠 Иллюстрация комнаты (синяя, centered)
 - Title: "Выберите комнату" (чёрный, bold, large, centered)
 - Search Field:
 - Icon: 🔍 (left)
 - Placeholder: "Поиск комнаты или класса"
 - Type: text
 - Rooms List (Radio buttons):
 - Radio button (один выбор)
 - Room name (чёрный, bold)
 - Location + participants: "Москва, 78 участников" (серый текст + синяя ссылка)
 - Empty State Text:
 - "Нет подходящей комнаты?" (серый)
 - "Создайте свою комнату" (синяя ссылка)
 - Buttons:
 - "Выбрать комнату" (синяя, primary, large)
 - "Создать комнату" (синяя, secondary, large)

Валидация:

- Кнопка "Выбрать комнату" активна только когда выбрана одна комната (radio button)
- Поиск: такой же алгоритм как и в тз с 3 символа и с debounce meilisearch

Ошибки:

- Комната не найдена → Empty state: "Комнат не найдено. Создайте свою!"
- Ошибка загрузки → Toast: "Не удалось загрузить комнаты"
- Ошибка вступления → Toast: "Не удалось присоединиться к комнате"

Состояния:

Что показывается:

- Иллюстрация комнаты 
- Заголовок "Выберите комнату"
- Поле поиска (пустое)
- Список всех публичных комнат в городе пользователя
- Кнопка "Выбрать комнату" DISABLED (серая, пока ничего не выбрано)
- Кнопка "Создать комнату" ACTIVE

Список комнат:

Code

- ☐ Футбол
Москва, 78 участников
- ☐ Суши
Москва, 8 участников
- ☐ Пятничная встреча
Москва, 25 участников
- ☐ Выпускники 10 школы
Москва, 13 участников

Откуда берутся комнаты:

- Показываются только публичные комнаты из города пользователя
- Город определяется из профиля (выбор при регистрации)
- Сортировка: по количеству участников (от большего к меньшему)

Search Field Focus:

- Синий border на поле
- Клавиатура открывается
- Placeholder исчезает при вводе

Search Typing (поиск по названию):

Пользователь вводит текст, например "футб":

Live search:

- Поиск начинается после 1 символа
- Фильтрация списка комнат по названию
- Case-insensitive (не важен регистр)
- Частичное совпадение (substring match)

Результаты:

Code

 футб

☒ Футбол

Москва, 78 участников

☐ Футбольный клуб

Москва, 15 участников

Если ничего не найдено:

Code

 балет

(пусто)

Нет подходящей комнаты?

Создайте свою комнату

Пользователь выбрал комнату "Футбол":

- Radio button становится синим ☒ (filled)
- Остальные остаются пустыми ☐
- Кнопка "Выбрать комнату" становится ACTIVE (синяя, кликабельная)

Можно выбрать только ОДНУ комнату (radio button behaviour)

Link "78 участников" Clicked:

Процесс:

1. Пользователь нажал на ссылку "78 участников" и сразу может вступить и начать голосование

Что показывается:

- Название комнаты
- Город
- Количество участников (ЧИСЛО, но не имена)
- Описание что такое комната
- Тип комнаты (Публичная / По ссылке)
- Кнопка "Вступить в комнату"

НЕ показывается:

-  Список имён участников (приватность)
-  История голосований
-  Админы комнаты

Кнопка "Вступить в комнату":

- Автоматически вступает в комнату (без подтверждения админа)
 - Закрывает modal
 - Комната добавляется в "Мои комнаты" пользователя
 - Toast: "✅ Вы вступили в комнату «Футбол»" и сразу начинается голосование
-

Button "Выбрать комнату" Clicked:

Процесс:

1. Пользователь выбрал комнату (например "Футбол")
2. Нажал "Выбрать комнату"
3. Автоматическое вступление в комнату:
 - POST /api/rooms/join
 - Body: { roomId: "room_123" }
4. Loading: кнопка disabled, spinner
5. Success:
 - Toast: "✅ Вы вступили в комнату «Футбол»"
 - Переход на Home_Screen (главный экран с голосованиями этой комнаты)

Что происходит:

- Пользователь становится участником комнаты
 - Теперь видит голосования этой комнаты на главном экране
 - Может участвовать в голосованиях
-

Button "Создать комнату" Clicked:

Переход на Create_Room_Screen (создать обычный простой экран после ревью он тоже удалится, главное первое ревью пройти)

"Создать комнату" → переход на Create_Room_Screen

ЧТО ТАКОЕ КОМНАТА:

Определение:

Code

Комната = группа пользователей, которые получают ОДИНАКОВЫЕ вопросы от нас для голосования

Система автоматически генерирует вопросы (НЕ пользователи):

- "Кто самый умный?"
- "Кто лучший друг?"
- etc.

Все участники комнаты видят одни и те же вопросы и голосуют друг за друга.

Типы комнат:

1. Публичная комната:

- Видна в списке всем пользователям города
- Любой может вступить (без подтверждения)
- Показывается на Room_Screen

2. Комната по ссылке (приватная):

- НЕ видна в публичном списке
- Вступить можно ТОЛЬКО по ссылке-приглашению
- Ссылка: <https://oister.app/room/invite/abc123>

Отличие от школьного голосования:

Code

Школьное голосование (Before_Vote_1/2):

- Голосуешь за одноклассников из ШКОЛЫ
- Привязано к учебному заведению
- Нужно 5+ человек из школы

Комната:

- Голосуешь за участников КОМНАТЫ
- НЕ привязано к школе
- Можно создать по любому интересу (футбол, суши, etc.)
- Город + название (без школы)

Важно:

- ⚠ Room = same questions: в комнате все получают одинаковые вопросы от системы
- ⚠ System-generated: вопросы генерируются автоматически (не пользователями)
- ⚠ Two types: публичная (видна всем) / по ссылке (приватная)
- ⚠ Public rooms: показываются только из города пользователя
- ⚠ Auto-join: вступление без подтверждения админа (для публичных)
- ⚠ No names shown: при просмотре комнаты видно только количество участников
- ⚠ Link join: приватные комнаты доступны только по ссылке-приглашению
- ⚠ One selection: radio button (можно выбрать только одну комнату)
- ⚠ Search by name: поиск только по названию комнаты
- ⚠ City-based: комнаты привязаны к городу (не к школе)
- ⚠ No back button: нет кнопки назад (убрали по требованию)

timer_screen

UI элементы:

- Background: Сплошной синий фон (solid blue gradient)
- Decorative Capsules: 2 золотые/коричневые 3D капсулы (декоративные, centered вверху)
- Question Text: "Хочешь попробовать еще раз?" (белый текст, крупный шрифт, bold, centered)
- Timer Display: "Новый опрос через 40:00" (белый текст, крупный шрифт, centered) - обратный отсчёт в реальном времени
- Separator: "или" (белый текст, меньший размер, centered)
- Invitation Text: "Пригласи друга и сразу начинай голосовать" (белый текст, средний размер, centered)
- Button: "📞 Пригласить друга" (white background, rounded, blue text с иконкой телефона, centered)

- Bottom Navigation Bar: 5 табов (Главная, Поиск, Лайки, Профиль, Ещё)
 - Tab 1: 🏠 Главная (активный - синий)
 - Tab 2: 🔍 Поиск
 - Tab 3: ❤️ Лайки
 - Tab 4: 👤 Профиль
 - Tab 5: ☰ Ещё

Валидация:

- Нет валидации - это экран ожидания с таймером

Ошибки:

- Ошибка загрузки таймера → "Не удалось загрузить данные. Попробуйте позже"
- Ошибка синхронизации времени → "Проверьте подключение к интернету"

Состояния:

- Timer Running:
 - Таймер показывает обратный отсчёт: 40:00 → 39:59 → 39:58 → ... → 00:01 → 00:00
 - Обновляется каждую секунду
 - Формат: MM:SS (минуты:секунды)
 - Таймер работает даже в фоновом режиме (когда приложение свёрнуто)
- Timer Completed:
 - Когда таймер достигает 00:00:
 - Отправляется push-уведомление: "Новые 12 вопросов готовы! 📢"
 - При открытии приложения (тап на уведомление или открытие вручную):
 - Автоматический переход на экран голосования (vote1_screen)
 - Начинается новый раунд с вопросом 1/12
 - Таймер сбрасывается
- Background Mode:
 - Если пользователь закрывает/сворачивает приложение:
 - Таймер продолжает работать в фоне
 - По завершении таймера отправляется push-уведомление
 - При возврате в приложение таймер показывает актуальное оставшееся время
- Referral Bypass:
 - Если пользователь пригласил друга и тот установил приложение:
 - Отправляется push-уведомление: "Ваш друг установил приложение! Вы можете голосовать прямо сейчас."
 - Таймер сбрасывается
 - Автоматический переход на экран голосования (vote1_screen)
 - Начинается новый раунд с вопросом 1/12
- Invite Button Clicked:
 - При нажатии "Пригласить друга":
 - Открывается экран с реферальной ссылкой и инструкцией
 - Пользователь может поделиться ссылкой: WhatsApp, Telegram
 - Tab Navigation:
 - Пользователь может переключаться между табами:
 - Главная (текущий экран) - показывает таймер
 - Поиск - переход на экран поиска
 - Лайки - переход на экран лайков
 - Профиль - переход на экран профиля
 - Ещё - переход на экран дополнительного меню
 - При переключении табов таймер продолжает работать в фоне

Важно:

- ⚠️ Timer countdown: таймер обратный отсчёт от 40:00 до 00:00 (по секундам)
- ⚠️ Real-time update: обновляется каждую секунду (MM:SS формат)
- ⚠️ Background timer: работает в фоновом режиме (даже когда приложение свёрнуто)
- ⚠️ Push notification: при завершении таймера отправляется push "Новые 12 вопросов готовы! 📢"
- ⚠️ Auto-navigation: при тапе на push или открытии приложения после таймера → автоматический переход на vote1_screen
- ⚠️ Referral bypass: если друг установил приложение → таймер сбрасывается, push "Ваш друг установил приложение!", сразу можно голосовать
- ⚠️ Decorative capsules: 2 капсулы сверху - только декоративные элементы (не интерактивны)
- ⚠️ Not a question: "Хочешь попробовать еще раз?" - это просто текст, не интерактивный элемент
- ⚠️ Invite button: единственная интерактивная кнопка - "Пригласить друга"
- ⚠️ Current tab: экран находится в табе "Главная" (активный таб)
- ⚠️ Stars already received: до этого экрана пользователь уже получил 1000 звёздочек(экран star_taking_screen)

- ⚠ Tab bar persistent: нижняя навигация всегда видна на этом экране
- ⚠ Timer persists: при переключении табов таймер не сбрасывается, продолжает работать
- ⚠ Таймер обновляется каждую секунду НА УСТРОЙСТВЕ (локально)
- ⚠ Запросы к серверу: 1-2 за весь период таймера (40 минут)
- ⚠ Нагрузка на сервер: минимальная (~0.05 запросов в минуту на пользователя)
- ⚠ Нагрузка на базу: минимальная (1 SELECT при загрузке)
- ⚠ Потребление памяти: минимальное (~50 bytes на клиенте, 0 KB на сервере)

invite_friend_screen

UI элементы:

- Close Button: "X" (белый крестик, top-left)
- Header Text: Инструкция с промокодом (белый текст на темно-синем фоне):
 - "Пригласите друга — и пропустите 40 минут ожидания!"
 - "Пусть он укажет ваш промокод при регистрации в приложении — иначе мы не узнаем, что он от вас."
 - "Выберите имя и отправьте готовое смс на WhatsApp."
 - "ВАШ ПРОМОКОД: BU87R" (белый текст, bold)
- Share Panel: "Пригласить через" (белая карточка с иконками соцсетей, centered)
 - Icon 1:  Ссылка (серая круглая кнопка)
 - Icon 2:  Чат (зелёная, SMS/iMessage)
 - Icon 3:  Telegram (синяя)
 - Icon 4:  WhatsApp (зелёная)
 - Icon 5:  Instagram Direct (градиент розово-фиолетовый)
- Search Field: "🔍 Поиск" (светло-синий фон, полупрозрачный, rounded)
- Friends List: Прокручиваемый список друзей (white background cards)
 - Андрей Малахов, "5 друзей"
 - Юлия Световая, "4 друга"
 - Oleksandr Shkolnyi, "1 друг"
 - Арлан Каиржанов, "4 друга"
 - Вадим Исаков, "8 друзей"

Валидация:

- Нет валидации - это экран выбора способа приглашения

Ошибки:

- Ошибка загрузки контактов → "Не удалось загрузить контакты. Попробуйте позже"
- Мессенджер не установлен → "[Название] не установлен. Установите приложение или выберите другой способ"
- Ошибка открытия мессенджера → "Не удалось открыть [название]. Попробуйте другой способ"

Состояния:

- Default:
 - Показывается панель "Пригласить через" с 5 иконками соцсетей
 - Ниже - поле поиска
 - Ниже - список друзей из контактов (только незарегистрированные)
 - Промокод отображается в header
 - Icon Clicked (Ссылка  - Текст ссылки: `https://app.example.com/invite?ref=BU87R`
- Icon Clicked (Чат  - SMS/iMessage):
 - Открывается системный SMS picker (выбор контакта)
 - Или если выбран друг из списка → SMS с этим контактом
 - Готовый текст предзаполнен:

Пример предзаполненного сообщения:

(Привет! 🙌)

Скачай крутое приложение для школы:

<https://app.example.com/invite?ref=BU87R>

Введи мой промокод BU87R при регистрации! 🎯

- Пользователь нажимает "Отправить"
- Icon Clicked (Telegram

- Если НЕТ → toast "Telegram не установлен"
- Icon Clicked (WhatsApp ):
 - Проверяется, установлен ли WhatsApp
 - Если ДА → открывается WhatsApp с готовым текстом аналогично
 - Если выбран друг из списка → автоматически открывается чат с ним по номеру телефона
 - Если НЕТ → toast "WhatsApp не установлен"
- Icon Clicked (Instagram Direct ):
 - Проверяется, установлен ли Instagram
 - Если ДА → открывается Instagram Direct с готовым текстом аналогично
 - ⚠ Важно: Instagram поддерживает предзаполнение текста через Instagram URL Scheme
Если НЕТ → toast "Instagram не установлен"
- Friend Card Clicked:
 - При клике на друга из списка (например, "Андрей Малахов"):
 - Запоминается выбранный контакт
 - Пользователь выбирает способ отправки (иконку сверху)
 - Открывается выбранный мессенджер с готовым текстом и автоматически выбранным контактом
- Search Active:
 - Фильтрация списка друзей по имени в реальном времени
 - Если ничего не найдено → "Друзей с таким именем не найдено"

Важно:

- ⚠ Quick share panel: панель "Пригласить через" с 5 иконками соцсетей для быстрого доступа
 - ⚠ Direct open: при клике на иконку соцсети сразу открывается этот мессенджер с готовым текстом
 - ⚠ Pre-filled message: во всех мессенджерах текст предзаполнен (кроме случаев, когда API не поддерживает)
 - ⚠ SMS/iMessage (Чат): иконка "💬 Чат" открывает системный SMS (работает на iOS и Android)
 - ⚠ Instagram Direct поддерживает предзаполнение: через Instagram URL Scheme можно отправить текст (см. код ниже)
 - ⚠ Friend selection: если выбран друг из списка → мессенджер открывается с этим контактом
 - ⚠ Contact source: список друзей из хешированных контактов (только незарегистрированные)
 - ⚠ Mutual friends: отображается количество общих друзей
 - ⚠ Permanent promocode: промокод BU87R(пример) постоянный, можно пригласить неограниченно
 - ⚠ Deep link: промокод встроено в URL ? ref=BU87R
 - ⚠ Close button: крестик закрывает экран и возвращает на timer_screen
 - ⚠ Там где значки соц сетей это скриншот нужно ее фон сделать также синим чтобы сочетался или белым растянуть чтобы выглядело лаконично
- Посмотрите как у snapchat сделано позвать друзей вот эти где 5 иконок соц сетей примерно также чтобы сразу открывал приложение с готовым сообщением

profile_screen

UI элементы:

- Background: Синий градиент фон
- Profile Card: Белая карточка с закруглёнными углами (centered, сверху)
 - Avatar: Круглый аватар с инициалами "МК" (синий градиент фон, белые буквы)
 - Name: "Максим Куприянов" (чёрный текст, крупный, bold)
 - Username: "@maxim_kupriyanov" (серый текст, средний размер)
 - School: "🎓 МГУ" (иконка портфеля + название школы)
 - Grade: "👤 10 класс" (иконка человека + класс)
 - Edit Button: "Изменить" (светло-синяя кнопка, rounded, centered)
 - Stats Row: 3 круглых блока со статистикой (светло-синий фон):
 - Friends: "15 друзей" (кликабельно открывает список друзей и если нажать на человека открывает его профиль)
 - Votes: "2 голоса" (кликабельно - открывает экран лайки)
 - Requests: "5 запросов" (кликабельно) **открывает экран запросы. нужно создать экран с списком запросов и на каждом выбор принять/отклонить**
 - Share Button: "Поделиться профилем" (синяя кнопка, rounded, слева внизу карточки)
 - Collection Button: "Коллекция" (синяя кнопка, rounded, справа внизу карточки)
- Friends Section: "Друзья: 15" (чёрный текст на синем фоне, bold) ← ИЗМЕНЕНО: было "54"
- Friends List: Прокручиваемый список друзей (белые карточки с закруглёнными углами)
 - Friend Card 1: ★ Асем Есенова (только имя, БЕЗ "Общие друзья")
 - Friend Card 2: ★ Ильяс Базарбаев
 - Friend Card 3: ★ Арман Исаков
 - Friend Card 4: ★ Николай Лавров
- Bottom Navigation Bar: 5 табов (Главная, Поиск, Лайки, Профиль, Ещё)

- Tab 1: 🏠 Главная (неактивный - серый)
- Tab 2: 🔍 Поиск (неактивный - серый)
- Tab 3: ❤️ Лайки (неактивный - серый)
- Tab 4: 👤 Профиль (активный - синий) ← АКТИВНЫЙ ТАБ
- Tab 5: ☰ Ещё (неактивный - серый)

Валидация:

- Редактирование профиля:
 - Имя, фамилия, класс, школа можно изменить 1 раз бесплатно после регистрации. То есть как он выбрал на регистрации еще раз один раз может. Но важно предупредить что только один раз в 3 месяца
 - Второе изменение возможно через 3 месяца после первого изменения

Ошибки:

- Ошибка загрузки профиля → "Не удалось загрузить профиль. Попробуйте позже"
- Ошибка сохранения изменений → "Не удалось сохранить изменения. Попробуйте снова"
- Попытка изменить данные до истечения 3 месяцев → "Вы сможете изменить данные через [X дней]"
- Ошибка загрузки друзей → "Не удалось загрузить список друзей"
- Ошибка загрузки запросов → "Не удалось загрузить запросы в друзья"
- Ошибка копирования ссылки → "Не удалось скопировать ссылку"

Состояния:

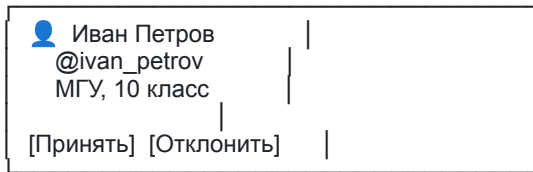
- Loading:
 - Skeleton loader для карточки профиля и списка друзей
- Default:
 - Показывается карточка профиля с аватаром, именем, username, школой, классом
 - Статистика:
 - "15 друзей" (кликабельно - открывает список друзей)
 - "2 голоса" (голоса, которые я получил кликабельно ведет на лайки там видно все запросы)
 - "5 запросов" (кликабельно - открывает экран список входящих запросов в друзья (его нужно создать))
 - Кнопки "Поделиться профилем" и "Коллекция" активны
 - Список друзей: "Друзья: 15" с прокручиваемым списком моих друзей
 - Каждая карточка друга показывает только имя и фамилию + звездочку ★ (БЕЗ "Общие друзья")
 - Bottom Navigation Bar: таб "Профиль" активен (синий)
- Edit Button Clicked:
 - При нажатии "Изменить":
 - Проверяется, можно ли редактировать (прошло ли 3 месяца с последнего изменения)
 - Если ДА → переход на экран редактирования профиля
 - Если НЕТ → показывается alert:
 - Code

"Вы сможете изменить данные через [X дней]"

[OK]

- Share Profile Button Clicked:
 - При нажатии "Поделиться профилем":
 - Генерируется ссылка на профиль:
`https://app.example.com/profile/@maxim_kupriyanov`
 - Ссылка копируется в буфер обмена
 - Показывается toast: "✅ Ссылка на профиль скопирована"
 - Кнопка меняет текст на "Скопировано" на 2 секунды, потом обратно на "Поделиться профилем"
- Collection Button Clicked:
 - При нажатии "Коллекция":
 - Открывается экран коллекции (collection_screen)
- Friends Count Clicked ("15 друзей"):
 - При нажатии на блок "15 друзей":
 - Открывается экран со списком моих друзей (friends_list_screen)
 - Показывается полный список с аватарами, именами, кнопками действий
 - Можно кликнуть на друга → перейти на его профиль
- Requests Count Clicked ("5 запросов"):
 - При нажатии на блок "5 запросов":
 - Открывается экран с входящими запросами в друзья (friend_requests_screen)

- Показывается список пользователей, которые отправили мне запрос
- Можно принять или отклонить каждый запрос
- Пример карточки запроса:
- Code



-
- Votes Count Clicked ("2 голоса"):
 - При нажатии на блок "2 голоса":
 - Ничего не происходит (не кликабельно)
 - Или прокручивается вниз к списку номинаций (если будет добавлено)
- Friend Card Clicked:
 - При клике на друга (например, "Асем Есенова"):
 - Открывается профиль этого друга (profile_forpeople_screen)
 - Без возможности писать сообщения (в приложении НЕТ переписок/чата)
- Stats Explanation:
 - "15 друзей": Количество моих друзей в приложении (принятые запросы)
 - "2 голоса": Количество голосов, которые я получил от других пользователей
 - "5 запросов": Количество входящих запросов в друзья, ожидающих принятия/отклонения
- Navigation:
 - Главная (🏠): Открывается главный экран
 - Поиск (🔍): Открывается экран поиска
 - Лайки (❤️): Открывается экран лайков
 - Профиль (👤): Текущий экран (активный таб - синий)
 - Ещё (☰): Открывается экран дополнительного меню

Важно:

- ⚠️ This is MY profile: это экран моего собственного профиля
- ⚠️ My friends: "15 друзей" = количество моих друзей в приложении (принятые запросы)
- ⚠️ Votes received: "2 голоса" = количество голосов, которые я получил от других
- ⚠️ Friend requests: "5 запросов" = количество входящих запросов в друзья (pending)
- ⚠️ Requests clickable: блок "5 запросов" кликабельный → открывает экран с запросами
- ⚠️ Friends clickable: блок "15 друзей" кликабельный → открывает полный список друзей
- ⚠️ No mutual friends shown: в списке друзей НЕТ "Общие друзья: X" (только имя + звёздочка)
- ⚠️ Edit restrictions: можно изменить данные 1 раз бесплатно, второй раз через 3 месяца
- ⚠️ Share profile: кнопка "Поделиться профилем" копирует ссылку в буфер обмена
- ⚠️ No chat/messages: в приложении НЕТ переписок/чата
- ⚠️ Collection: "Коллекция" → открывается экран со звёздочками, полученными за раунды
- ⚠️ Star icon: ★ = это просто декорация ничего не значит у всех так
- ⚠️ Active tab: таб "Профиль" активен (синий цвет)
- ⚠️ Bottom nav persistent: нижняя навигация всегда видна на этом экране
- ⚠️ Friend profile: при клике на друга открывается его профиль (profile_forpeople_screen)

profile_for_people_screen

UI элементы:

- Background: Синий градиент фон
- Profile Card: Белая карточка с закруглёнными углами (centered, сверху)
 - Avatar: Круглый аватар с инициалами "МК" (синий фон, белые буквы) (тут может быть или инициалы или фотка)
 - Name: "Антон Ловченко" (чёрный текст, крупный, bold)
 - School: "🎒 МГУ" (иконка портфеля + название школы)
 - Grade: "👤 10 класс" (иконка человека + класс)
 - Username: "@anton_lovchenko" (серый текст, средний размер)
 - Stats Row: 3 круглых блока со статистикой (светло-синий фон):
 - Friends: "54 друзей" (кликабельно) открывает экран где список друзей
 - Votes: "12 голосов"
 - Actions Menu: ".." (три точки - открывает меню действий)

- Action Button: Динамическая кнопка (синяя, rounded, меняется в зависимости от статуса дружбы):
 - "Добавить в друзья" (если НЕ друг)
 - "Запрос отправлен" (если запрос отправлен, неактивная/серая)
 - "Удалить друга" (если УЖЕ друг)
- Votes Section: "Голосов: 12" (белый текст на синем фоне, bold)
- Votes List: Прокручиваемый список номинаций (белые карточки с закруглёнными углами)
 - Vote Card 1: ★ "Выбран в голосовании 'Самый умный'"
 - Vote Card 2: ★ "Выбран в голосовании 'Самый внимательный'"
 - Vote Card 3: ★ "Выбран в голосовании 'Похож на Илона Маска'"
 - Bottom Navigation Bar: 5 табов (Главная, Поиск, Лайки, Профиль, Ещё)
 - Tab 1: 🏠 Главная (неактивный - серый)
 - Tab 2: 🔍 Поиск (неактивный - серый)
 - Tab 3: ❤️ Лайки (неактивный - серый)
 - Tab 4: 👤 Профиль (неактивный - серый)
 - Tab 5: ☰ Ещё (неактивный - серый)

Валидация:

- Нет валидации - это экран просмотра чужого профиля

Ошибки:

- Ошибка загрузки профиля → "Не удалось загрузить профиль. Попробуйте позже"
- Пользователь заблокировал вас → "Профиль недоступен"
- Ошибка отправки запроса в друзья → "Не удалось отправить запрос. Попробуйте снова"
- Ошибка удаления из друзей → "Не удалось удалить из друзей. Попробуйте снова"
- Ошибка блокировки → "Не удалось заблокировать пользователя"
- Ошибка отправки жалобы → "Не удалось отправить жалобу. Попробуйте позже"

Состояния:

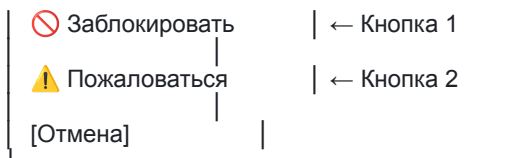
- Loading:
 - Skeleton loader для карточки профиля и списка номинаций
- Default (НЕ друг):
 - Показывается карточка профиля с аватаром, именем, username, школой, классом
 - Статистика: "54 друзей" (кликабельно), "12 голосов" (не кликабельно так как внизу уже видно в каких опросах его выбрали), ".." (меню)
 - Кнопка: "Добавить в друзья" (синяя, активная)
 - Список номинаций: "Голосов: 12" с карточками голосований
 - Все карточки имеют звёздочку ★ (декоративный элемент)
- Friend Request Sent (запрос отправлен):
 - Кнопка меняется на: "Запрос отправлен" (серая, неактивная, disabled)
 - Пользователь не может ничего сделать, пока запрос не будет принят/отклонён
- Already Friend (УЖЕ друг):
 - Кнопка меняется на: "Удалить друга" (синяя, активная)
- Blocked By User (заблокирован этим пользователем):
 - Показывается экран-заглушка: "Профиль недоступен"
 - Пользователь НЕ видит профиль, статистику, номинации - ничего
- Add Friend Button Clicked:
 - При нажатии "Добавить в друзья":
 - Отправляется запрос на добавление в друзья
 - Кнопка меняется на "Запрос отправлен" (серая, disabled)
 - У Антона в разделе "Запросы в друзья" появляется уведомление
 - Антон может принять или отклонить запрос
 - Если принят → кнопка меняется на "Удалить друга"
 - Если отклонён → кнопка возвращается к "Добавить в друзья"
- Remove Friend Button Clicked:
 - При нажатии "Удалить друга":
 - Показывается confirmation alert:

"Вы уверены, что хотите удалить из друзей?"

[Удалить] [Отмена]

- Если "Удалить" → пользователь удаляется из друзей
 - Кнопка меняется обратно на "Добавить в друзья"
 - Toast: "✔ Пользователь удалён из друзей"
- Three Dots Menu Clicked (..):
 - При нажатии на ".." открывается bottom sheet или modal с двумя опциями:
 - Вариант 1: Пользователь НЕ заблокирован





- Вариант 2: Пользователь УЖЕ заблокирован
- Code



- Block User (Заблокировать):

- При нажатии "Заблокировать":

- Показывается confirmation alert:

"Вы уверены, что хотите заблокировать этого пользователя?"

[Заблокировать] [Отмена]

- Если "Заблокировать":

- Пользователь блокируется
- Если были друзьями → автоматически удаляется из друзей
- Заблокированный пользователь НЕ видит ваш профиль
- Вы НЕ видите его в поиске он вас тоже не видит и его профиль вы тоже не видите
- Toast: "✅ Пользователь заблокирован"

- Кнопка в меню меняется на "Разблокировать"

- Unblock User (Разблокировать):

- При нажатии "Разблокировать":

- Пользователь разблокируется
- Toast: "✅ Пользователь разблокирован"
- Кнопка в меню меняется обратно на "Заблокировать"

- Report User (Пожаловаться):

- При нажатии "Пожаловаться":

- Открывается стандартная почтовая форма (системное приложение Email/Gmail)
- Email предзаполнен:
 - To: support@yourapp.com (ваша почта поддержки)
 - Subject: "Жалоба на пользователя @anton_lovchenko"
 - Body:

Здравствуйте,

Я хочу пожаловаться на пользователя:

Имя: Антон Ловченко

Username: @anton_lovchenko

ID: [user_id]

Причина жалобы:

[Пользователь заполняет сам]

С уважением,

[Ваше имя]

■

- Пользователь может редактировать текст и отправить

- Friends Count Clicked (54 друзей):

- При нажатии на блок "54 друзей":

- Открывается экран со списком друзей этого пользователя (Антон)
- Показывается список с аватарами, именами, общими друзьями
- Можно кликнуть на друга → перейти на его профиль

- Votes Count Clicked (12 голосов):

- При нажатии на блок "12 голосов":

- Ничего не происходит (не кликабельно)

- Или прокручивается вниз к списку номинаций
- Vote Card Clicked:
 - При нажатии на карточку номинации (например, "Самый умный"):
 - Ничего не происходит (не кликабельно, просто информация)
- Votes Logic (важно!):
 - "12 голосов" = Антон получил 12 голосов в разных номинациях
 - Карточки показывают ВОПРОСЫ, а не количество голосов
 - Пример:
 - Вопрос "Кто самый умный?" → за Антона проголосовало 5 человек → показывается 1 карточка: "Выбран в голосовании 'Самый умный' (5 раз)"
 - Вопрос "Кто самый красивый?" → за Антона проголосовало 3 человека → показывается 1 карточка: "Выбран в голосовании 'Самый красивый' (3 раза)"
 - Вопрос "Похож на Илона Маска?" → за Антона проголосовало 4 человека → показывается 1 карточка: "Выбран в голосовании 'Похож на Илона Маска' (4 раза)"
 - Итого: 5 + 3 + 4 = 12 голосов, но 3 карточки (по количеству вопросов), то есть они видят общую и количество в голосовании пишется общая от всех голосов включая сколько раз
 - Все видят:
 - Название вопроса/номинации
 - Сколько раз Антона выбрали в этом вопросе (5 раз, 3 раза, etc.)
 - Никто НЕ видит (платная функция "раскрытие"):
 - Кто именно голосовал за Антона (имена людей скрыты)

Важно:

- ⚠ Other user's profile: это профиль другого пользователя (Антон), не мой
- ⚠ Dynamic button: кнопка меняется в зависимости от статуса дружбы
- ⚠ Friend request system: есть система запросов в друзья (не мгновенное добавление)
- ⚠ Confirmation on remove: при удалении друга показывается confirmation alert
- ⚠ Actions menu: "... " открывает меню с блокировкой и жалобой
- ⚠ Block functionality: блокировка скрывает профиль полностью (для обеих сторон)
- ⚠ Report via email: жалоба отправляется через стандартную почтовую форму
- ⚠ Friends list: список друзей скрыт, открывается при клике на "54 друзей"
- ⚠ Public profile: профиль виден всем (если не заблокирован)
- ⚠ Votes by question: карточки показывают вопросы, а не отдельные голоса не видно кто именно голосовал
- ⚠ Vote count visible: количество голосов видно всем (например, "5 раз")
- ⚠ Voters hidden: имена голосовавших скрыты (платная функция "раскрытие")
- ⚠ Non-clickable cards: карточки номинаций не кликабельны (просто информация)
- ⚠ Star is decorative: звёздочка ★ у всех карточек (декоративный элемент)
- ⚠ No chat: в приложении НЕТ переписок/чата
- ⚠ Bottom nav visible: нижняя навигация всегда видна

edit_profile_screen

UI элементы:

- Background: Синий градиент фон (верхняя часть)
- Header: "Редактировать профиль" (белый текст, centered, крупный, bold)
- Close Button: X (белый крестик, top-left)
- Avatar Section:
 - Avatar: Круглый аватар с placeholder силуэтом (светло-синий фон)
 - Camera Icon: 📷 Иконка камеры в белом круге (bottom-right от аватара)
 - Change Photo Button: "Изменить фото" (белая кнопка с полупрозрачным фоном, rounded)
- Form Fields: (белый фон, список полей)
 - Name Field: "Имя" (серый label) → "Максим" (чёрный текст, editable)
 - Surname Field: "Фамилия" (серый label) → "Яковлев" (чёрный текст, editable)
 - Username Field: "Логин" (серый label) → "maxim_yakovlev" (чёрный текст, editable)
 - Gender Field: "Пол" (серый label) → "Мужской" (синий текст, clickable, стрелка →, НЕ редактируемый)
- Data Section: "ДАННЫЕ" (серый заголовок, uppercase)
 - School Row: 🏫 "Назарбаевская школа Астана", "Изменить" (серый текст, clickable, стрелка →)
 - Grade Row: 🎓 "10 класс", "Изменить" (серый текст, clickable, стрелка →)
- Save Button: "Сохранить" (синяя кнопка, rounded, внизу экрана) ← ДОБАВЛЕНО
- Bottom Navigation Bar: 5 табов (Главная, Поиск, Лайки, Профиль, Ещё)

Валидация:

- Все табы видны но неактивны (серые)

- Имя:
 - Минимум: 2 символа
 - Максимум: 50 символов
 - Разрешено: кириллица, латиница, дефис, пробел
 - Пример валидного: "Максим", "Мария-Анна", "John"
 - Ошибка: "Имя должно содержать от 2 до 50 символов"
- Фамилия:
 - Минимум: 2 символа
 - Максимум: 50 символов
 - Разрешено: кириллица, латиница, дефис, пробел
 - Пример валидного: "Яковлев", "Петров-Водкин", "Smith"
 - Ошибка: "Фамилия должна содержать от 2 до 50 символов"
- Логин (username):
 - Минимум: 3 символа
 - Максимум: 30 символов
 - Разрешено: только строчная латиница (a-z), цифры (0-9), подчёркивание (_)
 - Запрещено: заглавные буквы, кириллица, спецсимволы, пробелы
 - Обязательная проверка на уникальность (при сохранении)
 - Пример валидного: "maxim_yakovlev", "user123", "cool_guy"
 - Ошибка: "Логин может содержать только строчные латинские буквы, цифры и подчёркивание"
 - Ошибка: "Этот логин уже занят"
- Фото:
 - Максимальный размер: неограничен ток не загружать сервер, сжатие делать на стороне клиента (после сжатия на сервер). Также чтобы открывалось потом в полноэкранном формате квадрата
 - Формат: JPG, PNG
 - Автоматическое сжатие на клиенте (чтобы не загружать сервер)
 - Обрезка в квадрат (1:1 aspect ratio) автоматически
- Пол:
 - НЕ редактируемый (установлен при регистрации, изменить НЕЛЬЗЯ)
 - При попытке изменить → показывается alert: "Пол нельзя изменить"
- Школа и Класс:
 - Можно изменить 1 раз
 - Повторное изменение: через 3 месяца после первого изменения
 - Ошибка: "Вы сможете изменить школу/класс через [X дней]"

Ограничения редактирования:

⚠️ **ВАЖНО: Каждое поле считается ОТДЕЛЬНО! **

Поле	Лимит	Cooldown (ожидание)	Примечание
Имя	1 раз	3 месяца	Каждое изменение = 1 раз
Фамилия	1 раз	3 месяца	Каждое изменение = 1 раз
Логин	1 раз	3 месяца	Каждое изменение = 1 раз
Пол	0 раз	Навсегда	НЕЛЬЗЯ изменить
Школа	1 раз	3 месяца	1 раз (но нужно предпринять когда будете менять текстом что только один раз)
Класс	1 раз	3 месяца	1 раз (но нужно предпринять когда будете менять текстом что только один раз)

Пример сценария:

- День 1: Изменил имя "Максим" → "Макс" (1-й раз, осталось 0)
- День 2: Попытка изменить имя → ❌ Ошибка "Вы сможете изменить имя через 89 дней"
- День 2: Изменил фамилию "Яковлев" → "Петров" (1-й раз для фамилии, осталось 0)
- День 3: Изменил фото (1-й раз, осталось 3)

- День 90: Можно изменить имя снова (прошло 3 месяца)

Ошибки:

- Имя/Фамилия слишком короткие → "Имя/Фамилия должны содержать минимум 2 символа"
- Имя/Фамилия слишком длинные → "Имя/Фамилия не должны превышать 50 символов"
- Логин недопустимые символы → "Логин может содержать только строчные латинские буквы, цифры и подчёркивание"
- Логин занят → "Этот логин уже занят. Выберите другой"
- Логин слишком короткий → "Логин должен содержать минимум 3 символа"
- Логин слишком длинный → "Логин не должен превышать 30 символов"
- Фото неподдерживаемый формат → "Поддерживаются только JPG и PNG"
- Исчерпан лимит изменений → "Вы сможете изменить [поле] через [X дней]"
- Исчерпан лимит фото → "Вы исчерпали лимит изменений фото (4 раза)"
- Ошибка сохранения → "Не удалось сохранить изменения. Попробуйте снова"

Состояния:

- Loading:
 - Skeleton loader для полей формы при загрузке
- Default:
 - Показываются текущие данные профиля (имя, фамилия, логин, школа, класс, фото)
 - Все поля редактируемые (кроме "Пол")
 - Кнопка "Сохранить" неактивна/серая (пока не внесены изменения)
- Field Editing:
 - При изменении любого поля:
 - Кнопка "Сохранить" становится активной/синей
 - Поле подсвечивается (опционально)
- Change Photo Button Clicked:
 - При нажатии "Изменить фото":
 - Фото можно менять ток 15 раз
 - Если лимит есть → открывается выбор источника:
 -  Сделать фото (камера)
 -  Выбрать из галереи
 -  Отмена
 - После выбора фото:
 - Обрезка Сжатие до 250 KB (на клиенте, чтобы не нагружать сервер)
 - Фото отображается в аватаре
 - Кнопка "Сохранить" активируется даже после одного изменения но предупреждает что можно ток один раз менять данные. фото предупреждать не нужнр
 - Лимит изменений уменьшается только после сохранения
- Avatar Clicked (на экране профиля):
 - При нажатии на аватар на экране profile_screen:
 - Открывается полноэкранный просмотр фото (fullscreen image viewer)
 - Фото показывается в квадратном формате (1:1)
 - Можно закрыть (крестик или swipe down)
 - Можно зумить (pinch to zoom)
- Blocked List Clicked:
 - При нажатии на "Раскрытия":
 - Нужно создать экран с количеством простой Пример: "Осталось 5 раскрытий полного имени, Осталось 2 раскрытия первой буквы имени"
 - Это данные для тех кто купил подписку Premium чтобы они могли видеть сколько осталось
 - Можно вернуться назад → edit_profile_screen
- Gender Field Clicked:
 - При нажатии на "Пол":
 - Показывается alert: "Пол нельзя изменить"
 - Или ничего не происходит (поле disabled)
- School Row Clicked:
 - При нажатии на "🎓 Назарбаевская школа Астана, Изменить":
 - Проверяется можно ли редактировать (прошло 3 месяца?)
 - Если ДА → открывается экран выбора школы (school_selection_screen)
 - Если НЕТ → показывается alert: "Вы сможете изменить школу через [X дней]"
- Grade Row Clicked:
 - При нажатии на "👤 10 класс, Изменить":
 - Проверяется можно ли редактировать (прошло 3 месяца?)
 - Если ДА → открывается picker/dropdown с выбором класса (1-11 класс)

- Если НЕТ → показывается alert: "Вы сможете изменить класс через [X дней]"
 - Save Button Clicked:
 - При нажатии "Сохранить":
 - Валидация всех изменённых полей
 - Если есть ошибки → показываются под полями (красный текст)
 - Если логин изменён → проверка на уникальность (API запрос)
 - Если всё ОК → отправка данных на сервер
 - Показывается loading indicator на кнопке
 - Если успешно:
 - Toast: "✅ Профиль обновлён"
 - Возврат на profile_screen с обновлёнными данными
 - Обновление лимитов (счётчик изменений)
 - Если ошибка:
 - Toast: "❌ Не удалось сохранить изменения"
 - Close Button (X) Clicked:
 - При нажатии на крестик:
 - Если есть несохранённые изменения → показывается confirmation alert:
 - Code
- "Сохранить изменения?"
 [Сохранить] [Не сохранять] [Отмена]
- - Если нет изменений → просто закрывается экран, возврат на profile_screen
 - Username Real-time Validation:
 - При вводе логина (в реальном времени):
 - Проверка формата (только строчные латинские + цифры + подчёркивание)
 - Показывается ошибка под полем (красный текст)
 - Debounce 500ms → проверка на уникальность (API запрос)
 - Если занят → "❌ Этот логин уже занят"
 - Если свободен → "✅ Логин доступен" (зелёный текст)

Важно:

- ⚠ Edit limits per field: каждое поле считается ОТДЕЛЬНО (имя - 1 раз, фамилия - 1 раз, логин - 1 раз)
 Даже если один критерий изменил кнопка сохранить становится активной
- ⚠ Photo limit: фото можно изменить только 4 раза (затем навсегда заблокировано)
- ⚠ Gender locked: пол НЕЛЬЗЯ изменить никогда
- ⚠ School/Grade cooldown: школа и класс можно изменить 1 раз, затем через 3 месяца
- ⚠ Photo compression: фото сжимается на клиенте до 250 KB (не нагружает сервер)
- ⚠ Square crop: фото обрезается в квадрат (1:1) автоматически
- ⚠ Fullscreen view: на profile_screen можно нажать на аватар → открывается полноэкранный просмотр
- ⚠ Username validation: логин проверяется на уникальность (real-time с debounce)
- ⚠ Lowercase only: логин только строчные латинские буквы
- ⚠ Save button: кнопка "Сохранить" активируется при изменении полей
- ⚠ Unsaved changes: при закрытии с несохранёнными изменениями → confirmation alert
- ⚠ Blocked list: "Заблокированные" открывает отдельный экран со списком
- ⚠ No bottom nav interaction: нижняя навигация вид

Экраны Настроек

collection_screen (внутриигровые покупки без реальных денег)

UI элементы:

- Background: Темно-синий градиент фон
- Header Card: Белая карточка с закруглёнными углами (top)
 - Close Button: X (черный крестик, left)
 - Title: "ВНУТРИ ИГРОВЫЕ БАЛЛЫ" (черный текст, крупный, bold, centered)
- Balance Section: "ВАШ БАЛАНС" (серый текст, uppercase, centered)
 - Balance Display: ★ "1000" (крупные белые цифры с иконкой звёздочки)
- Promotion Section:
 - Section Title: "Продвинь себя" (белый текст, крупный, centered)
 - Section Subtitle: "Используй звёздочки для продвижения" (серый текст, средний, centered)
- Boosts List: 2 карточки с бустами (темно-серые карточки с закруглёнными углами)
 - Boost Card 1:
 - Иконка ★ (слева, в белом квадрате с закруглёнными углами)
 - Текст: "Поставить свое имя в 4 голосованиях" (белый, 2 строки)
 - Цена: "5000" (синяя кнопка, справа, крупные цифры)
 - Boost Card 2:
 - Иконка ★ (слева, в белом квадрате)
 - Текст: "Поставить имя в вопрос про симпатию" (белый, 2 строки)
 - Цена: "20000" (синяя кнопка, справа, крупные цифры)
- Info Section:
 - Info Title: "Как получить звёздочки?" (белый текст, крупный, centered)
 - Info Text: "Участвуйте больше в опросах" (серый текст, средний, centered)

Валидация:

- Нет валидации - это экран просмотра и покупки бустов

Ошибки:

- Недостаточно звёздочек → "Недостаточно звездочек" (кнопка покупки неактивна/серая)
- Ошибка покупки → "Не удалось купить буст. Попробуйте снова"
- Ошибка загрузки баланса → "Не удалось загрузить баланс. Попробуйте позже"

Состояния:

- Loading:
 - Skeleton loader для баланса и карточек бустов
- Default (достаточно звёздочек):
 - Показывается баланс: "★ 1000"
 - Карточки бустов с ценами
 - Если баланс $\geq$ цены буста → кнопка активна/синяя
 - Если баланс $<$ цены буста → кнопка неактивна/серая + текст "Недостаточно звездочек" (под кнопкой или toast)
- Insufficient Balance (недостаточно звёздочек):
 - Пример: баланс 1000, цена буста 5000
 - Кнопка покупки неактивна/серая
 - При попытке нажать → toast: "Недостаточно звездочек"
- Boost 1 Purchase ("Поставить свое имя в 4 голосованиях"):
 - При нажатии кнопки "5000":
 - Проверяется баланс (≥ 5000 ?)
 - Если НЕТ → toast "Недостаточно звездочек"
 - Если ДА → показывается confirmation dialog:
 - Code

"Вы уверены, что хотите купить буст 'Поставить свое имя в 4 голосованиях' за 5000 звёздочек?"
[Купить] [Отмена]

-
- После подтверждения:
 - Списывается 5000 звёздочек с баланса
 - Активируется буст СРАЗУ (работает с текущего момента)
 - Сервер случайно добавляет имя пользователя в 4 вопроса (с учётом алгоритма "Лепесток")

- Пользователь НЕ видит, в какие вопросы добавлено его имя (это видят другие пользователи при голосовании)
 - Toast: "✅ Буст куплен! Ваше имя появится в 4 случайных вопросах"
 - Баланс обновляется: $1000 \rightarrow (1000 - 5000) = -4000$ (если было недостаточно) или правильный расчёт
 - Это одноразовая покупка → работает для 4 вопросов → затем нужно покупать снова
- Boost 2 Purchase ("Поставить имя в вопрос про симпатию"):
 - При нажатии кнопки "20000":
 - Проверяется баланс (≥ 20000 ?)
 - Если НЕТ → toast "Недостаточно звездочек"
 - Если ДА → показывается confirmation dialog:
 - Code

"Вы уверены, что хотите купить буст 'Поставить имя в вопрос про симпатию' за 20000 звёздочек?"
[Купить] [Отмена]

- После подтверждения:
 - Списывается 20000 звёздочек
 - Активируется сразу
 - Сервер добавляет имя пользователя в 1 вопрос категории "Симпатия"(рандомно, с учётом алгоритма "Лепесток")
 - Категория "Симпатия" включает вопросы типа: "Кто самый красивый?", "Кто тебе нравится?", "С кем хочешь дружить?" (4 вопроса из 12)
 - Toast: "✅ Буст куплен! Ваше имя появится в вопросе про симпатию"
 - Баланс обновляется
 - Это одноразовая покупка → работает для 1 вопроса → затем нужно покупать снова
- Multiple Boosts Purchase (одновременно):
 - Пользователь может купить оба буста, если хватает звёздочек
 - Пример: баланс 30000
 - Покупает буст 1 (5000) → баланс 25000
 - Покупает буст 2 (20000) → баланс 5000
 - Бусты суммируются: имя появится в 4 обычных вопросах + 1 вопрос про симпатию = 5 вопросов
- Close Button (X) Clicked:
 - Закрывается экран, возврат на предыдущий экран (profile_screen)
- Balance Update:
 - Баланс звёздочек обновляется в реальном времени после покупки
 - Если баланс стал меньше цены буста → кнопка становится неактивной/серой
- Info Section:
 - "Как получить звездочки?" → просто информационный текст (не кликабельно)
 - "Участвуйте больше в опросах" → объясняет, что звёздочки даются только за прохождение раундов голосований (1000 ★ за раунд) (не кликабельно просто текст)

Важно:

- ⚠ Stars currency: звёздочки ★ = внутриигровая валюта (не реальные деньги)
- ⚠ Stars from voting only: звёздочки можно получить только за раунды голосований (1000 ★ за раунд)
- ⚠ Stars never expire: звёздочки остаются навсегда (не сгорают)
- ⚠ Boost 1: 4 random questions: буст добавляет имя в 4 случайных вопроса (любая категория кроме симпатии (в симпатию не добавляет))
- ⚠ Boost 2: 1 sympathy question: буст добавляет имя в 1 вопрос категории "Симпатия"
- ⚠ Question categories: всего 12 вопросов в раунде: 4 обычные + 4 юмор + 4 симпатия
- ⚠ Lepestok algorithm: сервер использует алгоритм "Лепесток" при добавлении имён (равномерное распределение?)
- ⚠ User doesn't see: пользователь НЕ видит, в какие вопросы добавлено его имя (видят только другие при голосовании)
- ⚠ One-time purchase: бусты одноразовые → работают 1 раз → нужно покупать снова
- ⚠ Instant activation: бусты активируются сразу после покупки (не в следующем раунде)
- ⚠ Multiple boosts: можно купить оба буста одновременно (если хватает звёздочек)
- ⚠ Insufficient balance: если звёздочек меньше цены → кнопка неактивна/серая + toast "Недостаточно звездочек"
- ⚠ Confirmation required: перед покупкой показывается confirmation dialog
- ⚠ No IAP: НЕТ покупок за реальные деньги на этом экране (это другой экран)
- ⚠ Info text only: секции "Продвинь себя" и "Как получить звездочки?" = просто информационный текст (не функционал)

friend_requests_screen

UI элементы:

- Background: Светло-серый фон
- Header: "ЗАПРОСЫ ОТ ДРУЗЕЙ: 8" (черный текст, крупный, bold, top-left)
 - Счётчик показывает общее количество запросов (8)
- Requests List: Прокручиваемый список карточек с запросами (белый/прозрачный фон)
 - Request Card 1:
 - Аватар (круглое фото)
 - Имя: "Ethan Carter" (черный, bold)
 - Общие друзья: "1 общий друг" (синий текст, small)
 - Кнопка "Скрыть" (серая, rounded, left)
 - Кнопка "Добавить" (синяя, rounded, right)
 - Request Card 2: "Sophia Bennett", "2 общих друга"
 - Request Card 3: "Liam Harper", "3 общих друга"
 - Request Card 4: "Арман Исаков", "3 общих друга"
 - Request Card 5: "Сергей Иванов", "3 общих друга"
 - Request Card 6: "Мария Кравец", "3 общих друга"
 - Request Card 7: "Leonardo Di", "3 общих друга"
 - Request Card 8: "No Name", "3 общих друга"
- Infinite Scroll: Автоматическая подгрузка запросов при прокрутке вниз

Валидация:

- Нет валидации - это экран просмотра и обработки запросов

Ошибки:

- Ошибка загрузки запросов → "Не удалось загрузить запросы. Попробуйте позже"
- Ошибка принятия запроса → "Не удалось добавить в друзья. Попробуйте снова"
- Ошибка скрытия запроса → "Не удалось скрыть запрос. Попробуйте снова"
- Пустой список → "У вас нет запросов в друзья" (centered text + иллюстрация)

Состояния:

- Loading:
 - Skeleton loader для карточек запросов при первой загрузке
- Default:
 - Показывается список входящих запросов в друзья
 - Заголовок показывает общее количество запросов: "ЗАПРОСЫ ОТ ДРУЗЕЙ: 8"
 - Каждая карточка содержит: аватар, имя, количество общих друзей, кнопки "Скрыть" и "Добавить"
 - Список прокручиваемый (scroll)
 - При прокрутке вниз автоматически подгружаются следующие запросы (infinite scroll, по 10 запросов за раз)
- Pull to Refresh:
 - Пользователь тянет вниз список → показывается индикатор загрузки (spinner)
 - Обновляется список запросов (новые запросы появляются сверху)
 - Счётчик в заголовке обновляется (например, "8" → "10")
 - После обновления → индикатор исчезает
- Hide Button Clicked ("Скрыть"):
 - При нажатии "Скрыть" (например, на карточке "Ethan Carter"):
 - а. Карточка исчезает из списка (анимация slide out)
 - б. Запрос НЕ удаляется полностью, а скрывается
 - в. Через некоторое время (например, через 24 часа или при следующем обновлении) запрос появится снова, но в конце списка
 - г. Отправитель запроса (Ethan) НЕ узнаёт, что его запрос скрыт (нет уведомления)
 - д. Счётчик в заголовке НЕ уменьшается (запрос скрыт, но не отклонён): "ЗАПРОСЫ ОТ ДРУЗЕЙ: 8"
 - е. Счётчик "5 запросов" на profile_screen НЕ уменьшается
 - г. Опционально: показывается undo toast на 3-5 секунд:
 - h. Code

"Запрос скрыт [Отменить]"

- и. Если нажать "Отменить" → карточка возвращается на место
- Accept Button Clicked ("Добавить"):
 - При нажатии "Добавить" (например, на карточке "Sophia Bennett"):
 - а. Карточка исчезает из списка (анимация slide out)
 - б. Пользователь добавляется в друзья (обе стороны становятся друзьями)

- c. Показывается toast: "✅ Sophia Bennett добавлена в друзья"
- d. Счётчик в заголовке уменьшается: "ЗАПРОСЫ ОТ ДРУЗЕЙ: 8" → "ЗАПРОСЫ ОТ ДРУЗЕЙ: 7"
- e. Счётчик "5 запросов" на profile_screen уменьшается → "4 запроса"
- f. Счётчик "15 друзей" на profile_screen увеличивается → "16 друзей"
- g. Отправитель запроса (Sophia) получает push-уведомление:
- h. Code

"Ваш запрос на дружбу принят"

или

"[имя] принял(а) ваш запрос в друзья"

- i. Запрос удаляется из списка навсегда
- Infinite Scroll (автоподгрузка):
 - При прокрутке списка до конца (остаётся 3-5 карточек):
 - a. Автоматически загружаются следующие 10 запросов
 - b. Показывается небольшой loading indicator внизу списка
 - c. Новые карточки добавляются в конец списка
 - d. Если запросов больше нет → loading indicator исчезает
- Empty State (нет запросов):
 - Если список запросов пустой:
 - a. Заголовок меняется на: "ЗАПРОСЫ ОТ ДРУЗЕЙ: 0"
 - b. Показывается centered текст: "У вас нет запросов в друзья"
 - c. Опционально: иллюстрация или иконка 👤
 - d. Кнопка "Назад" или автоматический возврат на profile_screen
- Push Notification Received:
 - Когда кто-то отправляет запрос в друзья:
 - a. Пользователь получает push-уведомление:
 - b. Code

"Иван Петров хочет добавить вас в друзья"

- c. При клике на уведомление → открывается этот экран(Friend_Requests_Screen)
- d. Счётчик в заголовке увеличивается: "ЗАПРОСЫ ОТ ДРУЗЕЙ: 8" → "ЗАПРОСЫ ОТ ДРУЗЕЙ: 9"
- e. Счётчик "5 запросов" на profile_screen увеличивается → "6 запросов"
- Navigation:
 - Открытие экрана: клик на блок "5 запросов" на profile_screen
 - Закрытие экрана: кнопка "Назад" (стрелка ← top-left) или swipe gesture → возврат на profile_screen
 - При возврате счётчик запросов обновляется (если были приняты/скрыты запросы)

Важно:

- ⚠ Screen name: Friend_Requests_Screen
- ⚠ Header counter: заголовок показывает общее количество запросов: "ЗАПРОСЫ ОТ ДРУЗЕЙ: 8"
- ⚠ Incoming requests: это входящие запросы (другие пользователи отправили мне)
- ⚠ Mutual friends: "X общих друзей" = количество общих друзей в приложении (не контакты)
- ⚠ Hide ≠ Decline: кнопка "Скрыть" НЕ отклоняет запрос, а временно скрывает (появится снова в конце списка)
- ⚠ Sender not notified: отправитель НЕ узнаёт о скрытии запроса (нет уведомления)
- ⚠ Counter on hide: счётчик НЕ уменьшается при скрытии (запрос всё ещё существует)
- ⚠ Accept adds to friends: кнопка "Добавить" принимает запрос и добавляет в друзья
- ⚠ Counter on accept: счётчик уменьшается при принятии: "8" → "7"
- ⚠ Push notification on accept: отправитель получает push "Ваш запрос на дружбу принят"
- ⚠ Push on new request: при получении нового запроса → push-уведомление "Иван хочет добавить вас в друзья"
- ⚠ Notification opens screen: клик на push → открывается Friend_Requests_Screen
- ⚠ Counter updates: счётчики обновляются при принятии/скрытии (в заголовке и на profile_screen)
- ⚠ No real-time: список НЕ обновляется в реальном времени
- ⚠ Undo option: опционально показывается "Отменить" toast после скрытия (на 3-5 секунд)

Функция "Ещё 30" (Efficient List Loading)

Принцип работы:

- При загрузке экрана сервер возвращает ВСЕ данные сразу (например, 200 элементов) в одном запросе
- Клиент сохраняет все данные в памяти
- Показываются первые 5-10 элементов
- Внизу отображается счётчик: "Ещё X" (где X = оставшиеся элементы)

- При прокрутке вниз → подгружаются следующие 15-20 элементов из памяти (без запроса к серверу)
- Счётчик обновляется автоматически

Преимущества:

-  1 запрос к серверу вместо 10-20 (минимальная нагрузка)
-  Мгновенная прокрутка (данные в памяти, нет задержек)
-  Простая реализация (не нужна сложная пагинация)

search_screen

UI элементы:

- Background: Светло-серый фон
- Header: "Поиск" (черный текст, крупный, bold, centered)
- Search Field:
 - Иконка  (слева)
 - Placeholder: "Введите имя" (серый текст)
 - Clear Button: X (справа, появляется при вводе текста)
 - Светло-серый фон, rounded
- Suggestions Section (по умолчанию, без поиска):
 - Section 1: "Возможные знакомства" (черный текст, bold)
 - Первые 5 карточек пользователей (аватар, имя, "3 общих друга", кнопки "Скрыть" и "Добавить")
 - "Ещё 30" (серый текст, внизу секции) ← счётчик оставшихся элементов
 - Section 2: "С одной школы" (черный текст, bold)
 - Первые 5 карточек пользователей (аватар, имя, "11 Класс", кнопки "Скрыть" и "Добавить")
 - "Ещё 30" (серый текст, внизу секции)
- Search Results Section (при активном поиске):
 - Section Title: "Результаты поиска" (черный текст, bold)
 - Карточки найденных пользователей
 - Или "Ничего не найдено" (centered, серый текст)
- Bottom Navigation Bar: 5 табов (Главная, Поиск, Лайки, Профиль, Ещё)
 - Tab 2:  Поиск (активный - синий) ← АКТИВНЫЙ ТАБ

Валидация:

- Поиск:
 - Минимум 3 символа для начала поиска
 - Если меньше 3 символов → поиск не выполняется
 - Debounce 300ms (запрос отправляется через 300ms после окончания ввода)
- ВАЖНО поиск ток по логину, по имени фамилии школе поиска нету.

Ошибки:

- Ошибка загрузки рекомендаций → "Не удалось загрузить рекомендации. Попробуйте позже"
- Ошибка поиска → "Не удалось выполнить поиск. Попробуйте снова"
- Ошибка отправки запроса в друзья → "Не удалось отправить запрос. Попробуйте снова"
- Ошибка скрытия пользователя → "Не удалось скрыть пользователя"
- Пустой результат поиска → "Ничего не найдено" (centered text)

Состояния:

- Loading:
 - Skeleton loader для карточек рекомендаций при первой загрузке
- Default (без поиска):
 - При открытии экрана отправляется 2 запроса к серверу:
 - GET /api/search/suggestions? type=friends_of_friends&limit=200 → получаем все

- 200 пользователей для секции "Возможные знакомства"
 - b. GET /api/search/suggestions?type=same_school&limit=200 → получаем все 200 пользователей для секции "С одной школы"
- Все данные сохраняются в памяти на клиенте
- Показываются две секции рекомендаций:
 - a. "Возможные знакомства":
 - Показываются первые 5 карточек
 - Формируется по алгоритму "Друзья Друзей" (друзья моих друзей, которые не являются моими друзьями) Алгоритм опишем ниже
 - Под именем: "X общих друзей"
 - Внизу секции: "Ещё 195" (если всего 200 пользователей)
 - b. "С одной школы":
 - Показываются первые 5 карточек
 - Все пользователи из моей школы, которые НЕ являются моими друзьями
 - Сортировка: по убыванию количества общих друзей (больше общих → выше)
 - Под именем: класс пользователя ("11 Класс", "10 Класс", etc.)
 - Внизу секции: "Ещё 195" если есть столько людей если нету то показывает столько сколько есть
- Tab "Поиск" активен (синий)
- Scroll Down (прокрутка вниз):
 - При достижении конца секции "Возможные знакомства":
 - a. Подгружаются следующие 15 карточек из памяти (без запроса к серверу!)
 - b. Счётчик обновляется: "Ещё 195" → "Ещё 180"
 - c. Прокрутка мгновенная (нет задержек, нет loading indicator)
 - При достижении конца секции "С одной школы" - аналогично
 - Когда все карточки показаны → счётчик "Ещё X" исчезает
- Search Field Focus:
 - При клике на поле поиска → курсор в поле, клавиатура появляется
 - Placeholder "Введите имя" остаётся до начала ввода
- Search Input (ввод текста):
 - Пользователь вводит текст (например, "Ива")
 - Если < 3 символов → поиск НЕ выполняется, показываются секции по умолчанию
 - Если ≥ 3 символов → запускается debounce 300ms:
 - a. Ожидание 300ms после последнего символа
 - b. Если пользователь продолжает вводить → таймер сбрасывается
 - c. Если прошло 300ms без ввода → отправляется 1 запрос на сервер: GET /api/search/users?query=Ива
 - Появляется крестик X справа в поле (для очистки)
 - Показывается loading indicator под полем поиска
- Search Results (результаты поиска):
 - Секции "Возможные знакомства" и "С одной школы" скрываются
 - Показывается новая секция "Результаты поиска"
 - Результаты поиска НЕ сохраняются в памяти целиком (только то, что сервер вернул)
 - Карточки найденных пользователей:
 - a. Аватар, имя, фамилия
 - b. Username (если искали по username)
 - c. Школа и класс (под именем)
 - d. Количество общих друзей (если есть)
 - e. Кнопки "Скрыть" и "Добавить"
 - Поиск выполняется по:
 - a. Username (@ivan_petrov)
- Search Empty State (ничего не найдено):
 - Если поиск не вернул результатов:

- a. Показывается centered текст: "Ничего не найдено"
 - b. Опционально: иконка 
 - c. Подсказка: "Попробуйте изменить запрос"
- Clear Search (X кнопка):
 - При нажатии на крестик X:
 - a. Поле поиска очищается
 - b. Секция "Результаты поиска" скрывается
 - c. Возвращаются секции "Возможные знакомства" и "С одной школы" (данные уже в памяти, НЕТ запроса)
 - d. Клавиатура остаётся открытой (поле в фокусе)
- Hide Button Clicked ("Скрыть"):
 - При нажатии "Скрыть" (например, на карточке "Olivia Hayes"):
 - a. Карточка исчезает из списка (анимация slide out)
 - b. Пользователь скрывается навсегда из рекомендаций ("Возможные знакомства" и "С одной школы")
 - c. Удаляется из памяти (не появится снова в этой сессии)
 - d. Счётчик обновляется: "Ещё 195" → "Ещё 194"
 - e. Toast: "Пользователь скрыт"
 - f. Отправляется запрос: POST /api/users/hide
 - g. НО: если ввести его имя в поиск → он найдётся (скрытие действует только на рекомендации)
 - h. НЕТ списка скрытых пользователей (нельзя разблокировать через UI)
- Add Button Clicked ("Добавить"):
 - При нажатии "Добавить" (например, на карточке "Noah Foster"):
 - a. Отправляется запрос в друзья: POST /api/friends/send-request
 - b. Показывается toast:  Запрос в друзья отправлен Noah Foster"
 - c. Карточка исчезает из списка (анимация slide out)
 - d. Удаляется из памяти
 - e. Счётчик обновляется: "Ещё 195" → "Ещё 194"
 - f. Получатель (Noah) получает push-уведомление:
 - g. Code

"Иван Петров хочет добавить вас в друзья"

- h.
 - i. Запрос попадает в Friend_Requests_Screen получателя
- Navigation:
 - Tab "Поиск" → текущий экран (активный - синий)
 - Другие табы переключают на соответствующие экраны

Важно:

- ⚠ One-time load: при открытии экрана 2 запроса к серверу (все данные сразу)
- ⚠ In-memory storage: все 200 пользователей для каждой секции хранятся в памяти
- ⚠ Zero-latency scroll: прокрутка мгновенная (данные из памяти, НЕТ запросов к серверу)
- ⚠ Counter updates: счётчик "Ещё X" обновляется автоматически при прокрутке, скрытии, добавлении
- ⚠ No pull-to-refresh: функция убрана (чтобы не нагружать сервер)
- ⚠ Friends of Friends algorithm: секция "Возможные знакомства" по алгоритму "Друзья Друзей"
- ⚠ Same school sorting: секция "С одной школы" отсортирована по убыванию общих друзей
- ⚠ Search debounce 300ms: запрос на поиск через 300ms после окончания ввода
- ⚠ Min 3 chars: минимум 3 символа для начала поиска
- ⚠ Search results section: результаты в отдельной секции "Результаты поиска"
- ⚠ Clear returns to cache: кнопка X возвращает данные из памяти (без запроса)
- ⚠ Hide forever: кнопка "Скрыть" удаляет из рекомендаций навсегда (но найдётся через поиск)
- ⚠ No hidden list: НЕТ списка скрытых пользователей
- ⚠ Add sends request: кнопка "Добавить" отправляет запрос в друзья
- ⚠ Card disappears: после действия карточка исчезает + счётчик обновляется
- ⚠ Push on add: получатель получает push
- ⚠ Active tab: таб "Поиск" активен (синий)

likes_filled_screen

UI элементы:

- Background: Светло-серый фон
- Header Card: Темно-синий фон с закруглёнными углами
 - Title: "Лайки" (белый текст, крупный, bold, left)
 - Counter: "Всего: 7" (белый текст на тёмно-синей кнопке, rounded, top-right)
 - Subtitle: "Узнайте, кто выбрал вас в голосовании" (белый текст, small)
- Section Title: "Сегодня" (чёрный текст, bold, left)
- Likes List: Прокручиваемый список карточек с лайками (белый фон)
 - Like Card 1:
 - Иконка  (сердечко в синем круге, left)
 - Текст: "За тебя проголосовали с 10 класса" (чёрный, bold)
 - Бейдж: "Девочка" (синий фон, белый текст, small, rounded)
 - Время: "Час назад" (серый текст, small, right)
 - Like Card 2: "11 класса", "Мальчик", "Час назад"
 - Like Card 3: "9 класса", "Девочка", "Час назад"
 - Like Card 4: "11 класса", "Небинарный", "Час назад"
- Bottom Navigation Bar: 5 табов (Главная, Поиск, Лайки, Профиль, Ещё)
 - Tab 3:  Лайки (активный - синий) ← АКТИВНЫЙ ТАБ

Валидация:

- Нет валидации - это экран просмотра уведомлений

Ошибки:

- Ошибка загрузки лайков → "Не удалось загрузить лайки. Попробуйте позже"
- Пустой список → "У вас пока нет лайков" (centered text + иллюстрация)

Состояния:

- Loading:
 - Skeleton loader для карточек лайков при первой загрузке
- Default:
 - Показывается список всех голосов, которые я получил
 - Счётчик "Всего: 7": общее количество голосов (7 человек выбрали меня)
 - Каждый голос = отдельная карточка (отдельное уведомление)
 - Секция "Сегодня": показываются лайки, полученные сегодня (с 00:00 до 23:59)
 - Секция "За все время": показывает все остальные лайки
 - На экране видны первые 4 карточки
 - Ещё 3 карточки не видны (нужно прокрутить вниз)
 - Tab "Лайки" активен (синий)
- Like Card Info:
 - Каждая карточка показывает минимальную информацию:
 - Класс голосовавшего ("10 класса", "11 класса", "9 класса")
 - Пол голосовавшего ("Девочка", "Мальчик", "Небинарный")
 - Время голосования ("Час назад", "2 часа назад", "Вчера")
 - НЕ показывается:
 - Имя голосовавшего (скрыто)
 - Вопрос, в котором выбрали (скрыто)
 - Иконка  одинаковая для всех карточек (не зависит от типа вопроса)
- Multiple Votes from Same Question:
 - Пример: Вопрос "Кто самый умный?" → за меня проголосовало 5 человек
 - Я вижу 5 отдельных карточек:
 - Code

 За тебя проголосовали с 10 класса - Девочка
 За тебя проголосовали с 11 класса - Мальчик
 За тебя проголосовали с 10 класса - Девочка
 За тебя проголосовали с 9 класса - Мальчик
 За тебя проголосовали с 11 класса - Небинарный

- Каждое уведомление считается отдельным (нет группировки)
- Same Person, Different Questions:

- Если один человек выбрал меня в 2 разных вопросах:
 - Я вижу 2 отдельные карточки (по одной на каждый вопрос)
 - Пример:
 - Code
 - ♥ За тебя проголосовали с 10 класса - Девочка (вопрос "Самый умный")
 - ♥ За тебя проголосовали с 10 класса - Девочка (вопрос "Самый красивый")
 -
 - Нельзя определить, что это один и тот же человек (имя скрыто)
 - Like Card Clicked:
 - При нажатии на карточку лайка:
 - Открывается детальный экран → Likes_Result_Screen
 - На новом экране показывается:
 - Подробная информация о голосе
 - Возможность купить подписку (за реальные деньги), чтобы узнать имя голосовавшего
 - Примечание: Likes_Result_Screen - это отдельный экран (будет описан позже)
 - Scroll Down (прокрутка вниз):
 - При прокрутке вниз показываются следующие карточки (5-я, 6-я, 7-я)
 - Все 7 лайков уже загружены (нет дополнительных запросов к серверу при прокрутке)
 - Counter Updates:
 - Счётчик "Всего: 7" обновляется в реальном времени:
 - Кто-то проголосовал за меня → приходит push-уведомление → счётчик: "Всего: 7" → "Всего: 8"
 - Новая карточка добавляется вверху списка (самые новые сверху)
 - Push Notification Received:
 - Когда кто-то выбирает меня в голосовании:
 - Я сразу получаю push-уведомление:
 - Code
- "За тебя проголосовали!"
- - При клике на push → открывается Likes_Filled_Screen
 - Новая карточка появляется вверху списка (с анимацией)
 - Счётчик увеличивается: "Всего: 7" → "Всего: 8"
- Empty State (нет лайков):
 - Если список лайков пустой (счётчик "Всего: 0"):
 - Показывается centered текст: "У вас пока нет лайков"
 - Опционально: иллюстрация ♥
 - Подсказка: "Участвуйте в голосованиях, чтобы получить лайки"
- Section Grouping:
 - "Сегодня": лайки, полученные сегодня (с 00:00)
 - Опционально (если есть старые лайки):
 - "Вчера": лайки за вчера
 - "На этой неделе": лайки за неделю
 - "Ранее": старые лайки
- Navigation:
 - Tab "Лайки" → текущий экран (активный - синий)
 - Другие табы переключают на соответствующие экраны

Важно:

- ⚠ One vote = one card: каждый голос = отдельная карточка (нет группировки)
- ⚠ Multiple votes same question: если 5 человек выбрали меня в одном вопросе → 5 отдельных карточек
- ⚠ Same person, different questions: если 1 человек выбрал меня в 2 вопросах → 2 отдельные карточки
- ⚠ Counter = total votes: "Всего: 7" = 7 голосов (7 человек выбрали меня)
- ⚠ Hidden info: имя голосовавшего и вопрос скрыты (нужна подписка)
- ⚠ Gender shown: пол голосовавшего показывается бесплатно ("Девочка", "Мальчик", "Небинарный")
- ⚠ Class shown: класс голосовавшего показывается бесплатно ("10 класса", "11 класса")
- ⚠ Real-time push: при новом голосе сразу приходит push-уведомление
- ⚠ Click opens detail: клик на карточку → открывается Likes_Result_Screen (детальный экран)
- ⚠ Subscription unlock: на детальном экране можно купить подписку (за реальные деньги), чтобы раскрыть информацию
- ⚠ Sorted by time: лайки отсортированы по времени (новые сверху)
- ⚠ Section "Today": секция "Сегодня" показывает лайки за текущий день
- ⚠ Active tab: таб "Лайки" активен (синий)
- ⚠ All data loaded: все лайки загружены сразу (нет пагинации при прокрутке)

likes_empty_screen

UI элементы:

- Status Bar: (не виден на скриншоте)
- Background: Белый фон
- Modal/Screen Container: Белый фон с синей рамкой (пунктирная)
- Header:
 - Close Button: X (черный крестик, top-left)
 - Title: "Лайки" (черный текст, крупный, bold, centered)
 - Divider: Тонкая пунктирная линия под заголовком
- Empty State Card: Синяя карточка с закруглёнными углами (centered)
 - Badge: "❤️ Твои будущие лайки" (темно-синий бейдж, rounded, top)
 - Icons Row: 3 иконки (★ ❤️ ✨) в кружках (темно-синие круги)
 - Main Text: "ПОКА ПУСТО" (белый текст, крупный, bold, uppercase, centered)
 - Description Text: "Когда тебя выберут в голосовании, все лайки появятся здесь. Хочешь много голосов проходи больше опросов." (белый текст, средний, centered, 3 строки)
 - Action Button: "Перейти к опросам" (темно-синяя кнопка, белый текст, rounded, centered)

Валидация:

- Нет валидации - это информационный экран

Ошибки:

- Нет ошибок - это пустое состояние (empty state)

Состояния:

- Default (Empty State):
 - Показывается, когда у пользователя нет лайков (счётчик "Всего: 0")
 - Экран объясняет:
 - Что такое лайки: "Когда тебя выберут в голосовании, все лайки появятся здесь"
 - Как получить лайки: "Хочешь много голосов проходи больше опросов"
 - Кнопка "Перейти к опросам" - основное действие (CTA - Call To Action)
- Close Button Clicked (X):
 - При нажатии на крестик:
 - Закрывается экран
 - Возврат на предыдущий экран (откуда пришли)
 - Или возврат на главный экран (Home Screen)
- Action Button Clicked ("Перейти к опросам"):
 - При нажатии на кнопку "Перейти к опросам":
 - Закрывается текущий экран
 - Открывается главный экран с голосованиями
 - Пользователь может начать участвовать в опросах, чтобы получить лайки
- Navigation:
 - Этот экран показывается вместо Likes_Filled_Screen, если лайков нет
 - Может открываться:
 - При клике на таб "Лайки" в нижней навигации
 - При клике на push-уведомление (если оно пришло, но лайков ещё нет)
 - При переходе из меню/профиля

Важно:

- ⚠️ Empty state: показывается вместо Likes_Filled_Screen, когда лайков нет
- ⚠️ Educational: объясняет пользователю, что такое лайки и как их получить
- ⚠️ Single CTA: только одна кнопка "Перейти к опросам" (главное действие)
- ⚠️ Close button: крестик X для закрытия экрана
- ⚠️ No navigation bar: нет нижней навигации (это modal или full-screen overlay)
- ⚠️ Encouraging tone: текст мотивирует пользователя участвовать в опросах
- ⚠️ Visual metaphor: иконки ★ ❤️ ✨ создают ожидание будущих лайков
- ⚠️ Badge "Твои будущие лайки": создаёт персональную связь с будущими результатами

likes_result_screen

UI элементы:

- Background: Синий градиент фон
- Screen Container: Синяя карточка с закруглёнными углами (full-screen)
- Question Section:
 - Emoji: 😊 (крупный эмодзи, centered, top)
 - Question Text: "Кто самый красивый ?" (белый текст, крупный, centered)
- Voting Results Section: "Кто участвовал в голосовании" (неявный заголовок)
 - User Card 1: Белая карточка с закруглёнными углами
 - Аватар (круглое фото)
 - Имя: "Арман Никитенко" (чёрный текст, bold) ← СЕРЫЙ ТЕКСТ (это Я - меня выбрали!)
 - Класс: "9 класс" (серый текст, small)
 - User Card 2: "Александр Иванов", "10 класс"
 - User Card 3: "Виталий Магомедов", "11 класс"
 - User Card 4: "Алия Исакова", "10 класс"
- Share Section:
 - Text: "Поделится результатом в социальных сетях" (белый текст, small, centered)
 - Social Icons: 📷 Instagram (красная иконка) + 🎵 TikTok (чёрная иконка)
- CTA Button: 🛒 Узнать кто проголосовал за тебя" (белая кнопка с синим текстом, крупная, rounded, bottom)

Валидация:

- Нет валидации - это информационный экран

Ошибки:

- Ошибка загрузки данных лайка → "Не удалось загрузить результаты. Попробуйте позже"

Состояния:

- Loading:
 - Skeleton loader для карточек при первой загрузке
- Default (Free User - без подписки):
 - Этот экран открывается при клике на карточку лайка в Likes_Filled_Screen
 - Показывается информация об ОДНОМ конкретном голосовании:
 - Вопрос: "Кто самый красивый?"
 - Список участников голосования (все, кто был в выборе): 4 человека
 - Я в списке (серым текстом, чтобы показать "меня выбрали")
 - НЕ показывается: кто именно проголосовал за меня (это скрыто, нужна подписка)
 - Имена участников показываются бесплатно (это люди, которые были в выборе вместе со мной голосования, но тот кто проголосовал за меня не показывается)
- Understanding the Screen Logic:
Пример ситуации:
 - Был вопрос "Кто самый красивый?"
 - Варианты ответа (из кого выбирали):
 - Арман Никитенко (Я) ← МЕНЯ ВЫБРАЛИ!
 - Александр Иванов
 - Виталий Магомедов
 - Алия Исакова
 - Кто-то проголосовал и выбрал Армана (меня)
 - Я получил лайк (уведомление в Likes_Filled_Screen)
 - Кликнул на карточку → открылся этот экран
- Что показывается:
 - ✓ Вопрос: "Кто самый красивый?"
 - ✓ Список участников (кто был в выборе): 4 человека
 - ✓ Моё имя серым (чтобы показать "меня выбрали")
 - ✗ НЕ показывается: кто именно проголосовал (нужна подписка "Premium")
- My Name Highlighted:
 - "Арман Никитенко" показывается серым текстом (вместо чёрного)
 - Это визуально показывает: "Тебя выбрали из этих 4 человек"
 - Остальные имена чёрным текстом (они были в списке, но их не выбрали)
- User Card Info (участники голосования):
 - Каждая карточка показывает:
 - Аватар пользователя (реальное фото)
 - Имя и фамилия (реальные данные)
 - Класс (реальный класс)
 - Это НЕ те, кто голосовал, а те, кто был в выборе (варианты ответа)

- Показывается бесплатно (без подписки)
- User Card Clicked:
 - При клике на карточку (например, "Александр Иванов"):
 - Открывается профиль этого пользователя → profile_forpeople_screen
 - Можно посмотреть его фото, класс, друзей, etc.
 - Можно добавить в друзья, если ещё не друзья
- Share Section:
 - "Поделится результатом в социальных сетях"
 - При клике на Instagram :
 - Открывается Instagram Share Sheet
 - Пользователь может поделиться:
 - Instagram Stories (рекомендуемый вариант)
 - Instagram Feed
 - Instagram DM
 - Что публикуется:
 - Скриншот/изображение этого экрана (вопрос + список участников + моё имя выделено)
 - Текст: "Меня выбрали самым красивым в школе! 😊
#НазваниеПриложения"
 - Ссылка на приложение (для привлечения новых пользователей)
 - При клике на TikTok :
 - Открывается TikTok приложение
 - Пользователь может:
 - Поделится видео с результатом в Тик Ток
 - Что публикуется: аналогично Instagram
- CTA Button Clicked ("🛒 Узнать кто проголосовал за тебя"):
 - При нажатии на кнопку:
 - Открывается экран подписок → (Подписки "Premium")
- Subscription Active (после покупки Подписки "Premium"):
 - После покупки подписки этот экран обновляется:
 - Добавляется секция "Кто проголосовал за тебя": и открывается экран с выбором раскрытия полного имени или первой буквы имени

Кто проголосовал за тебя:

Если он выбрал полное имя : [Аватар] Мария Петрова, 10 класс

Если он выбрал первую букву : [Аватар] Первая буква имени: М, 11 Класс

- Navigation:
 - Открытие: клик на карточку лайка в Likes_Filled_Screen
 - Закрытие:
 - Swipe down (жест вниз)
 - Или кнопка "Назад" (если есть)
 - Возврат на Likes_Filled_Screen

Важно:

- ⚠ One like details: экран показывает детали ОДНОГО конкретного лайка (одно голосование)
- ⚠ Question shown: показывается вопрос ("Кто самый красивый?")
- ⚠ Participants shown: показываются участники голосования (кто был в выборе)
- ⚠ My name highlighted: моё имя серым текстом (чтобы показать "меня выбрали")
- ⚠ Free info: имена участников показываются бесплатно (это НЕ те, кто голосовал)
- ⚠ Hidden info: кто проголосовал скрыто (нужна подписка Премиум)
- ⚠ CTA to subscription: кнопка "🛒 Узнать кто проголосовал" открывает экран подписок
- ⚠ Cards clickable: карточки участников кликабельны → открывается профиль
- ⚠ Share feature: можно поделиться в Instagram и TikTok
- ⚠ Share content: публикуется скриншот экрана + текст + ссылка на приложение
- ⚠ Educational UX: показываются имена участников (бесплатно), чтобы заинтересовать купить подписку
- ⚠ Sorted by selection: моё имя сверху (или выделено серым), остальные ниже

share_results_screen

UI элементы:

- Нативный Share Sheet:
 - Instagram или TikTok (в зависимости от выбора пользователя)

- Нативный интерфейс соц. сети (системный)
- Генерируемое изображение (автоматически):
 - Background: Синий градиент фон (такой же, как на Likes_Result_Screen)
 - Emoji: 😊 (или соответствующий вопросу)
 - Question Text: "Кто самый красивый?" (белый текст, centered)
 - User Cards: 4 карточки участников голосования (белый фон, rounded)
 - Аватар, Имя, Класс
 - Первая карточка (я) - может быть выделена серым текстом
 - App Branding (внизу изображения):
 - Название приложения (например, "Скачай OISTER" или просто "OISTER")
 - Логотип приложения (опционально)
 - Ссылка/QR-код (опционально)

Валидация:

- Нет валидации - это экран генерации изображения для share

Ошибки:

- Ошибка генерации изображения → "Не удалось создать изображение. Попробуйте снова"
- Instagram/TikTok не установлен → "Установите Instagram/TikTok для публикации"
- Ошибка открытия соц. сети → "Не удалось открыть приложение"

Состояния:

- Default:
 - Этот экран НЕ показывается пользователю как отдельный экран
 - Это автоматический процесс генерации изображения
 - Когда пользователь нажимает Instagram или TikTok на Likes_Result_Screen:
 - Генерируется изображение (на клиенте)
 - Открывается нативное приложение (Instagram/TikTok)
 - Изображение автоматически добавляется в Stories/новый пост
 - Пользователь может отредактировать и опубликовать
- Image Generation Process:

Шаг 1: Подготовка данных

 - Берём данные из Likes_Result_Screen:
 - Вопрос: "Кто самый красивый?"
 - Эмодзи: 😊
 - Участники: 4 человека (имя, класс, аватар)
 - Моё имя: для выделения серым цветом
- Шаг 2: Создание изображения (на клиенте)
 - Используется Canvas API (React Native, Flutter, или нативные SDK)
 - Создаётся холст нужного размера
 - Рисуются все элементы (фон, текст, карточки, аватары)
 - Добавляется название приложения внизу
 - Конвертируется в изображение (PNG/JPEG)
- Шаг 3: Открытие соц. сети
 - Изображение передаётся в Instagram или TikTok через нативное API
 - Открывается Stories (рекомендуемый вариант) или новый пост
 - Пользователь видит предзаполненный контент (изображение уже добавлено)
 - Может отредактировать (добавить стикеры, текст, фильтры)
 - Нажимает "Опубликовать"
- Share to Instagram:
 - При клике на Instagram на Likes_Result_Screen:
 - Генерируется изображение (формат 9:16 для Stories или 1:1 для Feed)
 - Используется Instagram Sharing API:
 - iOS: Instagram URL Scheme или UIActivityViewController
 - Android: Intent с MediaStore
 - Открывается Instagram Stories (если установлен)
 - Изображение автоматически добавлено
 - Пользователь публикует
- Share to TikTok:
 - При клике на TikTok на Likes_Result_Screen:
 - Генерируется изображение
 - Используется TikTok Share Kit (если доступен) или системный Share
 - Открывается TikTok приложение
 - Пользователь может создать видео с этим изображением или опубликовать как фото (если TikTok поддерживает)
- Error Handling:
 - Instagram/TikTok не установлен:

- Toast: "Установите Instagram/TikTok для публикации"
- Ошибка генерации:
 - Toast: "Не удалось создать изображение"
 - Предложить попробовать снова
- Ошибка открытия приложения:
 - Fallback на системный Share Sheet (выбор из всех приложений)

Важно:

- ⚠ Native share: используется нативный Instagram/TikTok Share (не кастомный экран)
- ⚠ Auto-generated image: изображение автоматически генерируется на клиенте
- ⚠ Cropped screenshot style: изображение выглядит как обрезанный скриншот Likes_Result_Screen
- ⚠ App branding: внизу изображения название приложения
- ⚠ Client-side generation: генерация на клиенте (не на сервере)
- ⚠ Optimized size: изображение оптимизировано (не занимает много памяти)
- ⚠ Direct app launch: сразу открывается Instagram/TikTok (без промежуточных экранов)
- ⚠ Pre-filled content: изображение уже добавлено в Stories/пост (пользователь только публикует)
- ⚠ Developer freedom: разработчик решает детали реализации (размеры, отступы, шрифты)
- ⚠ Match original screen: изображение визуально идентично Likes_Result_Screen (тот же стиль)
- ⚠ Fallback to gallery: если соц. сеть не установлена → сохранить в галерею

menu_screen

UI элементы:

- Background: Белый фон
- Header:
 - Back Button: ← (стрелка влево, синяя, top-left)
 - Title: "Ещё" (белый текст на синем фоне, centered)
 - Синий фон заголовка с закруглёнными углами
 - Стрелка → (серая, right)
- Social Links Section:
 - Instagram Icon:  (синий круг, кликабельно)
 - Snapchat Icon:  (оранжевый круг, кликабельно)
- Bottom Navigation Bar: 5 табов (Главная, Поиск, Лайки, Профиль, Ещё)
 - Tab 5: ≡ Ещё (активный - синий) ← АКТИВНЫЙ ТАБ

Валидация:

- Нет валидации - это навигационное меню

Ошибки:

- Нет ошибок - это статический список

Состояния:

- Default:
 - Показывается список из 8 пунктов меню
 - Каждый пункт кликабельный (открывает соответствующий экран)
 - Tab "Ещё" активен (синий)
- Menu Item 1 Clicked ("Подписки"):
 - Открывается экран подписок → premium_screen
 - Показываются тарифные планы:
 - Описание преимуществ подписки
 - Кнопка "Купить"
- Menu Item 2 Clicked ("Оповещения"): Activity_screen
 - Открывается экран Оповещения
 - Показывается активность по всей школе:
 - Кто выиграл в каких опросах
 - Например: "Мария Петрова выиграла в 'Самый умный' (10 класс)"
 - Лента активности всех пользователей школы
- Menu Item 3 Clicked ("Настройки"):
 - Открывается экран настроек → Settings_Screen
- Menu Item 4 Clicked ("FAQ"):

- Открывается экран FAQ → FAQ_Screen (текстовый экран с часто задаваемыми вопросами)
- Menu Item 5 Clicked ("Как Пользоваться"):
 - Открывается экран инструкций → How_To_Use_Screen
 - Показывается текст инструкции и правила использования приложения:
 - Как регистрироваться
 - Как участвовать в голосованиях
 - Как добавлять друзей
 - Правила поведения
 - etc.
 - Формат: прокручиваемый текст или пошаговый гайд с иллюстрациями
- Menu Item 6 Clicked ("Центр Безопасности"):
 - Открывается экран безопасности → Safety_Center_Screen
 - Показываются текстовые документы:
 - Политика конфиденциальности (Privacy Policy)
 - Условия использования (Terms of Service)
 - Рекомендации по безопасности
 - Как пожаловаться на пользователя
 - Правила сообщества
 - Формат: список документов (каждый открывается отдельно)
- Menu Item 7 Clicked ("Сообщить О Проблеме"):
 - Открывается экран обратной связи → это просто экран с заготовленными сообщениями на почту, нативный. ТО есть форма открывает приложение почта на клиенте и он может написать нам на корпоративную почту

Menu Item 8 Clicked ("Для Родителей"):

- Открывается экран для родителей → For_Parents_Screen
- Показывается текстовая информация:
 - Как работает приложение
 - Политика конфиденциальности (для родителей)
 - Как контролировать активность ребёнка
 - Контакты поддержки для родителей
- Формат: прокручиваемый текст
- Instagram Icon Clicked (📷):
 - Открывается Instagram страница приложения:
 - В браузере (если Instagram не установлен)
 - В приложении Instagram (если установлен)
 - URL: например, https://instagram.com/oister_app
- Snapchat Icon Clicked (📷):
 - Открывается Snapchat страница приложения:
 - В браузере (если Snapchat не установлен)
 - В приложении Snapchat (если установлен)
 - URL: например, https://snapchat.com/add/oister_app
- Back Button Clicked (←):
 - Возврат на главный экран → (таб "Главная")
- Navigation:
 - Tab "Ещё" → текущий экран (активный - синий)
 - Другие табы переключают на соответствующие экраны

Важно:

- ⚠ Navigation menu: это главное меню приложения (все важные разделы)
- ⚠ Подписки → Subscription: открывает экран подписок (тарифы, покупка)
- ⚠ Оповещения → Activity_screen: открывает активность всей школы (кто выиграл в опросах)
- ⚠ Настройки → Settings: открывает экран настроек
- ⚠ FAQ → Questions: открывает часто задаваемые вопросы (текстовый скинем позже)
- ⚠ Как Пользоваться → Instructions: открывает инструкции и правила (текстовый скинем позже)
- ⚠ Центр Безопасности → Legal Docs: открывает политику конфиденциальности и другие документы
- ⚠ Сообщить О Проблеме → Support: открывает форму обратной связи для написания на корпоративную почту
- ⚠ Scrollable: список меню прокручиваемый (если не помещается на экран)

activity_screen

UI элементы:

- Background: Светло-серый фон
- Header:
 - Title: "Оповещения" (белый текст на синем фоне, centered, крупный)
 - Синий фон заголовка (полная ширина экрана)
- Activity List: Вертикальный прокручиваемый список победителей голосований (белый фон карточек)
 - Activity Card 1:
 - Star Icon: ★ (золотая звезда в светло-сером круге, left)
 - Name: "Алия Есенова" (чёрный, bold, крупный)
 - Status: "выбрана в опросе" (чёрный, medium)
 - Poll Title: "Похож на Илона Маска" (чёрный, medium)
 - Voter Info: "От девочки с 9 класса" (серый, small)
 - Time: "8 ч" (серый, small, right, в светло-сером бейдже)
 - Activity Card 2: ★, "Макпал Исабекова", "выбрана в опросе", "Кто самый умный", "От девочки с 11 класса", "8 ч"
 - Activity Card 3: ★, "Александр Прилучный", "выбран в опросе", "Воин в душе", "От мальчика с 10 класса", "Вчера"
 - Activity Card 4: ★, "Тимур Батрудинов", "выбрана в опросе", "Похож на Илона Маска", "От девочки с 9 класса", "3 дня"
 - Activity Card 5: ★, "Дария Морозова", "выбрана в опросе", "Мощный человек", "От мальчика с 9 класса", "5 дней"
- Bottom Navigation Bar: 5 табов (Главная, Поиск, Лайки, Профиль, Ещё)
 - Все табы неактивны (этот экран открывается из Menu_Screen, не из таба)

Валидация:

- Нет валидации - это информационный экран (только просмотр)

Ошибки:

- Ошибка загрузки активности → "Не удалось загрузить активность. Попробуйте позже"
- Пустой список → Empty state "Пока нет активности в вашей школе" (centered text + иллюстрация ★)

Состояния:

- Loading:
 - Skeleton loader для карточек активности при первой загрузке (3-5 карточек-заглушек)
- Default (по умолчанию):
 - При открытии экрана отправляется 1 запрос к серверу:
 - GET /api/activity/wins? grade=9&limit=1000
 - Кэширование на сервере (Redis):
 - Сервер проверяет Redis: есть ли данные для этого класса?
 - Если данные есть в Redis (кэш свежий, меньше 30 минут) → отдаёт из Redis (быстро, ~5ms)
 - Если данных нет в Redis → ищет в базе данных (медленнее, ~100ms) → сохраняет в Redis на 30 минут → отдаёт клиенту
 - Кэширование на клиенте (телефон):
 - Клиент получает данные → сохраняет в локальное хранилище (AsyncStorage/SharedPreferences) на 30 минут
 - При повторном открытии экрана (если прошло меньше 30 минут) → показывает сохранённые данные БЕЗ запроса к серверу
 - Если прошло больше 30 минут → отправляет новый запрос
 - Сервер возвращает последние 1000 побед (не из моего класса, а кто выбрал учеников из моего класса)
 - Все данные сохраняются в памяти на клиенте
 - Показываются все карточки (прокрутка из памяти, без дополнительных запросов)
 - Сортировка: по времени (новые сверху)
- Activity Card Info:
 - Каждая карточка показывает:
 - Звезда ★ (золотая звезда в светло-сером круге) - вместо аватара (красиво и единообразно!)
 - Имя и фамилия победителя (кто был выбран)
 - Статус: "выбрана в опросе" (женский род) или "выбран в опросе" (мужской род)
 - Название опроса (например, "Похож на Илона Маска", "Кто самый умный")
 - Информация о голосовавшем:
 - "От девочки с 9 класса" (кто-то из 9 класса, девочка, проголосовал)
 - "От мальчика с 10 класса"

- НЕ показывается имя голосовавшего (для этого нужна подписка Премиум)
 - Время: когда произошло голосование ("8 ч", "Вчера", "3 дня", "5 дней")
- Gender Detection (определение рода):
 - Система автоматически определяет пол победителя и использует правильное окончание:
 - Женский род: "Алия Есенова выбрана в опросе"
 - Мужской род: "Александр Прилучный выбран в опросе"
- Voter Info Format (информация о голосовавшем):
 - Формат: "От [пол] с [класс] класса"
 - Примеры:
 - "От девочки с 9 класса"
 - "От мальчика с 11 класса"
 - "От небинарного с 10 класса" (если пол "небинарный")
 - Имя голосовавшего скрыто (нужна подписка для раскрытия)
- Logic Change (изменение логики):
 - ⚠ ВАЖНО: Экран показывает НЕ победителей из моего класса, а кто выбрал учеников из моего класса!

Пример:

Я учусь в 9 классе

Ученица Маша из 11 класса проголосовала в опросе "Кто самый умный?"

Маша выбрала Алию Есенову (9 класс)

→ В Activity_Screen показывается:

"Алия Есенова выбрана в опросе 'Кто самый умный'"

От девочки с 11 класса"

Потому что Алия из МОЕГО класса (9)!

- Фильтрация:
 - Показываются победы учеников из моего класса
 - Неважно, кто голосовал (могут быть из других классов)
 - Важно, что выбрали ученика из моего класса
- Aggregation (агрегация):
 - Один голос = одна запись
 - Логика:
 - Каждый раз когда кто-то выбирает ученика в опросе → создаётся одна запись в базе данных
 - НЕ агрегируем по опросам (как в предыдущей версии)
 - Если 5 человек выбрали Алию в опросе "Самый умный" → показываются 5 отдельных карточек

- Пример:

Опрос "Кто самый умный?"

- Маша (11 класс) выбрала Алию (9 класс) → запись #1

- Петя (10 класс) выбрал Алию (9 класс) → запись #2

- Катя (9 класс) выбрала Алию (9 класс) → запись #3

В Activity_Screen показываются 3 карточки:

1. "Алия Есенова выбрана... От девочки с 11 класса - 8 ч"

2. "Алия Есенова выбрана... От мальчика с 10 класса - 8 ч"

3. "Алия Есенова выбрана... От девочки с 9 класса - 8 ч"

- Почему так:
 - Это экран активности/лайков для всего класса
 - Интересно видеть каждый отдельный голос
 - НЕ перегружает базу (всё равно не больше 100 элементов показываем)
- Time Format (формат времени):
 - Относительное время:
 - "8 ч" = 8 часов назад
 - "8 м" = 8 минут назад
 - "Вчера" = если прошло 24-48 часов
 - "3 дня" = 3 дня назад
 - "5 дней" = 5 дней назад

- "05.12" = если больше 7 дней (показывается дата)
- Activity Card Clicked:
 - При клике на карточку (например, "Алия Есенова"):
 - Открывается профиль победителя → profile_forpeople_screen
 - Можно посмотреть профиль Алии
 - Можно добавить в друзья (если ещё не друг)
 - НЕ показывается, кто именно проголосовал (для этого нужна подписка)
- Empty State (нет активности):
 - Если в классе нет голосований за учеников:
 - Показывается centered текст: "Пока нет активности"
 - Иллюстрация ★
 - Подсказка: "Когда кто-то выберет ученика из вашего класса, здесь появится информация"
- Scroll Down (прокрутка вниз):
 - При прокрутке показываются следующие карточки из памяти (без запроса к серверу)
 - Все 1000 элементов уже загружены
 - Прокрутка мгновенная (нет задержек)
- No Pull-to-Refresh:
 - НЕТ pull-to-refresh (потянуть вниз для обновления)
 - Обновление только при повторном открытии экрана (через 30 минут)
 - Это снижает нагрузку на сервер
- Navigation:
 - Этот экран открывается из Menu_Screen (пункт "Оповещения")
 - НЕТ активного таба в нижней навигации
 - Кнопка "Назад" (системная или в Header) → возврат на Menu_Screen

Важно:

- ⚠ Votes for my class: показываются голоса за учеников из моего класса (не победители опросов!)
- ⚠ One vote = one card: каждый голос = отдельная карточка (даже если один опрос)
- ⚠ Gender detection: автоматическое определение рода ("выбран" vs "выбрана")
- ⚠ Voter info hidden: показывается пол и класс голосовавшего, но НЕ имя (нужна подписка)
- ⚠ Poll title shown: показывается название опроса
- ⚠ Time relative: время показывается относительно ("8 ч назад", "Вчера")
- ⚠ Sorted by time: список отсортирован по времени (новые сверху)
- ⚠ Limit 100: показываются только последние 100 голосов
- ⚠ Redis cache (server): сервер кэширует результаты на 30 минут (быстрые повторные запросы)
- ⚠ Client cache (phone): клиент сохраняет данные на 30 минут (меньше запросов к серверу)
- ⚠ All data loaded: 1 запрос при открытии → все данные в памяти → 0 запросов при прокрутке
- ⚠ No real-time: НЕТ автоматического обновления (только при повторном открытии)
- ⚠ No pull-to-refresh: НЕТ обновления по swipe down (снижение нагрузки)
- ⚠ Card clickable: клик на карточку → профиль победителя
- ⚠ Empty state: если нет активности → "Пока нет активности"

1. Кэш на телефоне (первый уровень):
 - Открыли экран → телефон проверяет: "Есть сохранённые данные?"
 - Если есть и свежие (меньше 30 минут) → показываем сразу, БЕЗ запроса к серверу
 - Если старые или нет → идём на сервер
2. Кэш на сервере Redis (второй уровень):
 - Телефон спросил сервер → сервер проверяет Redis: "Есть данные для 9 класса?"
 - Если есть (меньше 30 минут) → отдаёт из Redis (очень быстро, 5ms)
 - Если нет → ищет в PostgreSQL (медленнее, 100ms)
3. База данных PostgreSQL (третий уровень):
 - Если Redis пустой → сервер ищет в основной базе данных
 - Находит 1000 последних голосов
 - Сохраняет в Redis на 30 минут (для следующих запросов)
 - Отдаёт телефону
4. Сохранение на телефоне:
 - Телефон получил данные → сохраняет на 30 минут
 - В следующий раз (если меньше 30 минут) → не спрашивает сервер вообще

Экономия:

- 50,000 учеников открыли экран → только 1 запрос к базе данных (остальные из Redis)
- Повторное открытие (меньше 30 минут) → 0 запросов к серверу (с телефона)
- Скорость: первый раз 100ms, остальные 5ms, с телефона мгновенно

Обновление данных:

- Автоматического обновления НЕТ (не нагружаем сервер)
- Данные обновляются при повторном открытии экрана (если прошло 30 минут)
- Новые голоса появляются максимум через 30 минут (это нормально для такого экрана)

Клик на карточку:

Открывается профиль победителя (кого выбрали)

Важно:

Оптимизация:

- 3 уровня кэша: телефон (мгновенно) → Redis (5ms) → PostgreSQL (100ms)
- Redis TTL: данные живут 30 минут, потом автоматически удаляются
- Агрегация: каждый голос = отдельная запись (не группируем)
- Лимит 1000: показываем только последние 1000 голосов (быстрая загрузка)
- Один класс: только голоса за учеников из моего класса

Логика:

- Звёздочки вместо аватаров (красиво и единообразно)
- Автоопределение рода ("выбран" для мальчиков, "выбрана" для девочек)
- Имя голосовавшего скрыто (показываем только пол и класс)
- Время относительное ("8 ч назад", "Вчера")

Нагрузка на сервер:

- БЕЗ оптимизации: 50,000 запросов к базе данных → сервер падает
- С оптимизацией: 1 запрос к базе данных → остальные из Redis → сервер спокоен
- Экономия в 50,000 раз меньше нагрузки

settings_poll_screen

Валидация:

- Обязательно выбрать опцию перед нажатием "Выбрать"
- Если не выбрано → кнопка "Выбрать" неактивна (серая) или показывается подсказка

Ошибки:

- Нет ошибок (простой выбор из списка)

Состояния:

- Default (по умолчанию):
 - Показывается modal карточка с настройками
 - Dropdown закрыт, показывает текущую настройку: "Все"
 - Опции скрыты
- Dropdown Opened (выпадающий список открыт):
 - Пользователь нажал на dropdown "Все"
 - Раскрывается список из 3 опций:
 - Девочки (выбрано - синий radio button)
 - Мальчики (не выбрано)
 - Небинарный (не выбрано)
 - Можно выбрать только одну опцию (radio buttons, не checkboxes)
- Option Selected (опция выбрана):
 - Пользователь кликнул на опцию (например, "Девочки")
 - Radio button становится синим с точкой внутри
 - Предыдущая выбранная опция снимается (если была)
 - Dropdown остаётся открытым (можно передумать и выбрать другую)
- Button Clicked ("Выбрать"):
 - Пользователь нажал кнопку "Выбрать"
 - Настройка сохраняется
 - Modal закрывается
 - Возврат на предыдущий экран (Settings_Screen или экран создания опроса)
 - Применяются новые настройки:
 - Если выбрано "Девочки" → в опросах показываются только девочки
 - Если "Мальчики" → только мальчики
 - Если "Небинарный" → только небинарные пользователи

- Если "Все" → все пользователи (default)
- Back Clicked ("Назад"):
 - Пользователь нажал "Назад" (или свайп вниз для закрытия modal)
 - Modal закрывается БЕЗ сохранения изменений
 - Возврат на предыдущий экран

Важно:

- ⚠ Poll settings modal: это модальное окно настроек опроса (не полноэкранный экран)
- ⚠ Gender filter: фильтр по полу участников опроса
- ⚠ Radio buttons: можно выбрать только ОДНУ опцию (не множественный выбор)
- ⚠ Default value: по умолчанию выбрано "Все" (показывать всех)
- ⚠ Save on button click: настройки сохраняются только при нажатии "Выбрать"
- ⚠ Cancel on back: при нажатии "Назад" изменения НЕ сохраняются
- ⚠ Applied to polls: настройка применяется к опросам (кого показывать в вариантах ответа)

delete_account_screen

Валидация:

- Обязательно выбрать причину (хотя бы одну опцию) перед удалением
- Если выбрано "Другая причина" → обязательно написать текст в поле ввода
- Если не выбрана причина → кнопка "Удалить безвозвратно" неактивна (серая) или показывается предупреждение

Ошибки:

- Не выбрана причина → "Пожалуйста, выберите причину удаления"
- Выбрано "Другая причина", но не написан текст → "Пожалуйста, опишите причину"
- Ошибка удаления аккаунта → "Не удалось удалить аккаунт. Попробуйте позже"

Состояния:

- Default (по умолчанию):
 - Показывается экран с предупреждением и опросом
 - Все radio buttons пустые (ничего не выбрано)
 - Текстовое поле неактивно (серое)
 - Кнопка "Удалить безвозвратно" неактивна (серая)
- Option Selected (опция выбрана):
 - Пользователь выбрал одну из причин (например, "Опасения по поводу безопасности")
 - Radio button становится синим с точкой
 - Предыдущая выбранная опция снимается (можно выбрать только одну)
 - Кнопка "Удалить безвозвратно" становится активной (синей)
- "Другая причина" Selected:
 - Пользователь выбрал "Другая причина"
 - Текстовое поле становится активным (белый фон)
 - Пользователь может ввести свой текст
 - Кнопка "Удалить безвозвратно" активна только ЕСЛИ текст написан (минимум 10 символов)
- Delete Button Clicked ("Удалить безвозвратно"):
 - Показывается финальное подтверждение (второе предупреждение):
 - a. Alert/Dialog: "Вы уверены? Это действие нельзя отменить!"
 - b. Кнопки: "Да, удалить" (красная) и "Отмена" (серая)
 - Если нажал "Да, удалить":
 - a. Отправляется запрос на сервер с причиной удаления
 - b. Сервер помечает аккаунт как "удалённый" (soft delete или hard delete)
 - c. Удаляются все данные пользователя:
 - Профиль (имя, фото, класс)
 - Голоса в опросах
 - Лайки (отправленные и полученные)
 - Друзья (связи удаляются)
 - Премиум (подписка отменяется)
 - Монеты/звёздочки
 - d. Пользователь выходит из системы (logout)
 - e. Возврат на экран входа/регистрации (Login_Screen)

- f. Toast: "Ваш аккаунт был удалён"
- Cancel Button Clicked ("Не удалять"):
 - Пользователь передумал
 - Экран закрывается
 - Возврат на Settings_Screen (откуда пришёл)
 - Никакие данные НЕ удаляются
- Back Button Clicked ("Назад"):
 - То же самое, что "Не удалять"
 - Возврат на Settings_Screen
 - Данные остаются

Важно:

- ⚠ Critical action: НЕОБРАТИМОЕ действие (нельзя восстановить аккаунт!)
- ⚠ Double confirmation: два уровня подтверждения (экран + финальный alert)
- ⚠ Data deletion: удаляются ВСЕ данные пользователя
- ⚠ Survey required: обязательно указать причину удаления
- ⚠ Text input for "Other": если выбрано "Другая причина" → обязательно написать текст
- ⚠ Logout after delete: после удаления пользователь выходит из системы
- ⚠ No recovery: восстановить аккаунт НЕВОЗМОЖНО
- ⚠ Subscription cancel: Подписки "Premium" автоматически отменяется (возврат денег?)
- ⚠ Analytics: причины удаления сохраняются для аналитики (чтобы понять проблемы приложения)

change_my_name_screen - это пример для "Написать в поддержку", "Сообщить о проблеме", "Изменить имя"

⚠ ВАЖНО: Это НЕ отдельный экран!

Это НАТИВНОЕ почтовое приложение (Apple Mail / Gmail), которое открывается с предзаполненным письмом.

Где используется (только 3 места):

1. Menu_Screen → кнопка "Написать в поддержку"
2. Menu_Screen → кнопка "Сообщить о проблеме"
3. Settings_Screen → кнопка "Изменить имя"

Как работает:

Пользователь нажал кнопку

→ Приложение открывает нативный email клиент (Mail/Gmail)

→ Письмо УЖЕ предзаполнено:

- Кому: support@oister.app (пример)
- Тема: зависит от кнопки
- Начало текста: готовый шаблон

→ Пользователь дописывает свой текст

→ Нажимает "Отправить" в почтовом приложении

→ Письмо приходит нам на support@oister.app

Шаблоны писем (на русском!):

1. "Написать в поддержку"

Кому: support@oister.app

Тема: Обращение в поддержку

Здравствуйте!

У меня есть вопрос/проблема:

[Пользователь пишет здесь]

2. "Сообщить о проблеме"

Кому: support@oister.app

Тема: Сообщение о проблеме

Здравствуйте!

У меня возникла проблема:

[Пользователь описывает проблему]

Что произошло:

[Пользователь пишет]

Когда: [Пользователь пишет]

3. "Изменить имя"

Кому: support@oister.app

Тема: Изменить моё имя

Здравствуйте!

Прошу изменить моё имя в приложении.

Текущее имя: Иван Петров

Желаемое имя: [Пользователь пишет новое имя]

Таблица использования:

Где	Кнопка	Вызов функции
Menu_Screen	"Написать в поддержку"	openEmailSupport('general_support')
Menu_Screen	"Сообщить о проблеме"	openEmailSupport('report_problem')
Settings_Screen	"Изменить имя"	openEmailSupport('change_name')

Важно:

- ✓ Используй НАТИВНЫЙ email клиент (НЕ создавай свой экран)
- ✓ Весь текст НА РУССКОМ языке
- ✓ Письма отправляются на нашу корпоративную почту support@oister.app (пример)
- ✓ НЕТ API - просто открываешь email клиент
- ✓ Пользователь сам нажимает "Отправить" в Mail/Gmail

settings_screen

UI элементы:

- Background: Светло-серый фон
- Header:
 - Close Button: Крестик X (чёрный, left top)
 - Title: "Настройки" (чёрный, bold, centered)
- Section 1 - ИМЕНА В ГОЛОСОВАНИИ (серый текст, small, bold):
 - Button: "Все" (светло-синий фон, иконка людей слева, стрелка вправо →)

- Section 2 - УВЕДОМЛЕНИЯ (серый текст, small, bold):
 - Button: "Настроить уведомления" (светло-синий фон, иконка колокольчика 🔔 слева, стрелка вправо →)
- Section 3 - ОБЩИЕ (серый текст, small, bold):
 - Button 2: "Сменить язык" (светло-синий фон, иконка 🗣️ слева, стрелка вправо →)
 - Button 3: "Заблокированные" (светло-синий фон, иконка 🚫 слева, стрелка вправо →)
 - Button 4: "Скрытые пользователи" (светло-синий фон, иконка 👤 слева, стрелка вправо →)
 - Button 5: "Изменить имя" (светло-синий фон, иконка 👤 слева, стрелка вправо →)
 - Button 6: "Восстановить покупки" (светло-синий фон, иконка 🔄 слева, стрелка вправо →)
 - Button 7: "Выйти из аккаунта" (светло-синий фон, иконка 🚪 слева, стрелка вправо →)
- Delete Account Button: "🗑️ Удалить аккаунт" (красная кнопка, белый текст, large, rounded, bottom)
- Bottom Navigation Bar: 5 табов (Главная, Поиск, Лайки, Профиль, Ещё)
 - Таб "Ещё" активен (синий)

Валидация:

Нет валидации - это навигационный экран (только кнопки для открытия других экранов)

Ошибки:

- Ошибка загрузки настроек → Toast: "Не удалось загрузить настройки. Попробуйте позже"

Состояния:

"ПРЕДЛОЖИТЬ ВОПРОС" кнопки нету мы ее удалили она не нужна вообще.

Default (по умолчанию):

- Показывается экран с 3 секциями кнопок
- Все кнопки кликабельны
- Кнопка "Удалить аккаунт" внизу (красная, выделяется)
- Таб "Ещё" активен в нижней навигации (синий)

Close Button Clicked (крестик):

- Пользователь нажал крестик ✕
- Экран закрывается с анимацией (справа налево, обратная анимация открытия)
- Возврат на предыдущий экран (откуда открыли Settings)

"Все" Clicked (Имена в голосовании):

- Открывается Settings_Poll_Screen (экран с выбором: Все/Девочки/Мальчики/Небинарный)
- Анимация: слева направо (стандартный переход на новый экран)
- После выбора и возврата → текст кнопки обновляется на выбранное значение
 - Было: "Все"
 - Стало: "Девочки" (если выбрали "Девочки")

"Настроить уведомления" Clicked:

- Открывается Notifications_Settings_Screen (экран настройки уведомлений)
- Анимация: слева направо
- На том экране пользователь настраивает типы уведомлений (подробно опишем в следующем экране)

"Предложить вопрос" Clicked:

- Открывается нативный почтовый клиент (Apple Mail / Gmail)
- Письмо предзаполнено:
- Code

Кому: support@oister.app

Тема: Предложение нового вопроса для опроса

Здравствуй!

Я хочу предложить новый вопрос для опросов

Предлагаемый вопрос:

[Напишите вопрос здесь]

Описание (почему этот вопрос интересен):

[Опционально]

⚠ **ВАЖНО:** Пожалуйста, не предлагайте пошлые, оскорбительные или неприемлемые вопросы. Такие предложения будут проигнорированы.

Примечание: Мы рассматриваем все предложения, но решение о добавлении вопроса принимается командой. Вы не получите уведомление о статусе вашего предложения.

- Пользователь дописывает вопрос и отправляет
- Письмо приходит на support@oister.app
- Нет автоматической обработки - команда вручную рассматривает предложения
- Пользователь не получает уведомление об одобрении/отклонении

"Сменить язык" Clicked:

- Открывается Language_Settings_Screen (отдельный экран выбора языка)
- Анимация: слева направо
- На экране:
 - Список языков (radio buttons):
 - Кнопка "Сохранить" внизу
- Выбрал язык → нажал "Сохранить" → язык меняется мгновенно (без перезапуска)
- Сохраняется локально на устройстве (НЕ синхронизируется между устройствами)
- Дизайн экрана на усмотрение разработчика (в стиле приложения)

"Заблокированные" Clicked:

- Открывается Blocked_Users_Screen (список заблокированных пользователей)
- Анимация: слева направо
- Дизайн экрана: как Search_Screen (список пользователей с карточками)
- Каждая карточка:
 - Аватар (или иконка, если нет фото)
 - Имя и фамилия
 - Username (@username)
 - Класс
 - Кнопка "Разблокировать" (красная)
- При нажатии "Разблокировать":
 - Если подтвердил → пользователь удаляется из списка заблокированных
- Empty State: "Нет заблокированных пользователей" (если список пустой)

Что происходит при блокировке (логика):

- Заблокированный НЕ видит мой профиль (при поиске/переходе показывается "Профиль недоступен")
- Заблокированный МОЖЕТ голосовать за меня в опросах (блокировка не влияет на голосование)
- Заблокированный НЕ может отправить запрос в друзья
- Я ВИЖУ заблокированного в опросах/поиске (блокировка односторонняя - только он не видит меня)

"Скрытые пользователи" Clicked:

- Открывается Hidden_Users_Screen (список скрытых пользователей)
- Анимация: слева направо
- Дизайн: как Blocked_Users_Screen (список карточек)
- Кнопка на карточке: "Показать" (синяя)
- Empty State: "Нет скрытых пользователей"

Разница между "Заблокированные" и "Скрытые":

- Заблокированные: они не видят мой профиль, не могут добавить в друзья
- Скрытые: их профиль скрывается из моего поиска, но голосование работает (они могут голосовать за меня, я за них)
- Скрытие = лёгкая форма блокировки (просто не хочу видеть этого человека в поиске, но не хочу полностью блокировать)

"Изменить имя" Clicked:

- Показывается модальное окно (Change_Name_Modal)

- Модальное окно содержит:
 - Title: "Изменить имя"
 - Warning Text: "⚠ Вы можете изменить имя только один раз в 3 месяца" (оранжевый/красный текст)
 - Input Field 1: "Имя" (текущее имя предзаполнено)
 - Input Field 2: "Фамилия" (текущая фамилия предзаполнена)
 - Character Counter: "0/50" (под каждым полем)
 - Button: "Сохранить" (синяя, внизу)
 - Button: "Отмена" (серая, внизу)
- Пользователь вводит новое имя
- Валидация:
 - Минимум 2 символа
 - Максимум 50 символов
 - Только буквы (кириллица/латиница) и пробелы
- Нажал "Сохранить":
 - Отправляется запрос на сервер
 - Проверка: прошло ли 3 месяца с последнего изменения?
 - Если ДА → имя изменяется мгновенно, модалка закрывается, Toast: "✅ Имя изменено!"
 - Если НЕТ → показывается ошибка: "❌ Вы сможете изменить имя снова через [осталось дней] дней"
- Нет модерации - имя меняется автоматически (без проверки админом)
- Дизайн модального окна на усмотрение разработчика

"Восстановить покупки" Clicked:

- Показывается Loading: "Проверяем покупки..." (спиннер/индикатор загрузки)
- Отправляется запрос к App Store / Google Play (системный API)
- Результаты:
 - Успех (нашли покупку Подписки "Premium"):
 - Loading исчезает
 - Toast: "✅ Покупки восстановлены! Подписка "Premium" активирован"
 - Подписка "Premium" активируется в приложении
 - Нет покупок:
 - Loading исчезает
 - Toast: "i Покупок не найдено"
 - Ошибка (нет интернета, проблема с App Store):
 - Loading исчезает
 - Toast: "❌ Не удалось восстановить покупки. Попробуйте позже"

Когда это нужно:

- Переустановил приложение
- Сменил телефон (залогинился на новом устройстве)
- Удалил и скачал приложение заново
- Подписки "Premium" не отображаются (баг)

"Выйти из аккаунта" Clicked:

- Показывается Alert/Dialog подтверждение:
 - Title: "Выйти из аккаунта?"
 - Message: "Вы уверены, что хотите выйти?"
 - Кнопка: "Выйти" (красный текст)
 - Кнопка: "Отмена" (серый текст)
- Если нажал "Отмена" → alert закрывается, ничего не происходит
- Если нажал "Выйти":
 - Alert закрывается
 - Token НЕ удаляется (остаётся на устройстве для быстрого входа)
 - Данные НЕ удаляются (кэш, настройки остаются)
 - Пользователь перенаправляется на Login_Screen (экран входа)
 - На Login_Screen может быстро войти (нажать кнопку "Войти как [Имя]" - автологин)
- Мультиаккаунт:
 - Выход на одном устройстве НЕ влияет на другие устройства
 - Каждое устройство независимо (можно быть залогиненным на телефоне и планшете одновременно)

"Удалить аккаунт" Button Clicked (красная кнопка):

- Открывается Delete_Account_Screen (который мы описали ранее)

- Анимация: слева направо

Важно:

Кнопка написать отзыв ведет в стандартное app store, google play для того чтобы он мог оставить отзыв

- ⚠ Navigation screen: экран-меню для доступа к настройкам
- ⚠ Opened from "Ещё" tab: открывается из нижней навигации (таб "Ещё")
- ⚠ Animation left-to-right: анимация открытия слева направо (стандартный push)
- ⚠ Close with X: закрывается крестиком (обратная анимация справа налево)
- ⚠ Poll names filter: "Имена в голосовании" влияет только на мои созданные опросы (кого показывать в вариантах)
- ⚠ Block vs Hide: "Заблокированные" = они не видят меня, "Скрытые" = я не вижу их в поиске
- ⚠ Name change limit: имя можно менять раз в 3 месяца, без модерации
- ⚠ Email for questions: предложения вопросов через email, без автоматической обработки
- ⚠ Language local: язык сохраняется локально (НЕ синхронизируется между устройствами)
- ⚠ Restore purchases: восстановление только Подписки "Premium" (других покупок нет)
- ⚠ Logout keeps data: выход не удаляет данные/token (быстрый повторный вход)
- ⚠ Independent devices: выход на одном устройстве не влияет на другие

notifications_settings_screen

Валидация:

Нет валидации - это настройки с toggles (вкл/выкл)

Ошибки:

- Ошибка сохранения настроек → Toast: "Не удалось сохранить настройки. Попробуйте позже"

Состояния:

Default (по умолчанию):

- Показывается экран с 7 типами уведомлений (6 типов + 1 мастер-toggle)
- Все toggles загружаются с сервера (текущие настройки пользователя)
- "Все уведомления" внизу списка

Master Toggle Logic (мастер-переключатель "Все уведомления"):

Мастер-toggle OFF:

- Все 6 типов уведомлений автоматически становятся OFF
- Все 6 toggles становятся неактивными (disabled, серые)
- Пользователь НЕ может изменить отдельные типы (пока мастер-toggle OFF)
- На сервер отправляется: allNotifications: false

Мастер-toggle ON:

- Все 6 toggles становятся активными (можно менять)
- Каждый тип можно настроить независимо
- На сервер отправляется: allNotifications: true

Автообновление мастер-toggle:

- Если пользователь вручную выключил все 6 типов → мастер-toggle автоматически становится OFF
- Если пользователь включил хотя бы один тип → мастер-toggle автоматически становится ON

Toggle Switched (переключение любого типа):

- Пользователь нажал на toggle
- Toggle меняет состояние с анимацией: OFF ↔ ON
- Настройка сохраняется автоматически (запрос на сервер)
- НЕТ кнопки "Сохранить" - изменения мгновенные

Типы уведомлений (что означает каждый):

1. "Новые голоса":
 - ON = уведомления когда кто-то проголосовал за меня
 - Пример: "Маша проголосовала за вас в опросе 'Кто самый умный' 🗳️"
2. "Лайки":
 - ON = уведомления когда кто-то лайкнул профиль
 - Пример: "Петя лайкнул ваш профиль ❤️"
3. "Запросы в друзья":
 - ON = уведомления о новых запросах в друзья
 - Пример: "Катя хочет добавить вас в друзья 👤"
4. "Результаты опросов":
 - ON = уведомления когда опрос завершился
 - Пример: "Опрос 'Кто самый умный' завершён! Смотрите результаты 📊"
5. "Подписки "Premium"":
 - ON = уведомления о подписке
 - Примеры:
 - "Ваша подписка Подписка "Premium" истекает через 3 дня 🕒"
 - "Подписка "Premium" успешно продлён! 🎉"
6. "Новости приложения":
 - ON = уведомления о новостях/обновлениях
 - Примеры:
 - "Новое обновление OISTER! 🚀"
 - "Специальное предложение: скидка 50% на Подписку "Premium" 💰"
7. "Все уведомления" (master-toggle):
 - OFF = отключить ВСЕ уведомления (все 6 типов неактивны)
 - ON = можно настраивать каждый тип отдельно

Close Button Clicked (крестик):

- Экран закрывается с анимацией (справа налево)
- Возврат на Settings_Screen
- Все изменения уже сохранены

Важно:

- ⚠️ 7 items total: 6 типов уведомлений + 1 мастер-переключатель "Все уведомления"
- ⚠️ Master toggle at bottom: "Все уведомления" внизу списка (последний элемент)
- ⚠️ Master toggle bold: текст "Все уведомления" жирный (bold), чтобы выделялся
- ⚠️ Auto-save: настройки сохраняются автоматически (нет кнопки "Сохранить")
- ⚠️ Disable when master OFF: если мастер-toggle OFF → все 6 типов неактивны (серые, disabled)
- ⚠️ Auto-update master: если выключить все 6 типов вручную → мастер-toggle автоматически OFF
- ⚠️ Legal compliance: соответствует GDPR, Apple, Google (можно выключить всё одной кнопкой)
- ⚠️ Simple for developer: простая логика (один IF для мастер-toggle)

premium_screen

Валидация:

Нет валидации - это экран презентации подписок (только кнопки покупки)

Ошибки:

- Ошибка покупки → Alert от App Store/Google Play: "Не удалось завершить покупку"
- Ошибка восстановления → Toast: "Не удалось восстановить покупки. Попробуйте позже"
- Нет подключения к интернету → Alert: "Для покупки нужен интернет"

Состояния:

Default (по умолчанию):

- Показывается экран с 2 карточками подписок: Pro и Max
- Обе карточки доступны для покупки
- Если пользователь УЖЕ имеет подписку → показывается индикатор "Активна" на соответствующей карточке

Back Button Clicked (стрелка назад):

- Закрывается экран с анимацией (справа налево)
- Возврат на предыдущий экран (откуда открыли Premium_Screen)

"Управлять подпиской" Link Clicked:

- Открывается системная страница управления подписками:
 - iOS: перенаправляет в App Store → Subscriptions
 - Android: перенаправляет в Google Play → Subscriptions
- Пользователь может:
 - Отменить подписку
 - Изменить тариф (с Pro на Max или наоборот)
 - Посмотреть историю платежей

"Восстановить покупки" Link Clicked:

- Показывается Loading: "Проверяем покупки..." (спиннер)
- Отправляется запрос к App Store / Google Play (системный API восстановления)
- Результаты:
 - Успех (нашли подписку):
 - Loading исчезает
 - Toast: "✅ Покупки восстановлены! Premium активирован"
 - Подписка активируется в приложении
 - Нет покупок:
 - Loading исчезает
 - Toast: "ℹ Покупок не найдено"
 - Ошибка:
 - Loading исчезает
 - Toast: "❌ Не удалось восстановить покупки. Попробуйте позже"

Premium Pro (Карточка 1) ★

Период подписки:

- Еженедельная подписка (списание каждую неделю)
- Цена: 7.99 \$ в неделю

Функции Premium Pro:

1. "2 раза - раскройте полные имена тех кто выбрал вас в голосовании" ☰

Как работает:

- С Premium Pro можно 2 раза в неделю узнать полное имя
- Нажал на likes_result_screen на кнопку "Узнать кто проголосовал за тебя там появляется → показывается кнопка "👁 Узнать полное имя "
- Нажал → показывается: "От Маши Петровой, 9 класс"
- Лимит: 2 раза в неделю с момента подписки
- После 2 раз → показывается: "Лимит исчерпан. Оформите Premium Max для безлимита или ждите обновления"

2. "Звёздочки умножаются на два" ★

Как работает:

- За прохождение раунда голосования в приложении начисляются звёздочки (внутренняя валюта)
- Источники звёздочек:
 - Участие в голосованиях
 - Создание опросов
 - Ежедневная активность
- С Premium Pro все звёздочки ×2:
 -
- Работает всё время пока подписка активна

3. "2 раза - узнайте первую букву имени того, кто голосовал за тебя" ☰

Как работает:

- Это дополнительная функция (не альтернатива полному имени)
- Отдельный лимит: 2 раза в неделю (не связан с лимитом на полные имена)
- Нажал → вместо "От девочки с 9 класса" показывается "От М. (девочки) с 9 класса"
- **Зачем нужно? **: если не хочешь тратить лимит на полное имя, но хочешь узнать хоть что-то
- Лимит: 2 раза в неделю

4. "Значок подтверждения в профиле на неделю" 🏆

Как работает:

- Рядом с именем появляется галочка ✓ (как в Instagram)
- Показывается:
 - В твоём профиле (Profile_Screen)
 - В списке друзей (Friends_Screen)
 - В Activity_Screen (когда за тебя голосуют)
 - В опросах (когда твоё имя в варианте ответа)
- Длительность: на неделю (пока подписка Pro активна)
- Если отменить подписку → галочка исчезает

Кнопки покупки:

"7.99 долларов" (синяя кнопка):

- Показывает цену подписки
- Нажатие → открывает платёжное окно App Store / Google Play

🏆 "Получить Pro" (красная кнопка):

- Основная кнопка покупки (CTA - Call To Action)
- Нажатие → открывает платёжное окно App Store / Google Play
- Обе кнопки ведут на одно и то же (покупка Premium Pro)

Процесс покупки Premium Pro:

1. Пользователь нажал "Получить Pro"
2. Открывается системное окно App Store / Google Play
3. Показывается:
 - Название: "Premium Pro"
 - Цена: 7.99 \$ / неделю
 - Описание: "Автоматическое продление. Отменить можно в любой момент"
 - Кнопка: "Подписаться"
4. Пользователь подтверждает (Face ID / пароль)
5. Если успешно:
 - Платёжное окно закрывается
 - Toast: "✅ Premium Pro активирован!"
 - Возврат на предыдущий экран
 - Все функции Premium Pro активируются
6. Если ошибка:
 - Alert: "Не удалось завершить покупку"

Premium Max (Карточка 2) 🏆

Период подписки:

- Ежемесячная подписка (списание раз в месяц)
- Цена: 27.99 \$ в месяц

Функции Premium Max:

1. "Неограниченно - раскройте полные имена тех кто выбрал вас в голосовании" 💬

Отличие от Pro:

- БЕЗ ЛИМИТА (можно раскрывать имена сколько угодно раз)
- То же что в Pro (×2 звёздочки)

3. "Неограниченно - узнайте первую букву имени того, кто голосовал за тебя" 💬

Отличие от Pro:

- БЕЗ ЛИМИТА (можно узнавать первую букву сколько угодно раз)

4. "Значок подтверждения в профиле на месяц" 🏆

Отличие от Pro:

- Галочка ✓ показывается на месяц (пока подписка Max активна)
- То же расположение (профиль, друзья, активность, опросы)

Кнопки покупки:

"27.99 долларов" (синяя кнопка):

- Показывает цену
- Нажатие → платёжное окно App Store / Google Play

🏆 "Получить Max" (красная кнопка):

- СТА кнопка покупки Premium Max
- Нажатие → платёжное окно App Store / Google Play

Процесс покупки Premium Max:

- Аналогичен Premium Pro (системное окно → подтверждение → активация)

Сравнение Premium Pro vs Max:

Функция	Premium Pro	Premium Max
Цена	7.99 \$ / неделю	27.99 \$ / месяц
Итого в месяц	~32 \$	27.99 \$
Раскрыть имена	2 раза в неделю	∞ Неограниченно
Первая буква	2 раза в неделю	∞ Неограниченно
Звёздочки ×2	✓ Да	✓ Да
Значок ✓	На неделю	На месяц
Лучший выбор	Попробовать	Для активных

Рекомендация для пользователей:

- Premium Max выгоднее (27.99 вместо 32 \$/месяц) И даёт безлимит
- Premium Pro = для тех, кто хочет попробовать на неделю

Footer Links (внизу экрана):

"Условия использования" Link:

- Кликабельная ссылка (белый текст, подчёркнутый)
- Нажатие → открывает веб-страницу в браузере (Safari / Chrome)
- URL: <https://oister.app/terms> пример
- Страница содержит:
 - Правила использования приложения
 - Условия подписки (автопродление, отмена, возврат)
 - Ответственность пользователя

"Политика конфиденциальности" Link:

- Кликабельная ссылка (белый текст, подчёркнутый)
- Нажатие → открывает веб-страницу в браузере
- URL: <https://oister.app/privacy> пример
- Страница содержит:
 - Какие данные собираются
 - Как используются данные
 - Права пользователя (GDPR)
 - Контакты для запросов

Важно:

- ⚠ Two subscription tiers: 2 тарифа подписки (Pro и Max)
- ⚠ Pro = weekly: Pro списывается еженедельно (7.99 \$ / неделю)
- ⚠ Max = monthly: Max списывается ежемесячно (27.99 \$ / месяц)
- ⚠ Max is cheaper: Max выгоднее (27.99 вместо ~32 \$/месяц у Pro)
- ⚠ Limits for Pro: Pro имеет лимиты (2 раза в неделю на раскрытие имён)
- ⚠ Unlimited for Max: Max без лимитов (безлимитное раскрытие имён)
- ⚠ Stars multiplier: звёздочки умножаются на 2 (в обеих подписках)
- ⚠ Verified badge: синяя галочка ✓ в профиле (как в Instagram)
- ⚠ Badge locations: галочка показывается в профиле, друзьях, активности, опросах
- ⚠ Weekly badge for Pro: галочка на неделю (для Pro)

- ⚠ Monthly badge for Max: галочка на месяц (для Max)
- ⚠ Both buttons = purchase: синяя и красная кнопки ведут на покупку
- ⚠ Native payment: оплата через App Store / Google Play (не своя система)
- ⚠ Auto-renewal: подписки автоматически продлеваются
- ⚠ Can cancel anytime: можно отменить в любой момент (в настройках App Store/Google Play)
- ⚠ Restore purchases: кнопка восстановления покупок (если переустановил приложение)
- ⚠ Legal links: ссылки на условия использования и политику конфиденциальности (обязательно по закону)

premium_result_screen

Валидация:

- Нельзя раскрыть имя/букву если лимит исчерпан (кнопки становятся неактивными)
- Нельзя открыть экран если нет Premium подписки (показывается Premium_Screen)

Ошибки:

- Нет Premium подписки → открывается Premium_Screen
- Лимит исчерпан → Toast: "Лимит исчерпан. Оформите Premium Max для безлимита"
- Ошибка загрузки данных → Toast: "Не удалось загрузить результаты. Попробуйте позже"

Состояния:

Попадание на экран (Access Control):

Если НЕТ Premium подписки:

- Пользователь нажал "Узнать кто проголосовал за тебя" на premium_screen
- Проверка: есть ли активная подписка Premium Pro или Premium Max?
- НЕТ подписки → перенаправление на Premium_Screen (экран покупки)
- ЕСТЬ подписка → открывается Premium_Result_Screen

Если ЕСТЬ Premium подписка:

- Открывается текущий экран (Premium_Result_Screen)
- Загружаются данные опроса и участники

Default (по умолчанию с Premium Pro):

Загрузка экрана:

- Показывается вопрос опроса: "😊 Кто самый красивый ?"
 - Эмодзи из опроса (создатель выбрал при создании)
 - Вопрос динамически подставляется (название опроса)
- Показывается список участников (те кто были в опросе вместе со мной)
 - 4 карточки
 - Показывается счётчик лимитов (для Premium Pro):
- Code

У вас осталось:

2 раскрытия полного имени

1 раскрытие первой буквы имени

- Обе кнопки активны (белые, кликабельные)

Default (по умолчанию с Premium Max):

Отличия:

- Счётчик показывает:

У вас неограниченные раскрытия ∞

- ИЛИ:

У вас осталось:

∞ раскрытий полного имени

∞ раскрытий первой буквы имени

- Обе кнопки активны (лимитов нет)

Кнопка "Узнать первую букву имени" Clicked:

Процесс:

1. Пользователь нажал кнопку "Узнать первую букву имени"
2. Проверка лимита:
 - Если лимит > 0 → продолжаем
 - Если лимит = 0 → показывается Toast: "Лимит исчерпан. Оформите Premium Max"
3. При нажатии "Узнать первую букву" появляется первая буква имени класс и аватар
- 4.
5. Кнопка "Узнать полное имя" Clicked:

Процесс:

1. Пользователь нажал кнопку "Узнать полное имя"
2. Проверка лимита:
 - Если лимит > 0 → продолжаем
 - Если лимит = 0 → Toast: "Лимит исчерпан"
3. Показывается подсказка: "Нажмите на карточку того, кого хотите узнать"
4. На кнопке после раскрытия:
 - Имя: "Александр Иванов" (полное имя + фамилия)
 - Класс: "10 класс"
 - аватар кликабельный
5. Лимит уменьшается:
 - Счётчик обновляется: "2 раскрытия → 1 раскрытие"
 - Если лимит стал 0 → кнопка "Узнать полное имя" становится неактивной (серая)

Лимит исчерпан (все раскрытия использованы):

Что показывается:

Лимит исчерпан

Оформите Premium Max для безлимита

- Обе кнопки становятся неактивными (серые, не кликабельны)
- При попытке нажать → Toast: "Лимит исчерпан. Оформите Premium Max"
- Premium Max кликабельная ведет на premium_screen
- Можно нажать на уже раскрытые карточки → открыть профиль

Когда обновляется лимит (для Premium Pro):

- Пользователь получает уведомление: "Ваши лимиты раскрытия обновлены! 🎉"

Смешанное использование:

Сценарий:

- Раскрыл первую букву одного человека ("А...")
- Раскрыл полное имя другого человека ("Александр Иванов")
- Это разрешено (можно смешивать методы)
- Лимиты независимы:
 - Лимит на буквы: 2 → 1
 - Лимит на полные имена: 2 → 1

Close Button Clicked (крестик):

- Пользователь нажал крестик X
- Экран закрывается с анимацией (справа налево или сверху вниз)
- Возврат на Likes_Result_Screen (откуда пришли)
- Все раскрытые имена сохраняются (при повторном открытии экрана будут видны)

Card Clicked (нажатие на карточку):

Если имя НЕ раскрыто:

- Нажатие на карточку → раскрывается имя (в зависимости от выбранной кнопки)

Если имя раскрыто (полное):

- Нажатие на карточку → открывается Profile_Screen этого человека
- Можно посмотреть полный профиль, добавить в друзья, etc.

Нажал "Узнать полное имя"

6. КНОПКА ИСЧЕЗАЕТ

7. ВМЕСТО КНОПКИ появляются карточки:

- Маша Петрова, 9 класс

- Петя Иванов, 10 класс
- (те кто голосовал за меня)

8. Нажал "Узнать первую букву"

9. ВТОРАЯ КНОПКА ИСЧЕЗАЕТ

10. ВМЕСТО НЕЁ появляются карточки:

- А... , 9 класс
- П..., 10 класс
- (те же люди, но только первая буква)

- ⚠ Premium required: экран доступен ТОЛЬКО пользователям с Premium Pro или Premium Max
- ⚠ Without Premium: перенаправление на Premium_Screen
- ⚠ Poll participants: показываются участники опроса (кто был в вариантах ответа вместе со мной)
- ⚠ Hidden by default: все данные скрыты до раскрытия (имя, класс, аватар)
- ⚠ Two reveal methods: первая буква ИЛИ полное имя (разные лимиты)
- ⚠ First letter format: "А..." (только первая буква имени, БЕЗ фамилии)
- ⚠ Full name format: "Арман Никитенко" (имя + фамилия + аватар + класс)
- ⚠ Weekly limits for Pro: 2 полных имени + 2 первые буквы в неделю (обновляется каждый понедельник)
- ⚠ Unlimited for Max: Premium Max = безлимитное раскрытие
- ⚠ Limits per person: лимиты на всё приложение (не на каждый опрос)
- ⚠ Live counter: счётчик обновляется мгновенно после раскрытия
- ⚠ Buttons disabled: кнопки становятся серыми когда лимит исчерпан
- ⚠ Animation: плавная анимация раскрытия (0.3 секунды)
- ⚠ Profile clickable: можно открыть профиль после раскрытия полного имени
- ⚠ Data persists: раскрытые имена сохраняются (не скрываются при повторном открытии)
- ⚠ Works for all polls: функция работает для всех опросов в приложении

ЗАКАЗЧИК:

- ФИО: Yessen Amanay Bekezhanyuly
- Страна: Kazakhstan
- Адрес: Astana City, 15, kv.97, ulitsa Aiteke Bi
- Email: support@flyprox.com

ИСПОЛНИТЕЛЬ:

- ФИО: Kudr8vceva Antonina Olegovna
- Страна: Russian Federation
- Адрес: ul. Dzerzhinskogo 53, Stavropol, Stavropol Krai, 355000, Russian Federation
- Email: atrkk2024@gmail.com